PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> ..venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python --version
Python 3.14.2
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "import sys; print(sys.executable)"

C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Scripts\python.exe

Environment setup successful

Excellent! Everything is correct.
We have confirmed:

- Python 3.14.2 is installed.
- Virtual environment is activated.
- VS Code is using the Python interpreter inside `.venv`.
- No global Python packages will be required for this project.

Now we'll configure the project properly.

Step 3 — Create `pyproject.toml`

Before installing dependencies, we'll create a file that manages our project's metadata and dependencies.

What is `pyproject.toml` ?

Think of it as the configuration file for our Python project. It tells Python tooling:

- What our project is called.
- Which Python versions it supports.
- Which libraries it needs.
- Which libraries are used for development and testing.
- How the project should be packaged.

We'll use the current official MCP Python SDK v2, which supports Python 3.11 and newer.

MCP Python SDK GitHub

3.1 Create the file

In VS Code Explorer:

1. Right-click the `google-news-mcp` root folder.
2. Select **New File**.
3. Name it exactly:

```text
pyproject.toml
```

3.2 Add this configuration

toml

```toml
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"


[project]
name = "google-news-mcp"
version = "0.1.0"
description = "An open-source MCP server for Google News."
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "mcp[cli]>=2.0,<3.0",
    "httpx>=0.28,<1.0",
    "feedparser>=6.0,<7.0",
    "pydantic>=2.12,<3.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-asyncio>=0.24",
    "respx>=0.22",
    "ruff>=0.14",
]

[tool.hatch.build.targets.wheel]
packages = ["src/google_news_mcp"]

[tool.pytest.ini_options]
testpaths = ["tests"]
asyncio_mode = "auto"

[tool.ruff]
line-length = 100
target-version = "py314"
```

3.3 Understand the dependencies

Package	Why we're installing it
`mcp[cli]`	Creates MCP servers and provides development commands
`httpx`	Communicates with Google News over HTTP

`feedparser`	Converts RSS/XML feeds into Python objects
`pydantic`	Validates our input and output data
`pytest`	Runs automated tests
`pytest-asyncio`	Tests asynchronous Python functions
`respx`	Mocks HTTP requests during tests
`ruff`	Checks and formats our Python code
`hatchling`	Builds our Python package

We are deliberately not installing the Gemini SDK yet. That belongs to our separate AI demonstration client, not the core MCP server.

3.4 Save the file

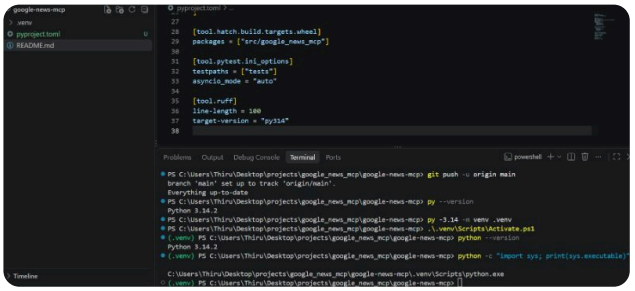
Press:

```text
Ctrl + S
```

Your project should now contain:

- `google-news-mcp/`
  - `.venv/` Virtual environment
  - `README.md`
  - `pyproject.toml` **New**
  - `.git/`

Stop here. Create and save `pyproject.toml`, then share a screenshot of your VS Code Explorer.
Next, we'll install the dependencies into `.venv` and verify that the MCP SDK works on your Python 3.14 installation.



verify

Your setup looks correct so far!
I checked your screenshot. Everything visible is in the right place.

Item	Status
`.venv`	Created
Python 3.14.2	Verified
Virtual environment	Active
`README.md`	Present
`pyproject.toml`	Created

Your `pyproject.toml` visible configuration also looks correct. The `mcp[cli]` dependency is appropriate for the official MCP SDK v2. MCP Python SDK GitHub

One important thing before installing dependencies

We need a `.gitignore` file. This prevents GitHub from receiving your virtual environment, private API keys and temporary Python files.

Step 1 — Create `.gitignore`

In VS Code, create a new file named `.gitignore` in the project root.
Paste:

gitignore

```
# Python
__pycache__/
*.py[cod]
*.egg-info/

# Virtual environment
.venv/
venv/

# Environment variables and secrets
.env
.env.*

# Testing
.pytest_cache/
.coverage
htmlcov/

# Code quality
.ruff_cache/

# Build artifacts
build/
dist/

# IDE
.vscode/
```

Save it.

Step 2 — Validate `pyproject.toml`

Run:

```powershell
python -c "import tomllib; tomllib.load(open('pyproject.toml', 'rb')); print('pyproject.toml is valid')"
```

Expected:

```text
pyproject.toml is valid
```

Step 3 — Check Git

```powershell
git status
```

We should see `pyproject.toml` and `.gitignore` as untracked files, but **not** `.venv`.
Send me the output of these two checks.
Next, we'll install the dependencies and verify the official MCP SDK installation before creating our application code.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
(use "git add ..." to include in what will be committed)
.gitignore
pyproject.toml

nothing added to commit but untracked files present (use "git add" to track)
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>

Perfect! **Git configuration verified**
Your output confirms that:

- `.gitignore` is present.
- `pyproject.toml` is present.
- `.venv` is correctly excluded from Git.
- Your local repository is still connected to GitHub.

We're ready to install dependencies.

Step 4 — Install project dependencies

First, let's install the dependencies defined in `pyproject.toml`.
Run:

```powershell
python -m pip install -e ".[dev]"
```

What does this command mean?

- `pip install`: Installs Python packages.
- `-e`: Editable installation. Changes to our source code won't require reinstalling the project.
- `.`: Refers to the current project directory.
- `[dev]`: Also installs our development dependencies, such as Pytest and Ruff.

It will install:

Package	Purpose
MCP	Build our MCP server
HTTPX	Communicate with Google News
Feedparser	Parse RSS feeds
Pydantic	Validate data
Pytest	Test our code
RESPX	Mock HTTP requests
Ruff	Code quality checks
Hatchling	Build the package

Step 5 — Verify installation

After installation, run:

```powershell
python -m pip list
```

Then:

```powershell
python -m pip check
```

Expected final output:

```text
No broken requirements found.
```

If installation fails, don't randomly install or downgrade packages. Share the error with me first.
Your next task: Run the installation command and share the output. Then we'll create our actual Python package structure.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -m pip list
Package Version Editable project location

annotated-doc 0.0.6
annotated-types 0.8.0
anyio 4.16.0
attrs 26.1.0
certifi 2026.10.23
cffi 2.1.1
click 8.5.1
colorama 0.4.6
cryptography 50.0.2
feedparser 6.0.14
feedparser-sgmllib 2.1.0
google-news-mcp 0.1.0 C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp
h11 0.16.0
httpcore 1.0.9
httpcore2 2.16.0
httpx 0.28.1
httpx2 2.16.0
idna 3.20
iniconfig 2.3.1
jsonschema 4.26.0
jsonschema-specifications 2025.9.1
markdown-it-py 4.3.0
mcp 2.3.0
mcp-types 2.3.0
mdurl 0.1.2
opentelemetry-api 1.45.2
packaging 26.3
pip 25.3
pluggy 1.7.0
pycparser 3.12

pydantic 2.14.3
pydantic_core 2.50.4
Pygments 2.21.0
PyJWT 2.15.1
pytest 9.2.0
pytest-asyncio 1.4.1.post1
python-dotenv 1.2.4
python-multipart 0.0.33
pywin32 312
referencing 0.37.0
respx 0.23.1
rich 15.0.0
rpds-py 2026.9.1
ruff 0.17.4
shellingham 1.5.4
sse-starlette 3.7.0
starlette 1.10.0
truststore 0.10.5
typer 0.29.0
typing_extensions 4.16.0
typing-inspection 0.4.5
uvicorn 0.54.0
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>
python -m pip check
No broken requirements found.
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>



All dependencies installed successfully!

Excellent. Your installation is clean.

I checked your output:

Check	Result
MCP SDK	`2.3.0` installed
HTTPX	`0.28.1` installed
Feedparser	`6.0.14` installed
Pydantic	`2.14.3` installed
Pytest	`9.2.0` installed
RESPX	`0.23.1` installed
Ruff	`0.17.4` installed

Dependency conflicts	None

One important observation: your installed MCP SDK is version 2.3.0, so we'll use its v2 API consistently rather than mixing examples from older tutorials.

Step 5 — Create the project structure

Now we're moving from environment setup to actual application architecture.
In VS Code Explorer, create the following folders and files.



What are these folders for?

Folder	Responsibility
`models`	Define article and search response structures
`providers`	Fetch and parse Google News RSS
`services`	Process, filter and normalize news
`tools`	Define functions exposed to AI through MCP
`tests`	Verify everything works
`server.py`	Start the MCP server

This separation makes the code easier to maintain and test.

Step 5.1 — Create the folders

You can create them through VS Code Explorer, or run these commands in PowerShell:

```powershell
New-Item -ItemType Directory -Force -Path `
    "src/google_news_mcp/models", `
    "src/google_news_mcp/tools", `
    "src/google_news_mcp/services", `
    "src/google_news_mcp/providers", `
    "tests", `
    ".github/workflows"
```

Then create the Python package initialization files:

```powershell
New-Item -ItemType File -Force -Path `
    "src/google_news_mcp/__init__.py", `
    "src/google_news_mcp/models/__init__.py", `
    "src/google_news_mcp/tools/__init__.py", `
    "src/google_news_mcp/services/__init__.py", `
    "src/google_news_mcp/providers/__init__.py"
```

Don't add any Python implementation code yet.

Step 5.2 — Verify the structure

Run:

```powershell
tree /F
```

You should see all the folders and initialization files.
Send me the output or a screenshot of your VS Code Explorer.
Next, we'll create our first actual Python module: `config.py`, and understand how configuration should be managed in a production-quality MCP server.

Pasted text(20261111-070528).txt 52.1 KB

Your folder structure has been created successfully! I checked the `tree /F` output.

Project structure verified

I can confirm these directories and files exist:

```text
src/
└── google_news_mcp/
    ├── __init__.py
```

```
    ├── models/
    │   └── __init__.py
    ├── providers/
    │   └── __init__.py
    ├── services/
    │   └── __init__.py
    └── tools/
        └── __init__.py

tests/
```

One small observation

Your `tree /F` output includes the entire `.venv` directory, which is why it produced 1,000+ lines. That's normal. We don't need to worry about it.
For a cleaner project-only view, you can run:

```powershell
tree /F /A
```

It will still show `.venv`, but we'll know which files belong to our application.

Step 6 — Create `config.py`

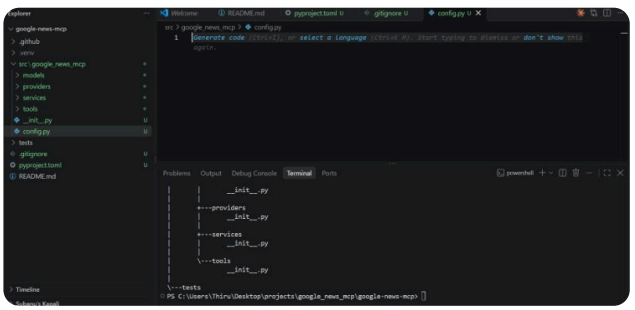
Now let's create our first actual application module.
In VS Code:

1. Open `src/google_news_mcp/`.
2. Right-click and select **New File**.
3. Name it `config.py`.

For now, leave it empty.
Why are we creating this file?
It will hold centralized configuration such as:

- Google News RSS base URL.
- Default language and region.
- HTTP request timeout.
- Maximum number of articles returned.

We will implement it in the next step and understand each setting.
Send me a screenshot once `config.py` is created.



Perfect! Your `config.py` has been created in the correct location. The folder structure looks good. Verified

Now let's write our first application code.

Step 6.1 — Implement `config.py`

Before writing code, understand what configuration means.

Imagine we want to change the default news language from English to Tamil.

Without centralized configuration, we might need to modify several Python files. Instead, we'll keep shared settings in one place.

Our configuration will manage:

- Default country: India (`IN`)
- Default language: English (`en`)
- Default result limit: 10 articles
- Maximum result limit: 50 articles
- HTTP timeout: 15 seconds
- Google News RSS endpoint

Paste this into `config.py`

python

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class Settings:
    """Application-wide configuration."""

    google_news_base_url: str = "https://news.google.com/rss"
    default_region: str = "IN"
    default_language: str = "en"
    default_max_results: int = 10
    maximum_results: int = 50
    request_timeout: float = 15.0


settings = Settings()
```

Save using `Ctrl + S` .

Step 6.2 — Understand the code

1. What is a dataclass?

`@dataclass` is a Python feature that automatically creates useful methods for a class that mainly stores data.
Instead of manually writing initialization code for six configuration values, Python generates it.

2. What does `frozen=True` mean?

It makes the configuration immutable.
For example, this will fail:

```python
settings.default_region = "US"
```

We don't want different parts of our application unexpectedly changing global settings.

3. What is `settings = Settings()` ?

This creates one shared configuration object that other modules can import:

```python
from google_news_mcp.config import settings

print(settings.default_region)
```

Expected output:

```text
IN
```

Step 6.3 — Verify it works

Run this in your activated VS Code terminal:

```powershell
python -c "from google_news_mcp.config import settings; print(settings)"
```

You should see the configuration values printed.

Stop after this verification and share the output.

Next, we'll build our first real integration: connecting Python to Google News RSS and retrieving actual news headlines.

PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from google_news_mcp.config import settings; print(settings)"
Traceback (most recent call last):
File "", line 1, in
from google_news_mcp.config import settings; print(settings)
^^^

No worries! This is a Python environment / package discovery issue. Your `config.py` is probably fine.

I noticed something important in your terminal screenshot: **your terminal currently does not show `(.venv)`**.

That means you're probably running the system-wide Python instead of the Python environment where we installed our project.

Let's fix this step by step.

Step 1 — Activate your virtual environment

Run:

```powershell
.\.venv\Scripts\Activate.ps1
```

Your terminal should now begin with:

```text
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>
```

Step 2 — Verify the Python interpreter

Run:

```powershell
python -c "import sys; print(sys.executable)"
```

Expected path:

```text
google-news-mcp\.venv\Scripts\python.exe
```

Step 3 — Reinstall the project in editable mode

Even though we installed it earlier, let's ensure the current environment knows about our `src` package.

```powershell
python -m pip install -e .
```

This command registers our project with the virtual environment so Python can discover `google_news_mcp`.

Step 4 — Test the import again

Run:

```powershell
python -c "from google_news_mcp.config import settings; print(settings)"
```

Expected output:

```text
Settings(
    google_news_base_url='https://news.google.com/rss',
    default_region='IN',
    default_language='en',
    default_max_results=10,
    maximum_results=50,
    request_timeout=15.0
)
```

Why did this happen?
Our project uses a `src/` layout:

```text
src/
└── google_news_mcp/
    ├── __init__.py
    └── config.py
```

Python doesn't automatically search every `src` directory. The editable installation tells Python where our package lives.
If it still fails, send me the outputs of Steps 2–4. We'll diagnose it before moving forward.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from google_news_mcp.config import settings; print(settings)"
Settings(google_news_base_url='https://news.google.com/rss', default_region='IN', default_language='en', default_max_results=10, maximum_results=50, request_timeout=15.0)
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>

Excellent! **Configuration module working**
We've fixed the Python package discovery issue. Your project is now correctly installed in editable mode, and Python can import our application modules.
Your output confirms that all six configuration values are loading correctly.

Step 7 — Create our News Article data model

Now we're going to create our first data model.
What is a data model?
Google News returns information in an RSS/XML format. We don't want different parts of our application working directly with that raw XML.

Instead, we'll convert every news article into a consistent Python object.
For example:

```text
Title: OpenAI announces new AI model
Publisher: Reuters
Published: 2026-11-11T...
URL: https://news.google.com/...
```

Every article will follow the same structure.

7.1 Create the file

In VS Code Explorer:
Navigate to:

```text
src/
└── google_news_mcp/
    └── models/
```

Create a new file:

```text
article.py
```

7.2 Add the following code

```python

from datetime import datetime

from pydantic import BaseModel, HttpUrl


class NewsArticle(BaseModel):
    """Represents a single news article returned by Google News."""

    title: str
    publisher: str
    published_at: datetime | None = None
    url: HttpUrl
```

7.3 Understand what we just created

Code	Purpose
`BaseModel`	Provides data validation through Pydantic

`title: str`	Article headline
`publisher: str`	Name of the news publisher
`published_at`	Publication date and time
`HttpUrl`	Ensures the article URL is valid HTTP/HTTPS
`datetime | None`	Allows missing publication dates

Why is this useful?
Suppose Google News returns an article without a publication date. Our application can still process it because `published_at` is optional.
But an invalid article URL should be rejected.

7.4 Test the model

Run this command in your activated virtual environment:

```powershell
python -c "from google_news_mcp.models.article import NewsArticle;
print('NewsArticle model imported successfully')"
```

Expected output:

```text
NewsArticle model imported successfully
```

Stop here and share your output.
Next, we'll build `providers/google_news.py` — the module that actually connects to Google News, fetches RSS feeds and converts them into our `NewsArticle` objects.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from google_news_mcp.models.article import NewsArticle; print('NewsArticle model imported successfully')"
NewsArticle model imported successfully

NewsArticle model verified

Great! Your first application model is working. Python successfully imported `NewsArticle`.
Now we're moving to the most important part of the project: **connecting Python to Google News and retrieving real news articles.**

Step 8 — Build the Google News provider

First, understand where this module fits:

Google News RSS

Provides RSS/XML data

↓

google_news.py

Fetches and parses the RSS feed

↓

NewsArticle

Validated Python article objects

Our provider's responsibility is to communicate with Google News. It doesn't know anything about MCP or Gemini.

8.1 Create the file

In VS Code Explorer, navigate to:

```text
src/
└── google_news_mcp/
    └── providers/
```

Create a new file:

```text
google_news.py
```

8.2 Write the implementation

Paste the following code:

```python

from datetime import datetime
from email.utils import parsedate_to_datetime

import feedparser
import httpx
from pydantic import ValidationError

from google_news_mcp.config import settings
from google_news_mcp.models.article import NewsArticle


class GoogleNewsProvider:
    """Fetches and parses news articles from Google News RSS."""

    def __init__(self) -> None:
```

```
        self.base_url = f"{settings.google_news_base_url}/search"
        self.timeout = settings.request_timeout

    async def search(
        self,
        query: str,
        region: str = settings.default_region,
        language: str = settings.default_language,
        max_results: int = settings.default_max_results,
    ) -> list[NewsArticle]:
        """Search Google News and return validated articles."""

        if not query.strip():
            raise ValueError("Search query cannot be empty.")

        if not 1 <= max_results <= settings.maximum_results:
            raise ValueError(
                f"max_results must be between 1 and "
                f"{settings.maximum_results}."
            )

        params = {
            "q": query.strip(),
            "hl": f"{language}-{region}",
            "gl": region,
            "ceid": f"{region}:{language}",
        }

        try:
            async with httpx.AsyncClient(
                timeout=self.timeout,
                headers={"User-Agent": "GoogleNewsMCP/0.1"},
            ) as client:
                response = await client.get(
                    self.base_url,
                    params=params,
                )
                response.raise_for_status()

        except httpx.HTTPError as exc:
            raise RuntimeError(
                f"Failed to retrieve Google News: {exc}"
            ) from exc

        feed = feedparser.parse(response.content)

        if feed.bozo and not feed.entries:
            raise ValueError("Google News returned an invalid RSS feed.")
```

```
    articles: list[NewsArticle] = []

    for entry in feed.entries[:max_results]:
        title = entry.get("title")
        article_url = entry.get("link")

        if not title or not article_url:
            continue

        source = entry.get("source") or {}
        publisher = source.get("title", "Unknown publisher")

        published_at: datetime | None = None

        published = entry.get("published")

        if published:
            try:
                published_at = parsedate_to_datetime(published)
            except (TypeError, ValueError, OverflowError):
                published_at = None

        try:
            article = NewsArticle(
                title=title,
                publisher=publisher,
                published_at=published_at,
                url=article_url,
            )

            articles.append(article)

        except ValidationError:
            continue

    return articles
```

Save the file using `Ctrl + S` .

8.3 Understand what we just implemented

There are four important concepts here.

1. HTTPX — communicating with Google News

```python
response = await client.get(
    self.base_url,
    params=params,
)
```

HTTPX sends an HTTP GET request to Google News.
We use `await` because the operation is asynchronous. Python can perform other work while waiting for the network response.

2. Feedparser — understanding RSS

```python
feed = feedparser.parse(response.content)
```

Google News returns XML data.
Feedparser converts that XML into Python objects that we can access.
For example:

```python
entry.title
entry.link
entry.published
```

3. Pydantic — validating articles

```python
article = NewsArticle(
    title=title,
    publisher=publisher,
    published_at=published_at,
    url=article_url,
)
```

Every parsed article must follow the data model we created in the previous step.

4. Error handling

We handle network failures, invalid RSS responses and invalid article data rather than allowing every failure to crash the entire operation.

8.4 Test the provider

Now we'll verify that our Python application can actually retrieve news.
Create a temporary file in your project root called:
`test_provider.py`
Paste:

python

```python
import asyncio

from google_news_mcp.providers.google_news import GoogleNewsProvider


async def main():
    provider = GoogleNewsProvider()

    articles = await provider.search(
        query="artificial intelligence",
        max_results=5,
    )

    print(f"\nRetrieved {len(articles)} articles\n")

    for index, article in enumerate(articles, start=1):
        print(f"{index}. {article.title}")
        print(f"   Publisher: {article.publisher}")
        print(f"   Published: {article.published_at}")
        print(f"   URL: {article.url}")
        print()


if __name__ == "__main__":
    asyncio.run(main())
```

Run this in your activated virtual environment:

powershell

```
python test_provider.py
```

Expected result You should see up to five news articles with their titles, publishers, publication dates and URLs.
This test requires an internet connection.
Important: This is a temporary integration test, not one of our final automated tests. We'll move proper tests into the `tests/` directory later.

Your task

Complete these three actions:

1. Create `providers/google_news.py`.
2. Create the temporary `test_provider.py`.
3. Run `python test_provider.py`.

Share the terminal output with me.

Once this works, we'll have achieved our first major milestone: **our Python application can independently communicate with Google News and retrieve structured news articles.**

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\src\google_news_mcp> python test_provider.py

Retrieved 5 articles

1. From "Who The Hell Is He" To Defending Sex Scandal: Trump On Jensen Huang - NDTV
   Publisher: NDTV
   Published: 2026-11-11 06:10:47+00:00
   URL:
   https://news.google.com/rss/articles/CBMiwgFBVV95cUxNTExrYmo0MkpoUnpaTUViZFlUY3RXM3Vsa2VFNmk4bkg2X3FnSU1nQkFXelBRMnZ4TEF4WlhXeE4xc0N2dms4bnRsNndDVGwxd04xdzlSSDRYcjdHUUJGbWV3c0VucXh1b2NZYndDdHZMem5tdGpVRUtQS1JHQVgyYkJLS2lEd0U4a2RKZl83dHRqVk5BUXg4SllmZUhoc1c2Y0RnMjQwbUE5VmtzdVMxbE52NlBna2ZJLWRxb1RrR1VDZ9IBygFBVV95cUxPLWtwR19reTEtVkdoQlFrWld0cjVWUm95X0dkWjdfR3VmQkhhNVg3bEY0cnZQNEl6enRta0VqRlJqOHpOYU0yWFdhX3RRLVVIWVRWZ2lndEFER3ptR0p6NVQ0Qk9sNHhyWm5QQ0lSak5BRkJiZ3dWYTN3OC1xLWttek5NcktGZmFPTktJTHR0Zm1hWEV6X3l3Sm1RMHJsbkZoNENVUmRpbzlqdDFMUWNUQXZVYkdPdjl6R0psbjBDT2VMRDZLWEVMNWZn?oc=5
2. Cancer Will Be History In 10 Years: Anthropic CEO's Big Promise - NDTV
   Publisher: NDTV
   Published: 2026-11-11 07:50:04+00:00
   URL:
   https://news.google.com/rss/articles/CBMitgFBVV95cUxNbGlIUU9rOF9fRU5VNnZrMk9VU0ZLUmVLX3U4LU0tVVRiQkYyTG1kTVNRclJ0cGk0VDAxN2xhdnZGcVBTTzkxY3lxVFNrWThyVU1PR2RDaGROTV9GWVdlRGhwYWZEZlVMNzhlT3NkaWZNeGR2dVI3WU9Ea2J5czR5RzZzRF96SXJRV1ZnQ25FdWlqTWRMYTR4QmswMXhxc3RTb1ZLX1ZpMWRNMWozUDFkR0pfbW9BUdIBvgFBVV95cUxNdWN4dGphV21JcFN3UlVjNS0wYnZrVUx3WXlzVlBkaTBzUE9UbzlxNmxpdGNQTGxHRWo5aExRekQ2WGRuS2wydjVOb2s5SnpNaWpUSHpnOEtIdDNwNmhCc0JDRmI0eGxkMTlKc0pwamhfWjQxWHpaUERYQkhJeTJSX2NTa0dHRmhNLWM2VjhvSFUxRVp2R3ZwX3lXYW9HMzI5SklFUDgyTl9HNVpnQ0hLRG1oMkpndExZXzhRalRn?oc=5
3. Claude chatbot maker set for $1.3tn stock market debut despite 'catastrophic' AI warning - BBC
   Publisher: BBC
   Published: 2026-11-10 17:56:12+00:00
   URL:

https://news.google.com/rss/articles/CBMiW0FVX3lxTE1HNWFGZG1teU8xOWttMlMzb1gtbVhmNW94MlFvanZrY3JCbVVkcExpX3VrTGFLOVgwaGhON21KMUN5SGI1R2hmM2pMMkJXQlFDaUU0TnV6TWVqNWM?oc=5

4. Artificial intelligence is desperately seeking its narrative - The Conversation
   Publisher: The Conversation
   Published: 2026-11-10 13:12:58+00:00
   URL:
   https://news.google.com/rss/articles/CBMimwFBVV95cUxQOWk0NGkyaTZjT29pR0VlM25mNGtrQmZqSnFxUS16cGgyU3dFcXNpQ0I5OGtrVzY4UmpsMHprcjJIQ0E5U1YwTTJtS3ZLTzNPMHhPMFd2TklGSHFMOUhsc2kwZkN6V21hY3ItVC1YM1hVSDd3UDheXF1QjR6bUJUbVBEbjdCa0E1bnIyc3VuNXhiX3o5enBGYTlaSQ?oc=5
5. U.S., China should ‘harbor no illusions’ as they cooperate, compete on AI, Bessent says - CNBC
   Publisher: CNBC
   Published: 2026-11-11 05:11:33+00:00
   URL:
   https://news.google.com/rss/articles/CBMicEFVX3lxTFBORGNIaVNVRlc0ZlQ3VGZhTTE4YjJ0dFgyektfQjByZEUwVExVNUpkaGplOFFfRlo0YXNuQ3BQb0xyZ3NGYVZJSTdfZC1QRW5zWE1nZGN2WGZKVk0wcEdTWkZCMUFoM25MeTlhZktFWVc?oc=5

Major milestone achieved!

Your Google News provider is working. It successfully:

- Connected to Google News RSS.
- Retrieved five articles.
- Parsed RSS/XML responses using Feedparser.
- Extracted article titles and publisher names.
- Converted publication dates into Python `datetime` objects.
- Validated article data using Pydantic.

This confirms that our **external data integration layer is functional**.

One small cleanup

I noticed your terminal path:

```text
...\google-news-mcp\src\google_news_mcp>
```

You ran the temporary test from inside the application package. It works, but test scripts shouldn't live inside our application package.
Move `test_provider.py` to the project root.
While you're in the current directory, run:

```powershell
Move-Item .\test_provider.py ..\..\test_provider.py
```

Then return to the project root:

```powershell
cd ..\..
```

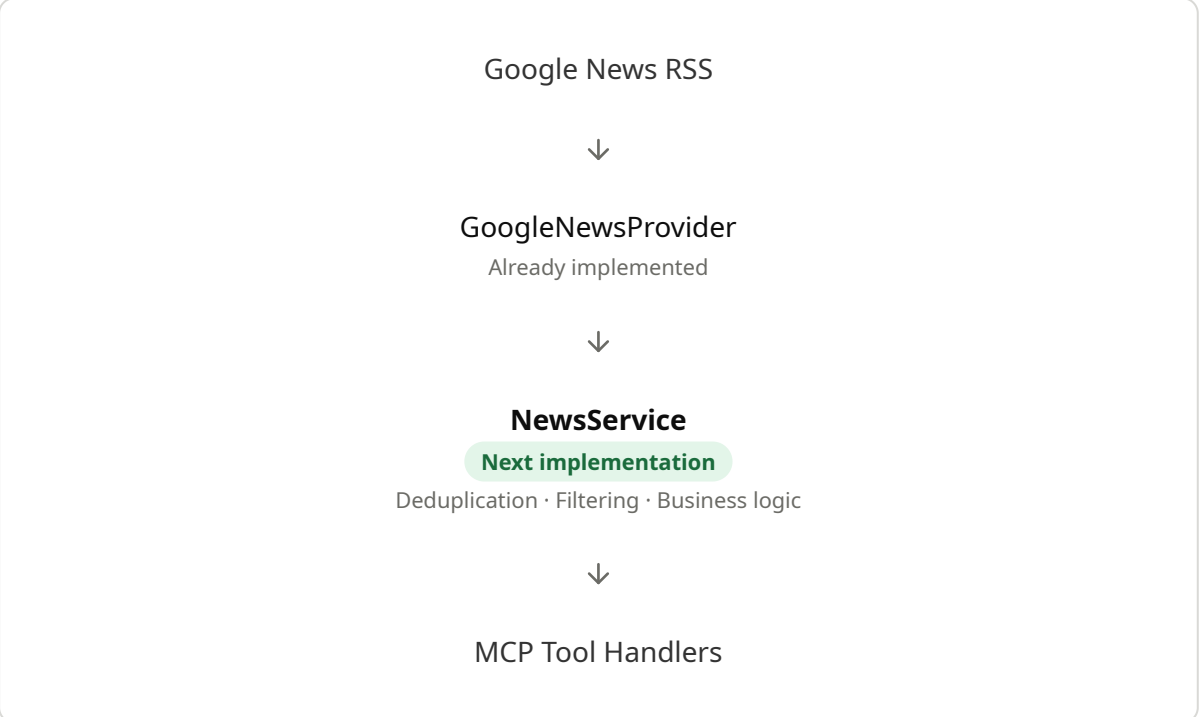
Run it again:

```powershell
python test_provider.py
```

It should still retrieve the articles.

Step 9 — What's next?

We're now ready to build our **News Service layer**.



The News Service will make our application smarter without depending on any AI model. First, complete the file move and verify the test still works. Then we'll implement `services/news_service.py`.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\src\google_news_mcp> Move-Item .\test_provider.py ....\test_provider.py
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\src\google_news_mcp> cd ....
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python test_provider.py

Retrieved 5 articles

1. From "Who The Hell Is He" To Defending Sex Scandal: Trump On Jensen Huang - NDTV
   Publisher: NDTV
   Published: 2026-11-11 06:10:47+00:00
   URL:
   https://news.google.com/rss/articles/CBMiwgFBVV95cUxNTExrYmo0MkpoUnpaTUViZFlUY3RXM3Vsa2VFNmk4bkg2X3FnSU1nQkFXelBRMnZ4TEF4WlhXeE4xc0N2dms4bnRsNndDVGwxd04xdzlSSDRYcjdHUUJGbWV3c0VucXh1b2NZYndDdHZMem5tdGpVRUtQS1JHQVgyYkJLS2lEd0U4a2RKZl83dHRqVk5BUXg4SllmZUhoc1c2Y0RnMjQwbUE5VmtzdVMxbE52NlBna2ZJLWRxb1RrR1VDZ9IBygFBVV95cUxPLWtwR19reTEtVkdoQlFrWld0cjVWUm95X0dkWjdfR3VmQkhhNVg3bEY0cnZQNEl6enRta0VqRlJqOHpOYU0yWFdhX3RRLVVIWVRWZ2lndEFER3ptR0p6NVQ0Qk9sNHhyWm5QQ0lSak5BRkJiZ3dWYTN3OC1xLWttek5NcktGZmFPTktJTHR0Zm1hWEV6X3l3Sm1RMHJsbkZoNENVUmRpbzlqdDFMUWNUQXZVYkdPdjl6R0psbjBDT2VMRDZLWEVMNWZn?oc=5
2. Cancer Will Be History In 10 Years: Anthropic CEO's Big Promise - NDTV
   Publisher: NDTV
   Published: 2026-11-11 07:50:04+00:00
   URL:
   https://news.google.com/rss/articles/CBMitgFBVV95cUxNbGlIUU9rOF9fRU5VNnZrMk9VU0ZLUmVLX3U4LU0tVVRiQkYyTG1kTVNRclJ0cGk0VDAxN2xhdnZGcVBTTzkxY3lxVFNrWThyVU1PR2RDaGROTV9GWVdlRGhwYWZEZlVMNzhlT3NkaWZNeGR2dVI3WU9Ea2J5czR5RzZzRF96SXJRV1ZnQ25FdWlqTWRMYTR4QmswMXhxc3RTb1ZLX1ZpMWRNMWozUDFkR0pfbW9BUdIBvgFBVV95cUxNdWN4dGphV21JcFN3UlVjNS0wYnZrVUx3WXlzVlBkaTBzUE9UbzlxNmxpdGNQTGxHRWo5aExRekQ2WGRuS2wydjVOb2s5SnpNaWpUSHpnOEtIdDNwNmhCc0JDRmI0eGxkMTlKc0pwamhfWjQxWHpaUERYQkhJeTJSX2NTa0dHRmhNLWM2VjhvSFUxRVp2R3ZwX3lXYW9HMzI5SklFUDgyTl9HNVpnQ0hLRG1oMkpndExZXzhRalRn?oc=5
3. Claude chatbot maker set for $1.3tn stock market debut despite 'catastrophic' AI warning - BBC
   Publisher: BBC
   Published: 2026-11-10 17:56:12+00:00
   URL:
   https://news.google.com/rss/articles/CBMiW0FVX3lxTE1HNWFGZG1teU8xOWttMlMzb1gtbVhmNW94MlFvanZrY3JCbVVkcExpX3VrTGFLOVgwaGhON21KMUN5SGI1R2hmM2pMMkJXQlFDaUU0TnV6TWVqNWM?oc=5
4. Artificial intelligence is desperately seeking its narrative - The Conversation
   Publisher: The Conversation
   Published: 2026-11-10 13:12:58+00:00
   URL:
   https://news.google.com/rss/articles/CBMimwFBVV95cUxQOWk0NGkyaTZjT29pR

0VlM25mNGtrQmZqSnFxUS16cGgyU3dFcXNpQ0I5OGtrVzY4UmpsMHprcjJIQ0E5U1YwTTJtS3ZLTzNPMHhPMFd2TklGSHFMOUhsc2kwZkN6V21hY3ItVC1YM1hVSDd3UDhheXF1QjR6bUJUbVBEbjdCa0E1bnIyc3VuNXhiX3o5enBGYTlaSQ?oc=5

5. U.S., China should 'harbor no illusions' as they cooperate, compete on AI, Bessent says - CNBC
   Publisher: CNBC
   Published: 2026-11-11 05:11:33+00:00
   URL: https://news.google.com/rss/articles/CBMicEFVX3lxTFBORGNIaVNVRlc0ZlQ3VGZhTTE4YjJ0dFgyektfQjByZEUwVExVNUpkaGplOFFfRlo0YXNuQ3BQb0xyZ3NGYVZJSTdfZC1QRW5zWE1nZGN2WGZKVk0wcEdTWkZCMUFoM25MeTlhZktFWVc?oc=5

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>

Excellent! Everything is working exactly as expected. Your file move was successful, and the provider still retrieves five live articles when executed from the project root.

Google News integration verified

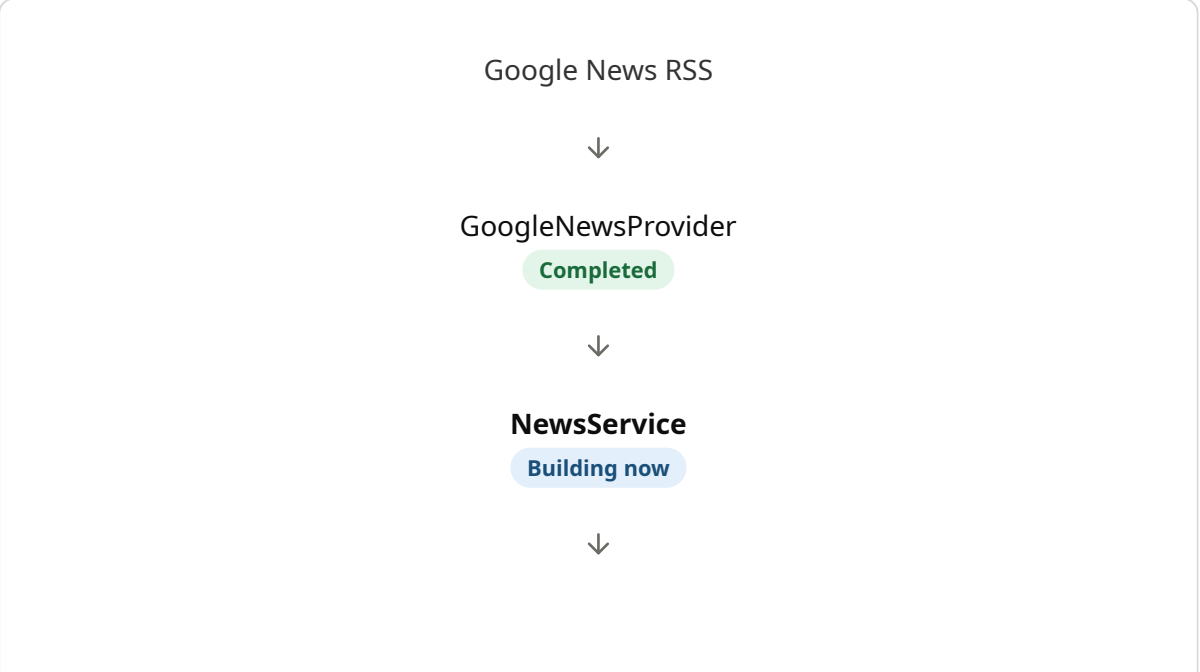
We've completed these components:

1. Python environment and dependencies.
2. Project structure.
3. Configuration module.
4. Pydantic article model.
5. Google News RSS provider.
6. Live news retrieval.

Now let's build the next architectural layer.

Step 9 — Build the News Service

Remember our architecture?



MCP Tools

9.1 What is the responsibility of NewsService?

Our provider knows how to fetch articles from Google News.
But imagine Google News returns duplicate articles.
We don't want every MCP tool to implement duplicate removal independently.
Instead, the News Service will handle shared business logic.
Its initial responsibilities:

- Validate search queries.
- Call the Google News provider.
- Remove duplicate article URLs.
- Return clean article data.
- Keep the service independent of the MCP protocol.

Provider = communicates with Google News.
Service = decides how our application processes the retrieved data.

9.2 Create the file

Navigate to:

```text
src/
└── google_news_mcp/
    └── services/
```

Create:

```text
news_service.py
```

9.3 Implement the News Service

Paste this code:

```python

from google_news_mcp.config import settings
from google_news_mcp.models.article import NewsArticle
from google_news_mcp.providers.google_news import GoogleNewsProvider


class NewsService:
    """Provides application-level news search functionality."""

    def __init__(
        self,
```

```
        provider: GoogleNewsProvider | None = None,
    ) -> None:
        self._provider = provider or GoogleNewsProvider()

    async def search_news(
        self,
        query: str,
        region: str = settings.default_region,
        language: str = settings.default_language,
        max_results: int = settings.default_max_results,
    ) -> list[NewsArticle]:
        """Search news and return deduplicated articles."""

        normalized_query = query.strip()

        if not normalized_query:
            raise ValueError("Search query cannot be empty.")

        articles = await self._provider.search(
            query=normalized_query,
            region=region,
            language=language,
            max_results=max_results,
        )

        unique_articles: list[NewsArticle] = []
        seen_urls: set[str] = set()

        for article in articles:
            article_url = str(article.url).rstrip("/")

            if article_url in seen_urls:
                continue

            seen_urls.add(article_url)
            unique_articles.append(article)

        return unique_articles[:max_results]
```

Save the file.

9.4 Understand three important concepts

1. Dependency Injection

Look at:

```python
def __init__(
    self,
    provider: GoogleNewsProvider | None = None,
):
    self._provider = provider or GoogleNewsProvider()
```

Why are we passing a provider into the service?
Because the service shouldn't be permanently tied to one provider instance.
Later, while writing tests, we can inject a mock provider instead of making actual Google News requests.
This is called dependency injection.

2. Deduplication

```python
seen_urls: set[str] = set()
```

A Python `set` stores unique values.
When we encounter a URL that has already been seen, we skip the duplicate.

3. Separation of responsibilities

Notice that this service doesn't know anything about HTTPX or RSS parsing.
It simply calls:

```python
await self._provider.search(...)
```

That's the benefit of our layered architecture.

9.5 Test the News Service

Let's update our temporary `test_provider.py` file in your project root.
Replace its contents with:

```python

import asyncio

from google_news_mcp.services.news_service import NewsService


async def main():
    service = NewsService()

    articles = await service.search_news(
        query="artificial intelligence",
        max_results=5,
```

```
    )

    print(f"\nRetrieved {len(articles)} unique articles\n")

    for index, article in enumerate(articles, start=1):
        print(f"{index}. {article.title}")
        print(f"   Publisher: {article.publisher}")
        print(f"   Published: {article.published_at}")
        print(f"   URL: {article.url}")
        print()


if __name__ == "__main__":
    asyncio.run(main())
```

Execute:

```powershell
python test_provider.py
```

You should again see five news articles.

What's coming next?

Once this works, we'll move into the actual MCP implementation.
Our upcoming tools will be:

MCP tool	Function
`search_news`	Search Google News
`get_top_headlines`	Retrieve general headlines
`search_news_by_publisher`	Find articles from a particular publisher

We'll register them with the official Python MCP SDK and test them using MCP Inspector.
Your task now: Create `news_service.py`, update the temporary test file, and run it.
Share the terminal output with me.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python test_provider.py

Retrieved 5 unique articles

1. From "Who The Hell Is He" To Defending Sex Scandal: Trump On Jensen Huang - NDTV
   Publisher: NDTV
   Published: 2026-11-11 06:10:47+00:00
   URL:

https://news.google.com/rss/articles/CBMiwgFBVV95cUxNTExrYmo0MkpoUnpaTUViZFlUY3RXM3Vsa2VFNmk4bkg2X3FnSU1nQkFXelBRMnZ4TEF4WlhXeE4xc0N2dms4bnRsNndDVGwxd04xdzlSSDRYcjdHUUJGbWV3c0VucXh1b2NZYndDdHZMem5tdGpVRUtQS1JHQVgyYkJLS2lEd0U4a2RKZl83dHRqVk5BUXg4SllmZUhoc1c2Y0RnMjQwbUE5VmtzdVMxbE52NlBna2ZJLWRxb1RrR1VDZ9IBygFBVV95cUxPLWtwR19reTEtVkdoQlFrWld0cjVWUm95X0dkWjdfR3VmQkhhNVg3bEY0cnZQNEl6enRta0VqRlJqOHpOYU0yWFdhX3RRLVVIWVRWZ2lndEFER3ptR0p6NVQ0Qk9sNHhyWm5QQ0lSak5BRkJiZ3dWYTN3OC1xLWttek5NcktGZmFPTktJTHR0Zm1hWEV6X3l3Sm1RMHJsbkZoNENVUmRpbzlqdDFMUWNUQXZVYkdPdjl6R0psbjBDT2VMRDZLWEVMNWZn?oc=5

2. Cancer Will Be History In 10 Years: Anthropic CEO's Big Promise - NDTV
   Publisher: NDTV
   Published: 2026-11-11 07:50:04+00:00
   URL:
   https://news.google.com/rss/articles/CBMitgFBVV95cUxNbGlIUU9rOF9fRU5VNnZrMk9VU0ZLUmVLX3U4LU0tVVRiQkYyTG1kTVNRclJ0cGk0VDAxN2xhdnZGcVBTTzkxY3lxVFNrWThyVU1PR2RDaGROTV9GWVdlRGhwYWZEZlVMNzhlT3NkaWZNeGR2dVI3WU9Ea2J5czR5RzZzRF96SXJRV1ZnQ25FdWlqTWRMYTR4QmswMXhxc3RTb1ZLX1ZpMWRNMWozUDFkR0pfbW9BUdIBvgFBVV95cUxNdWN4dGphV21JcFN3UlVjNS0wYnZrVUx3WXlzVlBkaTBzUE9UbzlxNmxpdGNQTGxHRWo5aExRekQ2WGRuS2wydjVOb2s5SnpNaWpUSHpnOEtIdDNwNmhCc0JDRmI0eGxkMTlKc0pwamhfWjQxWHpaUERYQkhJeTJSX2NTa0dHRmhNLWM2VjhvSFUxRVp2R3ZwX3lXYW9HMzI5SklFUDgyTl9HNVpnQ0hLRG1oMkpndExZXzhRaIRn?oc=5

3. Claude chatbot maker set for $1.3tn stock market debut despite 'catastrophic' AI warning - BBC
   Publisher: BBC
   Published: 2026-11-10 17:56:12+00:00
   URL:
   https://news.google.com/rss/articles/CBMiW0FVX3lxTE1HNWFGZG1teU8xOWttMlMzb1gtbVhmNW94MlFvanZrY3JCbVVkcExpX3VrTGFLOVgwaGhON21KMUN5SGI1R2hmM2pMMkJXQlFDaUU0TnV6TWVqNWM?oc=5

4. Artificial intelligence is desperately seeking its narrative - The Conversation
   Publisher: The Conversation
   Published: 2026-11-10 13:12:58+00:00
   URL:
   https://news.google.com/rss/articles/CBMimwFBVV95cUxQOWk0NGkyaTZjT29pR0VlM25mNGtrQmZqSnFxUS16cGgyU3dFcXNpQ0I5OGtrVzY4UmpsMHprcjJIQ0E5U1YwTTJtS3ZLTzNPMHhPMFd2TklGSHFMOUhsc2kwZkN6V21hY3ItVC1YM1hVSDd3UDhheXF1QjR6bUJUbVBEbjdCa0E1bnIyc3VuNXhiX3o5enBGYTlaSQ?oc=5

5. U.S., China should ‘harbor no illusions’ as they cooperate, compete on AI, Bessent says - CNBC
   Publisher: CNBC
   Published: 2026-11-11 05:11:33+00:00
   URL:

https://news.google.com/rss/articles/CBMicEFVX3lxTFBORGNIaVNVRlc0ZlQ3VGZhTTE4YjJ0dFgyektfQjByZEUwVExVNUpkaGplOFFfRlo0YXNuQ3BQb0xyZ3NGYVZJSTdfZC1QRW5zWE1nZGN2WGZKVk0wcEdTWkZCMUFoM25MeTlhZktFWVc?oc=5

Excellent! **News Service verified**

Our provider and service layers are now both working:

- Provider retrieves live news.
- Service processes the articles.
- Duplicate URLs are removed.
- The output is returned as validated `NewsArticle` objects.

We're ready to start exposing functionality through MCP.

Step 10 — Create the MCP response model

Before registering our first MCP tool, let's define the response structure it will return.

Why?

When an AI client calls our `search_news` tool, we don't want to return an unstructured block of text. We want predictable JSON data.

For example:

```json
{
  "query": "artificial intelligence",
  "region": "IN",
  "language": "en",
  "count": 2,
  "articles": [
    {
      "title": "Example news headline",
      "publisher": "BBC",
      "published_at": "2026-11-11T10:00:00Z",
      "url": "https://news.google.com/example"
    }
  ]
}
```

The Python MCP SDK v2 supports Pydantic models as structured tool outputs. It automatically derives an output schema from the model. MCP Python SDK

MCP Python SDK

10.1 Create a new file

Navigate to:

```text
src/
└── google_news_mcp/
    └── models/
```

Create:

```text
search.py
```

10.2 Add this code

```python
from pydantic import BaseModel, Field

from google_news_mcp.models.article import NewsArticle


class NewsSearchResponse(BaseModel):
    """Structured response returned by a Google News search."""

    query: str = Field(description="The original search query.")
    region: str = Field(description="Two-letter country code.")
    language: str = Field(description="Language used for the search.")
    count: int = Field(description="Number of articles returned.")
    articles: list[NewsArticle] = Field(
        description="List of matching news articles."
    )
```

Save the file.

10.3 Understand what we created

We already had `NewsArticle`, which represents one article.
Now `NewsSearchResponse` represents the complete search result.
Think of it this way:

- `NewsArticle` = one record.
- `NewsSearchResponse` = the complete API response containing multiple records and search metadata.

This is useful because our MCP tool will have a clear, machine-readable output contract.

10.4 Verify the model

Run this from your project root:

```powershell
python -c "from google_news_mcp.models.search import NewsSearchResponse;
print('NewsSearchResponse imported successfully')"
```

Expected output:

```text
NewsSearchResponse imported successfully
```

Stop here and share your output.
Next, we'll create our first actual MCP tool:
`search_news()`
We'll register it with `MCPServer`, connect it to our existing News Service, and test it using MCP Inspector.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from google_news_mcp.models.search import NewsSearchResponse; print('NewsSearchResponse imported successfully')"
NewsSearchResponse imported successfully

NewsSearchResponse verified
Excellent! Our data model is ready. Now we'll create our **first actual MCP tool**.

Step 11 — Implement `search_news` MCP tool

Until now, we have built Python functionality that can retrieve news.
But an AI assistant cannot automatically access our `NewsService`. We need to expose its functionality through the MCP protocol.
That's what an MCP tool does.
The official Python SDK uses the `@mcp.tool()` decorator to register a Python function as an MCP tool. It derives the input schema from the function's type hints and descriptions.
MCP Python SDK MCP Python SDK

11.1 Create the file

Navigate to:

```text
src/
└── google_news_mcp/
    └── tools/
```

Create:

```text
news_tools.py
```

11.2 Add the implementation

Paste this code:

python

```python
from typing import Annotated

from mcp.server import MCPServer
from pydantic import Field

from google_news_mcp.models.search import NewsSearchResponse
from google_news_mcp.services.news_service import NewsService


def register_news_tools(
    mcp: MCPServer,
    news_service: NewsService,
) -> None:
    """Register Google News search tools with the MCP server."""

    @mcp.tool()
    async def search_news(
        query: Annotated[
            str,
            Field(
                min_length=2,
                max_length=200,
                description="Topic or keywords to search for in Google News.",
            ),
        ],
        region: Annotated[
            str,
            Field(description="Two-letter country code, such as IN or US."),
        ] = "IN",
        language: Annotated[
            str,
            Field(description="Two-letter language code, such as en or ta."),
        ] = "en",
        max_results: Annotated[
            int,
            Field(
                ge=1,
                le=50,
                description="Maximum number of articles to return.",
            ),
        ] = 10,
    ) -> NewsSearchResponse:
```

```
        """Search Google News for articles matching a topic.

        Returns article titles, publishers, publication dates and URLs.
        """

        articles = await news_service.search_news(
            query=query,
            region=region.upper(),
            language=language.lower(),
            max_results=max_results,
        )

        return NewsSearchResponse(
            query=query.strip(),
            region=region.upper(),
            language=language.lower(),
            count=len(articles),
            articles=articles,
        )
```

Save using `Ctrl + S` .

11.3 Understand what we just built

There are four important concepts here.

1. MCP server registration

```python
@mcp.tool()
```

This decorator tells the MCP SDK:
"This Python function should be exposed as a callable MCP tool."

2. Annotated and Field

```python
query: Annotated[
    str,
    Field(min_length=2, max_length=200)
]
```

This describes the input requirements.
An AI client can discover that `query` is a string and must contain between 2 and 200 characters.

3. Dependency injection

```python
news_service: NewsService
```

We pass our existing NewsService into the registration function.
This avoids creating a separate provider for every tool and makes the code easier to test.

4. Structured response

```python
) -> NewsSearchResponse:
```

The MCP SDK can use our Pydantic response model to expose structured output to compatible clients.
So instead of manually constructing JSON-RPC messages, we let the SDK handle that.

11.4 Verify the file

Run this from your project root:

```powershell
python -c "from google_news_mcp.tools.news_tools import register_news_tools;
print('MCP tool registration module imported successfully')"
```

Expected:

```text
MCP tool registration module imported successfully
```

Our architecture now looks like this:

1. Google News RSS provider **Done**
2. NewsService **Done**
3. NewsSearchResponse **Done**
4. First MCP tool: `search_news` **Implemented**
5. MCP server entry point **Next**

Your task: Create `news_tools.py`, paste the implementation and run the verification command.
Next, we'll create `server.py`, instantiate `MCPServer`, register our tool and start the actual MCP server using STDIO.
That's when our application becomes a running MCP server rather than just a collection of Python modules.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from google_news_mcp.tools.news_tools import register_news_tools; print('MCP tool registration module imported successfully')"
MCP tool registration module imported successfully

First MCP tool registered successfully

Excellent! The import test passes. We've now implemented the Python function that exposes Google News search as an MCP tool.

Now comes an important milestone: **we'll create the actual MCP server entry point.**

Step 12 — Create `server.py`

Until now, we've built separate modules. We need one entry point that brings them together.

Our server will:

1. Create an MCP server instance.
2. Create a NewsService instance.
3. Register our news tools.
4. Start the server using STDIO transport.

The official SDK v2 uses `MCPServer` and `mcp.run()` for this. GitHub GitHub

12.1 Create the file

Navigate to:

```text
src/
└── google_news_mcp/
```

Create a new file:

```text
server.py
```

12.2 Add the implementation

```python
from mcp.server import MCPServer

from google_news_mcp.services.news_service import NewsService
from google_news_mcp.tools.news_tools import register_news_tools


# Create the MCP server.
mcp = MCPServer("Google News MCP")
```

```python
# Create the application service.
news_service = NewsService()


# Register all available news tools.
register_news_tools(mcp, news_service)


if __name__ == "__main__":
    mcp.run()
```

Save the file.

12.3 Understand the implementation

1. Create the MCP server

```python
mcp = MCPServer("Google News MCP")
```

This creates the MCP server instance and gives it a name.

2. Create the service

```python
news_service = NewsService()
```

This creates our business logic layer, which internally uses `GoogleNewsProvider`.

3. Register the tools

```python
register_news_tools(mcp, news_service)
```

This connects our `search_news` Python function to the MCP server.

4. Start the server

```python
mcp.run()
```

This starts the server. By default, it uses STDIO transport.
STDIO means the AI application communicates with our Python server through standard input and output. It doesn't require us to open an HTTP port. GitHub

12.4 Verify the server imports

From your project root, run:

```powershell
python -c "from google_news_mcp.server import mcp; print('Google News MCP server initialized successfully')"
```

Expected output:

```text
Google News MCP server initialized successfully
```

Notice that importing the server doesn't start it because we used:

```python
if __name__ == "__main__":
```

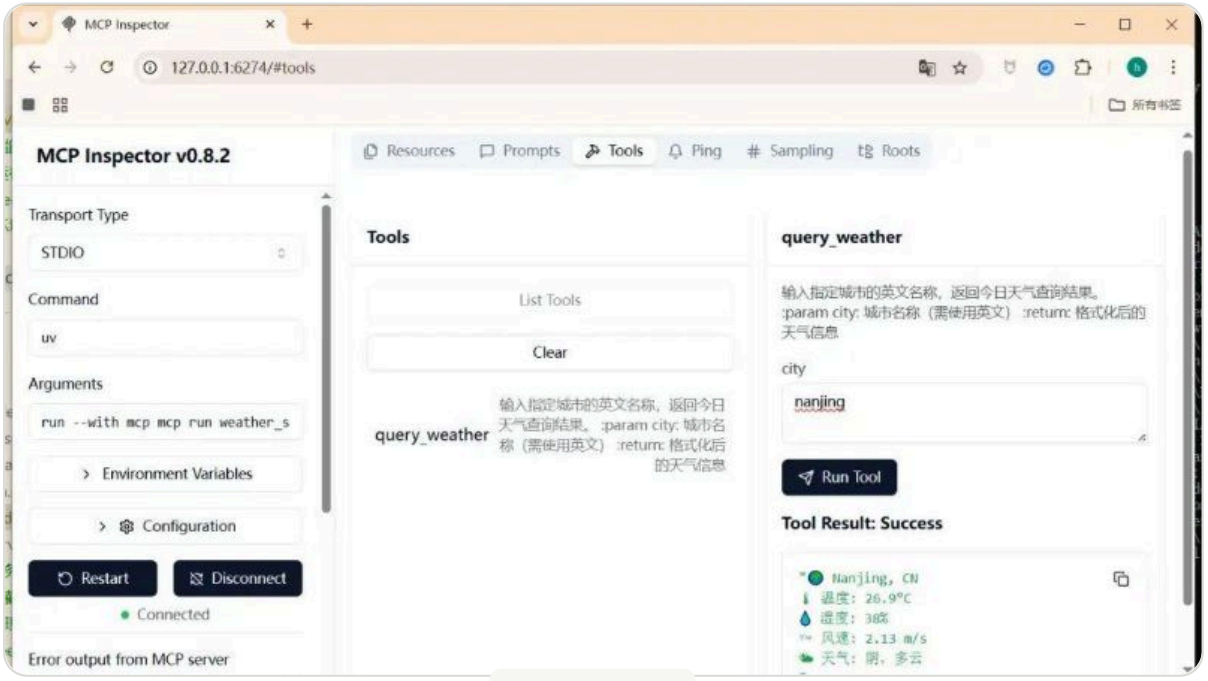
This allows development tools to import our server safely.

12.5 Launch MCP Inspector

Once the import test succeeds, run:

```powershell
mcp dev src/google_news_mcp/server.py
```

This launches the MCP Inspector, which lets us interactively discover and call our tools.

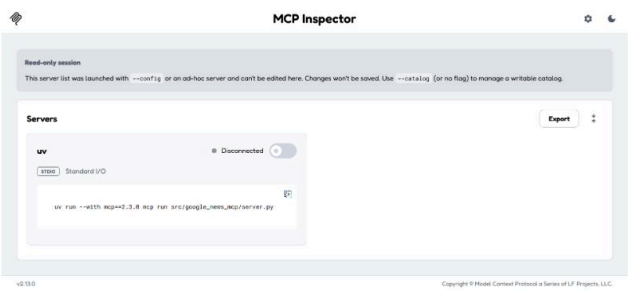


In the Inspector, we'll check whether `search_news` appears under the Tools tab. Don't worry about connecting Gemini yet. First, we need to prove that our MCP server works independently.

Current development progress

1. Python environment and dependencies **Done**
2. Google News provider **Done**
3. News service **Done**
4. News response models **Done**
5. MCP tool registration **Done**
6. MCP server entry point **Now**
7. Inspector testing **Next**
8. Additional tools, resources and prompts **Pending**
9. Gemini integration **Pending**

Your task: Create `server.py`, run the import verification command, and then launch `mcp dev src/google_news_mcp/server.py`.
Share what appears in the MCP Inspector.



Good! Your MCP Inspector has opened successfully. We're very close to testing our first MCP tool.
I can see three things in your screenshot:

1. **MCP Inspector is running.** **Success**
2. Your server launch configuration has been detected.
3. The server is currently **Disconnected**.

The grey "Read-only session" banner is normal here. It means Inspector is using the server configuration created by the development command.

Step 1 — Connect the server

On the server card, find the switch beside **Disconnected** (top-right of the card).
Click it once.
Inspector will attempt to launch your Python server and establish an MCP connection.

Step 2 — Check the result

Expected result The status should change from:
`Disconnected` → `Connected`
You should then be able to access the server's Tools section.

Step 3 — Find our tool

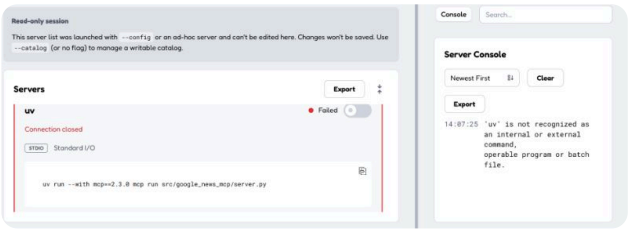
Navigate to the **Tools** tab.
You should find:

```text
search_news
```

Don't call it just yet. First, let's verify that the connection succeeds.
One possible issue: Inspector is launching the command using `uv`. If connection fails, run this in your VS Code terminal:

```powershell
uv --version
```

The official MCP SDK's development workflow uses `uv` to launch the server under Inspector. MCP Python SDK MCP Python SDK
Send me a screenshot after clicking the connection switch.



Found the issue! Your screenshot shows exactly what went wrong.
Error identified

```text
'uv' is not recognized as an internal or external command
```

MCP Inspector is trying to launch your server using **uv**, a Python package and project manager. But `uv` is not installed on your computer.
Your Python code has not failed. We just need to install the missing tool.

Step 1 — Install uv

Since your `.venv` is already activated, run this in your VS Code terminal:

```powershell
python -m pip install uv
```

This installs the `uv` command inside your active virtual environment.

Step 2 — Verify installation

Run:

```powershell
uv --version
```

You should see something similar to:

```text
uv 0.x.x
```

Step 3 — Restart MCP Inspector

Close the currently open Inspector and return to your VS Code terminal.
Make sure you're in the project root:

```text
google-news-mcp>
```

Make sure `(.venv)` is visible, then execute:

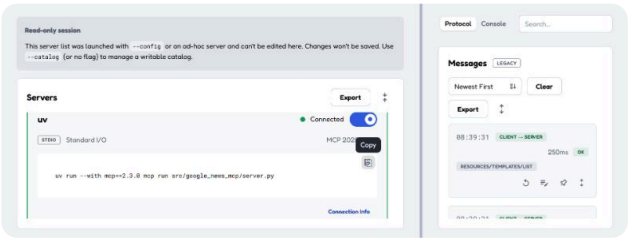
```powershell
mcp dev src/google_news_mcp/server.py
```

Open the Inspector and click the connection switch again.

Why are we installing uv?

Tool	Responsibility
Python	Runs our application
pip	Installs Python packages
uv	Runs and manages Python projects and environments
MCP SDK	Implements the MCP protocol
MCP Inspector	Tests our MCP server

The MCP SDK development workflow uses `uv run` to launch the server through Inspector. MCP Python SDK GitHub
Your task: Run the installation command and share the output of `uv --version`.
Once connected, we'll test our first `search_news` MCP tool!



Okay now it's connected now tell me next step

MCP server connected successfully!

Excellent! Your Google News MCP server is now running and MCP Inspector has established a connection.

Next, we're going to perform our **first actual MCP tool call**.

Step 13 — Test `search_news` in MCP Inspector

13.1 Open the Tools section

In MCP Inspector:

1. Scroll to the top of the page.
2. Look for the navigation tabs.
3. Select **Tools**.

The Tools section displays the tools registered by your MCP server. GitHub GitHub

You should find:

Available MCP tools Expected

search_news

Search Google News for articles matching a topic.

13.2 Select `search_news`

Click the tool and enter these parameters:

Parameter	Value
`query`	`artificial intelligence`
`region`	`IN`
`language`	`en`
`max_results`	`5`

Click **Run Tool** or **Call Tool**.

13.3 Expected response

You should receive structured data similar to:

```json
{
  "query": "artificial intelligence",
  "region": "IN",
  "language": "en",
  "count": 5,
  "articles": [
    {
```

```
      "title": "Example AI news headline",
      "publisher": "BBC",
      "published_at": "2026-11-11T...",
      "url": "https://news.google.com/..."
    }
  ]
}
```

The actual headlines will come from Google News.

What are we verifying?

This is a very important test because it verifies the complete application flow:

MCP Inspector

↓

MCP Server

↓

search_news tool

↓

NewsService

↓

GoogleNewsProvider

↓

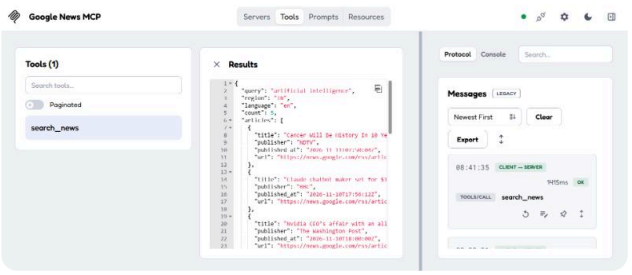
Google News RSS

Complete round trip

If this succeeds, we've proven that an external MCP client can discover and execute our Python tool and receive real news data.

Your task: Open Tools, execute `search_news` with the parameters above, and send me a screenshot of the result.

Then we'll build the remaining tools: `get_top_headlines` and `search_news_by_publisher`.



First MCP tool successfully tested!

Excellent! Your screenshot confirms that the complete MCP round trip works.

I can see:

- `Tools (1)` — our server registered one tool.
- `search_news` — successfully discovered by MCP Inspector.
- `TOOLS/CALL` — the tool was actually executed.
- `OK` — the request succeeded.
- `count: 5` — five articles were returned in structured JSON.
- Response time: approximately 1.4 seconds.

We now have a functioning Google News MCP server. This is our first major project milestone.

Step 14 — Add the second capability: Top headlines

Currently, our server can search news by keywords.
Next, we want an AI assistant to ask:

> Show me the latest headlines in India.

For this, Google News has a separate RSS endpoint:

```text
https://news.google.com/rss?hl=en-IN&gl=IN&ceid=IN:en
```

Unlike our existing search endpoint, this retrieves general headlines instead of searching for a specific keyword.

14.1 Update `google_news.py`

We'll make one small architectural improvement: both search and headline retrieval will use the same RSS-fetching and parsing method. This avoids duplicating code.
Open:
`src/google_news_mcp/providers/google_news.py`
Replace the entire file with:

```python
from datetime import datetime
from email.utils import parsedate_to_datetime

import feedparser
import httpx
from pydantic import ValidationError

from google_news_mcp.config import settings
from google_news_mcp.models.article import NewsArticle


class GoogleNewsProvider:
    """Fetches and parses news articles from Google News RSS."""
```

```
    def __init__(self) -> None:
        self.base_url = settings.google_news_base_url
        self.timeout = settings.request_timeout

    async def search(
        self,
        query: str,
        region: str = settings.default_region,
        language: str = settings.default_language,
        max_results: int = settings.default_max_results,
    ) -> list[NewsArticle]:
        """Search Google News using keywords."""

        if not query.strip():
            raise ValueError("Search query cannot be empty.")

        self._validate_max_results(max_results)

        params = {
            "q": query.strip(),
            "hl": f"{language}-{region}",
            "gl": region,
            "ceid": f"{region}:{language}",
        }

        return await self._fetch_articles(
            endpoint=f"{self.base_url}/search",
            params=params,
            max_results=max_results,
        )

    async def get_top_headlines(
        self,
        region: str = settings.default_region,
        language: str = settings.default_language,
        max_results: int = settings.default_max_results,
    ) -> list[NewsArticle]:
        """Retrieve general Google News headlines."""

        self._validate_max_results(max_results)

        params = {
            "hl": f"{language}-{region}",
            "gl": region,
            "ceid": f"{region}:{language}",
        }
```

```
        return await self._fetch_articles(
            endpoint=self.base_url,
            params=params,
            max_results=max_results,
        )

    def _validate_max_results(self, max_results: int) -> None:
        if not 1 <= max_results <= settings.maximum_results:
            raise ValueError(
                f"max_results must be between 1 and "
                f"{settings.maximum_results}."
            )

    async def _fetch_articles(
        self,
        endpoint: str,
        params: dict[str, str],
        max_results: int,
    ) -> list[NewsArticle]:
        """Fetch an RSS feed and convert entries into validated articles."""

        try:
            async with httpx.AsyncClient(
                timeout=self.timeout,
                headers={"User-Agent": "GoogleNewsMCP/0.1"},
            ) as client:
                response = await client.get(endpoint, params=params)
                response.raise_for_status()

        except httpx.HTTPError as exc:
            raise RuntimeError(
                f"Failed to retrieve Google News: {exc}"
            ) from exc

        feed = feedparser.parse(response.content)

        if feed.bozo and not feed.entries:
            raise ValueError("Google News returned an invalid RSS feed.")

        articles: list[NewsArticle] = []

        for entry in feed.entries[:max_results]:
            title = entry.get("title")
            article_url = entry.get("link")

            if not title or not article_url:
                continue
```

```
            source = entry.get("source") or {}
            publisher = source.get("title", "Unknown publisher")

            published_at: datetime | None = None
            published = entry.get("published")

            if published:
                try:
                    published_at = parsedate_to_datetime(published)
                except (TypeError, ValueError, OverflowError):
                    published_at = None

            try:
                article = NewsArticle(
                    title=title,
                    publisher=publisher,
                    published_at=published_at,
                    url=article_url,
                )
                articles.append(article)

            except ValidationError:
                continue

        return articles
```

Save the file.

14.2 Test the new provider method

Open your temporary `test_provider.py` in the project root.
Replace its contents with:

python

```python
import asyncio

from google_news_mcp.providers.google_news import GoogleNewsProvider


async def main():
    provider = GoogleNewsProvider()

    print("\nFetching top headlines...\n")

    articles = await provider.get_top_headlines(
        region="IN",
        language="en",
```

```
        max_results=5,
    )

    for index, article in enumerate(articles, start=1):
        print(f"{index}. {article.title}")
        print(f"   Publisher: {article.publisher}")
        print(f"   Published: {article.published_at}")
        print()


if __name__ == "__main__":
    asyncio.run(main())
```

Run from the project root:

```powershell
python test_provider.py
```

Expected result You should receive up to five general Google News headlines without supplying any search keywords.

Stop here and share your output.

After this works, we'll connect `get_top_headlines()` to `NewsService` and expose it as our second MCP tool in Inspector.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python test_provider.py

Fetching top headlines...

1. Several coaches of Sairang-Kolkata Express train derail at Digaru in Assam - The Hindu
   Publisher: The Hindu
   Published: 2026-11-11 03:38:56+00:00
2. "We Didn't Start Iran War": Pete Hegseth's Defence Amid Impeachment Threat - NDTV
   Publisher: NDTV
   Published: 2026-11-10 13:19:28+00:00
3. Delhi releases missing Diwali-night data; PM2.5 levels cross 3,700, over 60 times national standard - The New Indian Express
   Publisher: The New Indian Express
   Published: 2026-11-09 18:20:25+00:00
4. 'They will kill you too': Sonia Gandhi's turbulent journey in Indian politics - BBC
   Publisher: BBC
   Published: 2026-11-11 00:26:02+00:00

5. Turkey shuns Israel at Cop31 climate summit - The National
   Publisher: The National
   Published: 2026-11-10 18:13:56+00:00

Top headlines retrieval working!

Excellent! Your second Google News endpoint is working.
We have now verified two different retrieval operations:

Operation	Status
Search news by keywords	Working
Retrieve general headlines	Working

Your provider successfully retrieved headlines from The Hindu, NDTV, BBC and other publishers.
Now we need to expose this functionality through MCP.

Step 15 — Add top headlines to NewsService

Currently, `GoogleNewsProvider` can retrieve headlines, but our `NewsService` doesn't expose that functionality yet.
We need to connect the two layers.

15.1 Open `news_service.py`

Location:

```text
src/google_news_mcp/services/news_service.py
```

Replace the entire file with this updated implementation:

```python
from google_news_mcp.config import settings
from google_news_mcp.models.article import NewsArticle
from google_news_mcp.providers.google_news import GoogleNewsProvider


class NewsService:
    """Provides application-level news functionality."""

    def __init__(
        self,
        provider: GoogleNewsProvider | None = None,
    ) -> None:
        self._provider = provider or GoogleNewsProvider()
```

```python
    def _deduplicate(
        self,
        articles: list[NewsArticle],
    ) -> list[NewsArticle]:
        """Remove articles with duplicate URLs."""

        unique_articles: list[NewsArticle] = []
        seen_urls: set[str] = set()

        for article in articles:
            article_url = str(article.url).rstrip("/")

            if article_url in seen_urls:
                continue

            seen_urls.add(article_url)
            unique_articles.append(article)

        return unique_articles

    async def search_news(
        self,
        query: str,
        region: str = settings.default_region,
        language: str = settings.default_language,
        max_results: int = settings.default_max_results,
    ) -> list[NewsArticle]:
        """Search news by keywords."""

        normalized_query = query.strip()

        if not normalized_query:
            raise ValueError("Search query cannot be empty.")

        articles = await self._provider.search(
            query=normalized_query,
            region=region,
            language=language,
            max_results=max_results,
        )

        return self._deduplicate(articles)[:max_results]

    async def get_top_headlines(
        self,
        region: str = settings.default_region,
        language: str = settings.default_language,
        max_results: int = settings.default_max_results,
```

```
    ) -> list[NewsArticle]:
        """Retrieve general news headlines."""

        articles = await self._provider.get_top_headlines(
            region=region,
            language=language,
            max_results=max_results,
        )

        return self._deduplicate(articles)[:max_results]
```

Save the file.

15.2 Understand the improvement

We extracted duplicate-removal logic into one reusable method:

```python
def _deduplicate(self, articles):
```

Why?
Because both keyword searches and top headlines need deduplication.
Instead of writing the same logic twice, we reuse one method.
This is an example of the software engineering principle **DRY — Don't Repeat Yourself**.

15.3 Verify the service

Run this command from your project root:

```powershell
python -c "from google_news_mcp.services.news_service import NewsService;
print('NewsService updated successfully')"
```

Expected output:

```text
NewsService updated successfully
```

Stop here and share your terminal output.
Next, we'll register `get_top_headlines` as our second MCP tool, restart Inspector and test it alongside `search_news`.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp>
python -c "from google_news_mcp.services.news_service import NewsService;
print('NewsService updated successfully')"
NewsService updated successfully

NewsService updated successfully

Great! The service now supports both keyword searches and general headlines.
Let's expose the second capability through MCP.

Step 16 — Register `get_top_headlines` MCP tool

Open:

text

```
src/google_news_mcp/tools/news_tools.py
```

Find the existing `register_news_tools()` function.
Replace the entire file with this updated version. It includes both tools.

python

```python

from typing import Annotated

from mcp.server import MCPServer
from pydantic import Field

from google_news_mcp.models.search import NewsSearchResponse
from google_news_mcp.services.news_service import NewsService


def register_news_tools(
    mcp: MCPServer,
    news_service: NewsService,
) -> None:
    """Register Google News tools with the MCP server."""

    @mcp.tool()
    async def search_news(
        query: Annotated[
            str,
            Field(
                min_length=2,
                max_length=200,
                description="Topic or keywords to search for in Google News.",
            ),
        ],
        region: str = "IN",
        language: str = "en",
        max_results: Annotated[
            int,
            Field(ge=1, le=50),
        ] = 10,
    ) -> NewsSearchResponse:
```

```
        """Search Google News for articles matching a topic."""

        articles = await news_service.search_news(
            query=query,
            region=region.upper(),
            language=language.lower(),
            max_results=max_results,
        )

        return NewsSearchResponse(
            query=query.strip(),
            region=region.upper(),
            language=language.lower(),
            count=len(articles),
            articles=articles,
        )

    @mcp.tool()
    async def get_top_headlines(
        region: str = "IN",
        language: str = "en",
        max_results: Annotated[
            int,
            Field(ge=1, le=50),
        ] = 10,
    ) -> NewsSearchResponse:
        """Retrieve the latest general headlines from Google News."""

        articles = await news_service.get_top_headlines(
            region=region.upper(),
            language=language.lower(),
            max_results=max_results,
        )

        return NewsSearchResponse(
            query="Top headlines",
            region=region.upper(),
            language=language.lower(),
            count=len(articles),
            articles=articles,
        )
```

Save the file.

Step 16.1 — Verify the updated tools

Run:

```powershell
python -c "from google_news_mcp.tools.news_tools import register_news_tools;
print('Both MCP tools are registered in the module')"
```

Expected:

```text
Both MCP tools are registered in the module
```

Step 16.2 — Restart MCP Inspector

Stop the existing Inspector process in your VS Code terminal using:

```text
Ctrl + C
```

Then restart:

```powershell
mcp dev src/google_news_mcp/server.py
```

Reconnect to the server.

Step 16.3 — Verify both tools

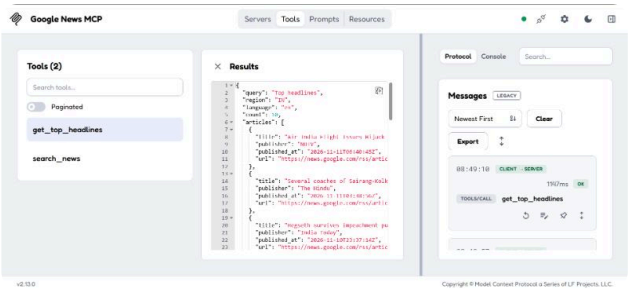
Open the Tools tab.
You should now see:



Call `get_top_headlines` with:

Parameter	Value
`region`	`IN`
`language`	`en`
`max_results`	`5`

You should receive five general headlines.
Your task: Update `news_tools.py`, restart Inspector and test the second tool. Send me the screenshot of the Tools tab and its result.
Next, we'll build our third tool: `search_news_by_publisher`, followed by MCP resources and prompts.



Second MCP tool successfully tested!

Excellent! Your screenshot confirms that everything is working.

I can see:

- `Tools (2)` — two tools are registered.
- `search_news` — working.
- `get_top_headlines` — working.
- `TOOLS/CALL` — successful execution.
- `count: 10` — ten articles returned.
- Response time: approximately 1.1 seconds.

We now have two working MCP tools.

Step 17 — Build our third MCP tool: `search_news_by_publisher`

This tool will let an AI assistant answer requests like:

- "Show me AI news published by BBC."
- "Find technology news from Reuters."
- "Get the latest banking news from CNBC."

We'll reuse our existing news service rather than build another RSS integration.

17.1 Update `news_service.py`

Open:

`src/google_news_mcp/services/news_service.py`

Add this method inside the `NewsService` class, after `get_top_headlines()` :

python

```
    async def search_news_by_publisher(
        self,
        query: str,
        publisher: str,
        region: str = settings.default_region,
        language: str = settings.default_language,
        max_results: int = settings.default_max_results,
    ) -> list[NewsArticle]:
        """Search Google News articles from a specific publisher."""
```

```
        normalized_publisher = publisher.strip()

        if not normalized_publisher:
            raise ValueError("Publisher name cannot be empty.")

        combined_query = (
            f"{query.strip()} site:{normalized_publisher}"
        )

        articles = await self._provider.search(
            query=combined_query,
            region=region,
            language=language,
            max_results=max_results,
        )

        return self._deduplicate(articles)[:max_results]
```

Important: This uses Google's RSS search syntax to restrict results to a publisher's domain. Publisher matching depends on Google News search behaviour; it is not a guaranteed exact-publisher filter.

17.2 Register the third MCP tool

Open:
`src/google_news_mcp/tools/news_tools.py`
Add this function inside `register_news_tools()`, after the existing two tool definitions:

python

```
    @mcp.tool()
    async def search_news_by_publisher(
        query: Annotated[
            str,
            Field(
                min_length=2,
                max_length=200,
                description="Topic to search for.",
            ),
        ],
        publisher: Annotated[
            str,
            Field(
                min_length=2,
                max_length=100,
                description="Publisher domain, such as bbc.com or reuters.com.",
            ),
```

```
        ],
        region: str = "IN",
        language: str = "en",
        max_results: Annotated[
            int,
            Field(ge=1, le=50),
        ] = 10,
    ) -> NewsSearchResponse:
        """Search Google News for articles from a specific publisher."""

        articles = await news_service.search_news_by_publisher(
            query=query,
            publisher=publisher,
            region=region.upper(),
            language=language.lower(),
            max_results=max_results,
        )

        return NewsSearchResponse(
            query=query.strip(),
            region=region.upper(),
            language=language.lower(),
            count=len(articles),
            articles=articles,
        )
```

Save both files.

17.3 Verify the implementation

Run:

powershell

```
python -c "from google_news_mcp.tools.news_tools import register_news_tools;
print('Three MCP tools are ready')"
```

Expected:

text

```
Three MCP tools are ready
```

17.4 Test using MCP Inspector

Restart Inspector:

powershell

```
mcp dev src/google_news_mcp/server.py
```

Reconnect and open the Tools tab.
You should now see:

Google News MCP — Available tools

`search_news` — Working

`get_top_headlines` — Working

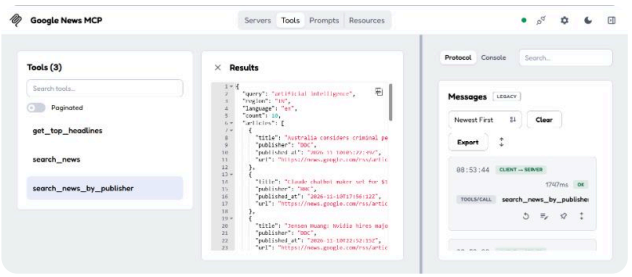
`search_news_by_publisher` — New

Test with these arguments:

Parameter	Value
`query`	`artificial intelligence`
`publisher`	`bbc.com`
`region`	`IN`
`language`	`en`
`max_results`	`5`

We should receive articles matching the topic and publisher restriction.

Your next milestone: Get all three tools visible and successfully test the third one.
After that, we'll add MCP Resources and Prompts. These will demonstrate that you understand more than just MCP tool registration—you'll have implemented all three major server-side primitives.



Third MCP tool successfully tested!

Excellent! All three MCP tools are now registered and working.
Your screenshot confirms:

- `Tools (3)` — all three tools discovered.
- `search_news` — working.
- `get_top_headlines` — working.

- `search_news_by_publisher` — successfully executed.
- `OK` — no MCP execution error.
- Returned articles show BBC as the publisher.

Our current project status

Google News MCP

3 / 3

MCP tools implemented and tested

search_news	Working
get_top_headlines	Working
search_news_by_publisher	Working

Step 18 — What should we build next?

We've completed the three primary tools. Now let's add the other two MCP primitives. MCP has three important server capabilities:

1. Tools Completed Functions an AI can execute, such as searching news.

---

2. Resources Next Data an AI application can read, such as available news categories and supported regions.

---

3. Prompts Upcoming Reusable prompt templates, such as summarising today's technology news.

What we'll implement next

MCP Resources

Resource URI	Purpose
`news://regions`	List supported country codes
`news://categories`	List available news categories
`news://about`	Server information and capabilities

MCP Prompts

Prompt	Purpose

`summarize_news`	Summarise retrieved articles
`compare_publishers`	Compare coverage across publishers
`daily_news_briefing`	Generate a daily news briefing

One important distinction: these prompts will initially be reusable templates, not AI-generated summaries. We'll connect Gemini later to perform actual summarisation.
Next step: We'll implement our first MCP Resource, `news://regions`, and test it in the Resources tab of Inspector.

okay

Perfect. Let's continue! We've finished all three MCP tools. Now we're moving to **MCP Resources**.

Step 18 — Create your first MCP Resource

First, understand the difference:

- **Tool:** The AI asks the server to perform an action, such as searching news.
- **Resource:** The AI application reads information exposed by the server, such as a list of supported countries.

Our first resource will be:
`news://regions`
When an MCP client reads this resource, it will receive a list of supported news regions.

18.1 Create the resource module

In VS Code, navigate to:

```text
src/
└── google_news_mcp/
    └── resources/
```

Inside `resources`, create a file:

```text
news_resources.py
```

Your project already has a `resources` folder, so we can use it.

18.2 Add the resource implementation

Paste this code into `news_resources.py`:

python

```python
import json

from mcp.server import MCPServer


def register_news_resources(mcp: MCPServer) -> None:
    """Register Google News resources."""

    @mcp.resource(
        "news://regions",
        name="Supported News Regions",
        description="Lists the country regions available for Google News
searches.",
        mime_type="application/json",
    )
    def get_supported_regions() -> str:
        """Return the supported Google News regions."""

        regions = {
            "regions": [
                {
                    "code": "IN",
                    "name": "India",
                    "language": "en",
                },
                {
                    "code": "US",
                    "name": "United States",
                    "language": "en",
                },
                {
                    "code": "GB",
                    "name": "United Kingdom",
                    "language": "en",
                },
                {
                    "code": "CA",
                    "name": "Canada",
                    "language": "en",
                },
                {
                    "code": "AU",
                    "name": "Australia",
                    "language": "en",
                },
```

```
            ]
        }

        return json.dumps(regions, indent=2)
```

What is happening here?

1. `@mcp.resource()` registers a resource with the MCP server.
2. `news://regions` is its unique address.
3. `get_supported_regions()` provides the resource content.
4. `json.dumps()` converts our Python dictionary into JSON text.
5. `mime_type="application/json"` tells clients that the content is JSON.

This follows the official Python MCP SDK resource registration pattern. MCP Python SDK GitHub

These five regions are our initial curated list, not an exhaustive list of every Google News region.

18.3 Register the resource in server.py

Open:

```text
src/google_news_mcp/server.py
```

Replace its contents with:

```python
from mcp.server import MCPServer

from google_news_mcp.services.news_service import NewsService
from google_news_mcp.tools.news_tools import register_news_tools
from google_news_mcp.resources.news_resources import register_news_resources


mcp = MCPServer("Google News MCP")

news_service = NewsService()

# Register MCP tools
register_news_tools(mcp, news_service)

# Register MCP resources
register_news_resources(mcp)
```

```
if __name__ == "__main__":
    mcp.run()
```

We've now connected our resource module to the main server.

18.4 Verify the implementation

Run this from your project root:

```powershell
python -c "from google_news_mcp.server import mcp; print('MCP server with resources initialized successfully')"
```

Expected output:

```text
MCP server with resources initialized successfully
```

18.5 Test in MCP Inspector

Restart the Inspector:

```powershell
mcp dev src/google_news_mcp/server.py
```

Reconnect to your server.
Then:

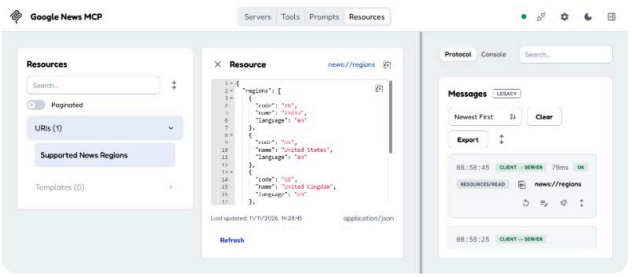
1. Open the **Resources** tab.
2. Look for `Supported News Regions`.
3. Select the resource.
4. Click **Read Resource**.

Expected resource content

```json
{
  "regions": [
    {
      "code": "IN",
      "name": "India",
      "language": "en"
    },
    {
      "code": "US",
      "name": "United States",
      "language": "en"
```

```
    },
    {
      "code": "GB",
      "name": "United Kingdom",
      "language": "en"
    },
    {
      "code": "CA",
      "name": "Canada",
      "language": "en"
    },
    {
      "code": "AU",
      "name": "Australia",
      "language": "en"
    }
  ]
}
```

Your task: Implement these two files, run the import verification, and read the resource through Inspector.
Once this works, we'll create `news://categories` and `news://about` before moving to MCP Prompts.



First MCP Resource successfully tested!

Excellent! Your screenshot confirms that `news://regions` is registered and working.
I can verify:

- **Resources (1)** — resource discovered.
- **Supported News Regions** — correctly registered.
- **URI:** `news://regions`
- **Content type:** `application/json`
- **RESOURCES/READ:** successful.
- **Response time:** 78 ms.

Your MCP server now supports both Tools and Resources.

Step 19 — Add two more MCP Resources

Let's add:

1. `news://categories` — available news categories.
2. `news://about` — information about our MCP server.

Open:
`src/google_news_mcp/resources/news_resources.py`
Replace the existing code with this complete version:

python

```python
import json

from mcp.server import MCPServer


def register_news_resources(mcp: MCPServer) -> None:
    """Register Google News resources."""

    @mcp.resource(
        "news://regions",
        name="Supported News Regions",
        description="Lists supported Google News regions.",
        mime_type="application/json",
    )
    def get_supported_regions() -> str:
        regions = {
            "regions": [
                {"code": "IN", "name": "India", "language": "en"},
                {"code": "US", "name": "United States", "language": "en"},
                {"code": "GB", "name": "United Kingdom", "language": "en"},
                {"code": "CA", "name": "Canada", "language": "en"},
                {"code": "AU", "name": "Australia", "language": "en"},
            ]
        }

        return json.dumps(regions, indent=2)

    @mcp.resource(
        "news://categories",
        name="News Categories",
        description="Lists suggested news search categories.",
        mime_type="application/json",
    )
    def get_news_categories() -> str:
        categories = {
            "categories": [
```

```
                {"id": "technology", "name": "Technology"},
                {"id": "business", "name": "Business"},
                {"id": "politics", "name": "Politics"},
                {"id": "science", "name": "Science"},
                {"id": "health", "name": "Health"},
                {"id": "sports", "name": "Sports"},
                {"id": "entertainment", "name": "Entertainment"},
                {"id": "world", "name": "World News"},
            ]
        }

        return json.dumps(categories, indent=2)

    @mcp.resource(
        "news://about",
        name="Google News MCP Information",
        description="Provides information about this MCP server.",
        mime_type="application/json",
    )
    def get_server_information() -> str:
        information = {
            "name": "Google News MCP",
            "version": "0.1.0",
            "description": "An MCP server for retrieving Google News articles.",
            "data_source": "Google News RSS",
            "capabilities": {
                "tools": [
                    "search_news",
                    "get_top_headlines",
                    "search_news_by_publisher",
                ],
                "resources": [
                    "news://regions",
                    "news://categories",
                    "news://about",
                ],
            },
        }

        return json.dumps(information, indent=2)
```

Step 19.1 — Verify

Run:

```powershell
python -c "from google_news_mcp.server import mcp; print('Three MCP resources
```

```
registered successfully')"
```

Expected:

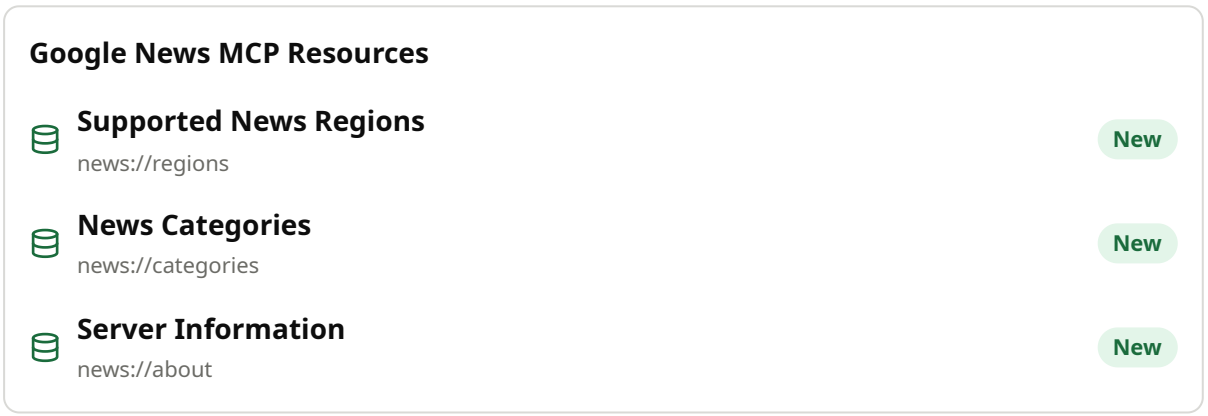
```text
Three MCP resources registered successfully
```

Step 19.2 — Test in MCP Inspector

Restart:

```powershell
mcp dev src/google_news_mcp/server.py
```

Reconnect and open Resources.
You should now see:



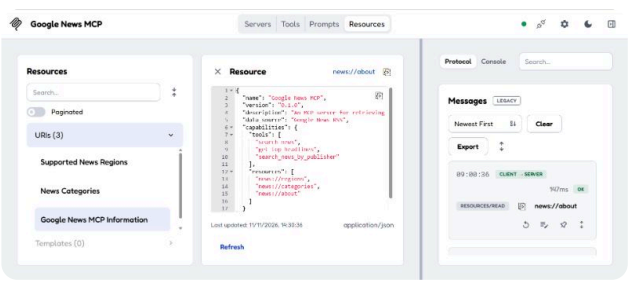
Read each resource and verify its JSON content.

What's next?

After resources, we'll implement MCP Prompts:

- `daily_news_briefing`
- `summarize_news`
- `compare_publishers`

These will demonstrate reusable prompt templates in your server.
Your task: Add the two resources, restart Inspector, and share a screenshot showing all three resources.



All 3 MCP Resources successfully verified!

Excellent! Your screenshot confirms that all three resources are registered and accessible.

I can see:

Resource	Status
`news://regions`	Working
`news://categories`	Working
`news://about`	Working

The `news://about` resource also correctly reports all three tools and all three resource URIs.

Our project now has **3 Tools + 3 Resources**.

Step 20 — Implement MCP Prompts

Now we'll implement the third major MCP primitive: Prompts.

Think of prompts as reusable instructions that an AI client can request from our server. For example, instead of manually writing instructions every morning, an AI assistant can request our `daily_news_briefing` prompt.

We'll implement three prompts:

1. `daily_news_briefing` — instructions for preparing a daily news briefing.
2. `summarize_news` — instructions for summarising news articles.
3. `compare_publishers` — instructions for comparing coverage from different publishers.

20.1 Create the prompt module

Navigate to:

```text
src/
└── google_news_mcp/
    └── prompts/
```

Create:

```text
news_prompts.py
```

20.2 Add the implementation

```python
from mcp.server import MCPServer
from mcp.server.fastmcp.prompts import base
```

```python
def register_news_prompts(mcp: MCPServer) -> None:
    """Register reusable Google News prompts."""

    @mcp.prompt()
    def daily_news_briefing(
        region: str = "India",
        focus: str = "Technology, Business and World News",
    ) -> list[base.Message]:
        """Create instructions for a daily news briefing."""

        return [
            base.UserMessage(
                f"""
                Prepare a daily news briefing for {region}.

                Focus areas: {focus}.

                Organize the briefing into clear sections.

                For each important story:
                - Explain what happened.
                - Explain why it matters.
                - Include the publisher and publication date.
                - Include the original article URL.

                Prioritize factual reporting.
                Clearly distinguish confirmed facts from speculation.
                """
            )
        ]

    @mcp.prompt()
    def summarize_news(
        articles: str,
        style: str = "concise",
    ) -> list[base.Message]:
        """Generate instructions to summarize news articles."""

        return [
            base.UserMessage(
                f"""
                Summarize the following news articles.

                Summary style: {style}.

                Instructions:
                - Identify the main event.
                - Explain the important facts.
```

```
                - Highlight important names, organizations and dates.
                - Avoid repeating information.
                - Do not invent missing information.
                - Preserve article source references.

                Articles:
                {articles}
                """
            )
        ]

    @mcp.prompt()
    def compare_publishers(
        topic: str,
        publisher_a: str,
        publisher_b: str,
    ) -> list[base.Message]:
        """Generate instructions to compare two publishers' coverage."""

        return [
            base.UserMessage(
                f"""
                Compare how two publishers cover the following topic:

                Topic: {topic}

                Publisher A: {publisher_a}
                Publisher B: {publisher_b}

                Analyze:
                - Main angle of each publisher.
                - Facts emphasized by each publisher.
                - Differences in framing and perspective.
                - Information common to both reports.
                - Important information reported by only one publisher.

                Do not assume that differences in coverage indicate bias.
                Distinguish factual differences from differences in emphasis.
                """
            )
        ]
```

20.3 Register prompts in `server.py`

Add this import:

```python
from google_news_mcp.prompts.news_prompts import register_news_prompts
```

Then add this line after your resource registration:

```python
register_news_prompts(mcp)
```

Your registration section should now look like:

```python
register_news_tools(mcp, news_service)
register_news_resources(mcp)
register_news_prompts(mcp)
```

20.4 Verify the implementation

Run:

```powershell
python -c "from google_news_mcp.server import mcp; print('MCP server with tools, resources and prompts initialized successfully')"
```

Expected:

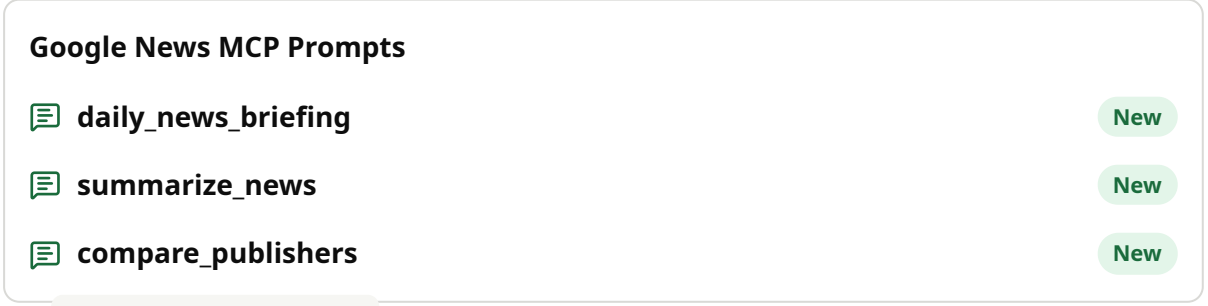
```text
MCP server with tools, resources and prompts initialized successfully
```

20.5 Test in MCP Inspector

Restart:

```powershell
mcp dev src/google_news_mcp/server.py
```

Reconnect and open the **Prompts** tab.
You should find:



Try `daily_news_briefing` with:

- `region` : India
- `focus` : Artificial Intelligence and Technology

Click **Get Prompt** and inspect the returned messages.
Important: These prompts provide instructions to the connected AI client. They do not independently call Gemini or fetch news. We'll add that integration later.

Your next milestone is to get all three prompts visible in Inspector. Share your screenshot once they're working!

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> from google_news_mcp.prompts.news_prompts import register_news_prompts
At line:1 char:1

- from google_news_mcp.prompts.news_prompts import register_news_prompt ...
-

The 'from' keyword is not supported in this version of the language.
+ CategoryInfo : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ReservedKeywordNotAllowed

No problem! You're trying to execute **Python code directly in PowerShell**. That's why you're getting this error.

```text
The 'from' keyword is not supported in this version of the language.
```

PowerShell understands PowerShell commands, not Python import statements.
You need to add that import to your **`server.py`** **file**, not execute it in the terminal.

Step 1 — Open `server.py`

Navigate to:

```text
src/google_news_mcp/server.py
```

Step 2 — Add this import at the top

```python
from google_news_mcp.prompts.news_prompts import register_news_prompts
```

Step 3 — Register the prompts

Find this section:

```python
register_news_tools(mcp, news_service)
register_news_resources(mcp)
```

Add:

```python
register_news_prompts(mcp)
```

Your complete `server.py` should look like:

```python

from mcp.server import MCPServer

from google_news_mcp.services.news_service import NewsService
from google_news_mcp.tools.news_tools import register_news_tools
from google_news_mcp.resources.news_resources import register_news_resources
from google_news_mcp.prompts.news_prompts import register_news_prompts


mcp = MCPServer("Google News MCP")

news_service = NewsService()

register_news_tools(mcp, news_service)
register_news_resources(mcp)
register_news_prompts(mcp)


if __name__ == "__main__":
    mcp.run()
```

Step 4 — Verify using PowerShell

Now execute this command in your terminal:

```powershell
python -c "from google_news_mcp.server import mcp; print('MCP server initialized successfully')"
```

This is the correct way to execute a Python import test from PowerShell.

Remember:

- Python code goes inside `.py` files.
- PowerShell commands go inside the terminal.

- To execute a Python statement directly in PowerShell, use `python -c "..."`.

Run the verification command and share the output.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from google_news_mcp.server import mcp; print('MCP server initialized successfully')"
Traceback (most recent call last):
File "", line 1, in
from google_news_mcp.server import mcp; print('MCP server initialized successfully')
^^^

```python
def register_news_prompts(mcp: MCPServer) -> None:
    """Register reusable Google News prompts."""

    @mcp.prompt()
    def daily_news_briefing(
        region: str = "India",
        focus: str = "Technology, Business and World News",
    ) -> str:
        """Create instructions for a daily news briefing."""

        return f"""
        Prepare a daily news briefing for {region}.

        Focus areas: {focus}.

        Organize the briefing into clear sections.

        For each important story:
        - Explain what happened.
        - Explain why it matters.
        - Include the publisher and publication date.
        - Include the original article URL.

        Prioritize factual reporting.
        Clearly distinguish confirmed facts from speculation.
        """

    @mcp.prompt()
    def summarize_news(
        articles: str,
        style: str = "concise",
    ) -> str:
        """Generate instructions to summarize news articles."""

        return f"""
        Summarize the following news articles.

        Summary style: {style}.

        Instructions:
        - Identify the main event.
        - Explain the important facts.
        - Highlight important names, organizations and dates.
        - Avoid repeating information.
        - Do not invent missing information.
        - Preserve article source references.
```

```
        Articles:
        {articles}
        """

    @mcp.prompt()
    def compare_publishers(
        topic: str,
        publisher_a: str,
        publisher_b: str,
    ) -> str:
        """Generate instructions to compare two publishers' coverage."""

        return f"""
        Compare how two publishers cover the following topic.

        Topic: {topic}

        Publisher A: {publisher_a}
        Publisher B: {publisher_b}

        Analyze:
        - Main angle of each publisher.
        - Facts emphasized by each publisher.
        - Differences in framing and perspective.
        - Information common to both reports.
        - Information reported by only one publisher.

        Do not assume that differences in coverage indicate bias.
        Distinguish factual differences from differences in emphasis.
        """
```

What changed?
Instead of importing `base.Message` and constructing `UserMessage` objects, we're returning ordinary Python strings.
The SDK automatically converts a returned string into a user message. This is supported by its official v2 prompt API. MCP Python SDK

Step 2 — Verify the server

Save the file and run:

powershell

```
python -c "from google_news_mcp.server import mcp; print('MCP server initialized successfully')"
```

Expected:

```text
MCP server initialized successfully
```

Step 3 — Test the prompts

If verification succeeds, restart Inspector:

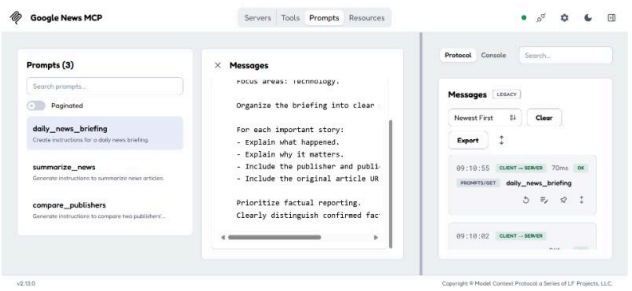
```powershell
mcp dev src/google_news_mcp/server.py
```

Open the **Prompts** tab.
You should see:

- `daily_news_briefing`
- `summarize_news`
- `compare_publishers`

Try `daily_news_briefing` with India and Technology as arguments.

One important lesson: We should always use SDK-version-compatible documentation rather than copying older MCP tutorials. Understanding API migrations is itself a useful engineering skill.
Send me the verification output. We'll proceed only after it succeeds.



All 3 MCP Prompts successfully registered!
Excellent! Your screenshot confirms that all three prompts are available:

MCP Prompt	Status
`daily_news_briefing`	Working
`summarize_news`	Registered
`compare_publishers`	Registered

I can also see that `daily_news_briefing` successfully returned its prompt messages.
We now have all three major MCP primitives implemented:

3	3	3
Tools	**Resources**	**Prompts**
Search and retrieve news	Provide server information	Reusable instructions

9 MCP capabilities registered

Step 21 — Test the other two prompts

Before moving forward, let's verify the remaining prompts.

Test 1: `summarize_news`

Open the Prompts tab, select `summarize_news`, and enter:

Parameter	Value
`articles`	`AI is transforming software development. Companies are investing in AI infrastructure.`
`style`	`concise`

Click **Get Prompt**.

Test 2: `compare_publishers`

Select `compare_publishers` and enter:

Parameter	Value
`topic`	`Artificial Intelligence`
`publisher_a`	`BBC`
`publisher_b`	`Reuters`

Click **Get Prompt**.

Both should return instruction messages.

Step 22 — Save our progress to GitHub

Once both prompts work, open your VS Code terminal and run:

```powershell
git add .
git commit -m "feat: implement Google News MCP tools resources and prompts"
git push
```

This saves our first complete MCP implementation to GitHub.

What comes next?

Next major phase

Gemini API integration

We'll build a separate Python AI client that:

1. Connects to our MCP server.
2. Discovers available MCP tools.
3. Passes those tools to Gemini.
4. Lets Gemini decide which tool to invoke.
5. Executes the selected MCP tool.
6. Sends the result back to Gemini.
7. Displays the final natural-language response.

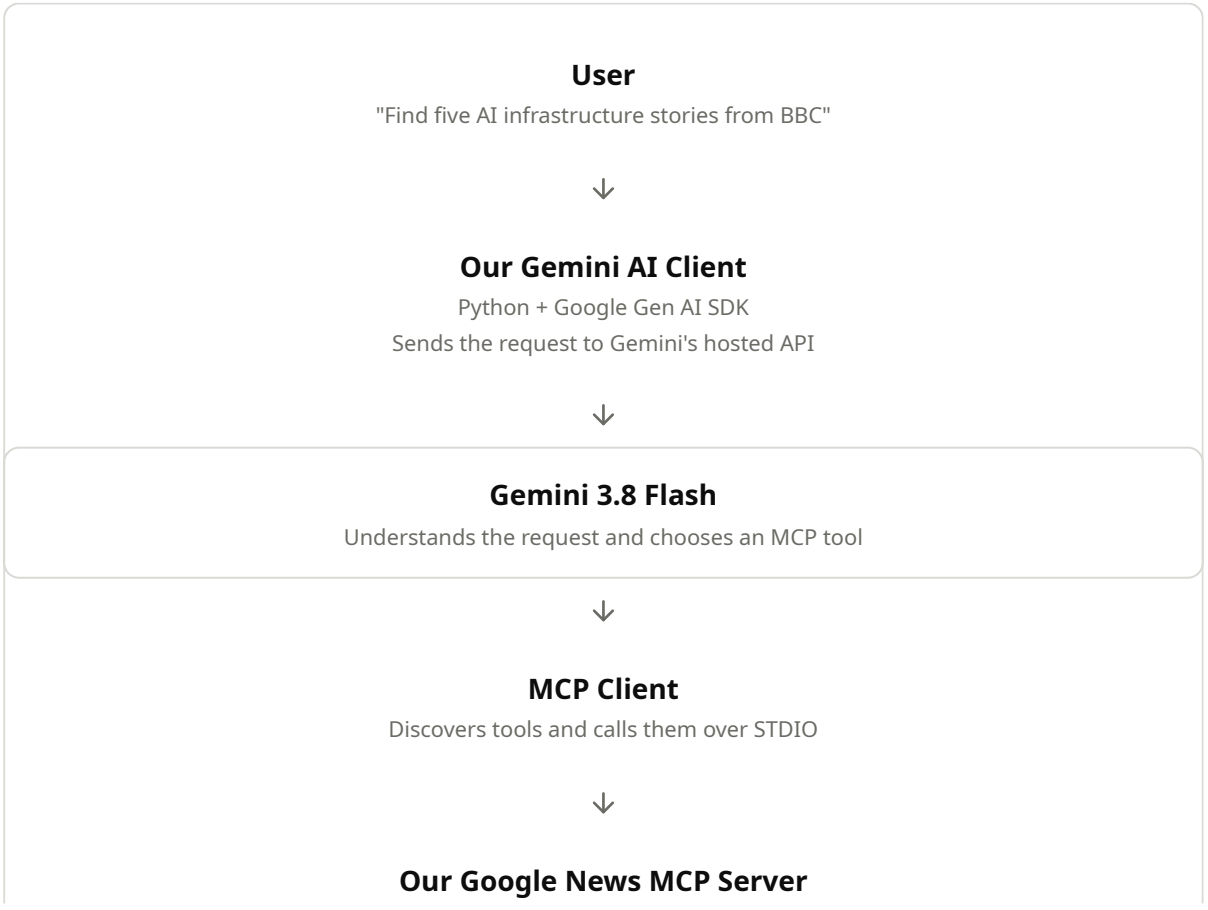
This is where we'll demonstrate something particularly valuable: **an AI model autonomously selecting and using tools exposed by an MCP server.**
First, verify the two remaining prompts and save our progress.

yes go a head

Great. Now we're entering the next major phase: **connecting Gemini to our Google News MCP server.**
So far, we've built an MCP server that works independently. Now we'll build a separate AI client that connects to it and allows Gemini to discover and invoke its tools.

Step 22 — Design the Gemini integration

First, let's understand what we're building.



search_news_by_publisher → Google News RSS

↑

Gemini generates the final answer

Summarizes retrieved headlines with publisher and links

The important design decision is that **Gemini will not be embedded inside our MCP server**. We'll build a separate client application.
This keeps our MCP server usable by Claude, ChatGPT, Gemini and other compatible hosts.

Which Gemini model?

We'll use `gemini-3.8-flash` .
Google currently lists this as a stable model for agentic workflows, and its pricing documentation lists a free tier with free input and output tokens, subject to usage limits.
Google AI Google AI
We'll use the official `google-genai` Python SDK. googleapis.github.io googleapis.github.io

Step 22.1 — Install the Gemini SDK

Open `pyproject.toml` .
Find your existing section:

toml

```toml
[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-asyncio>=0.24",
    "respx>=0.22",
    "ruff>=0.14",
]
```

Add an `ai` group below `dev` :

toml

```toml
ai = [
    "google-genai>=1.0.0",
    "python-dotenv>=1.0.0",
]
```

Your optional dependencies section should now contain both `dev` and `ai` .
Why are we creating a separate `ai` group?
Our MCP server should not require Gemini to function. The Gemini SDK is only needed when running our demonstration AI client.
This is called optional dependency management.

Install the new dependencies

Make sure `(.venv)` appears in your terminal, then run:

```powershell
python -m pip install -e ".[ai]"
```

Verify the installation

Run:

```powershell
python -c "from google import genai; import dotenv; print('Google Gen AI SDK installed successfully')"
```

Expected output:

```text
Google Gen AI SDK installed successfully
```

What we're going to implement after this

Our new project structure will eventually include:

```text
google-news-mcp/
│
├── src/
│   └── google_news_mcp/
│       ├── server.py
│       ├── config.py
│       ├── models/
│       ├── providers/
│       ├── services/
│       ├── tools/
│       ├── resources/
│       └── prompts/
│
├── examples/
│   └── gemini_client.py
│
├── tests/
│
├── .env
├── .env.example
├── .gitignore
├── pyproject.toml
└── README.md
```

We'll use the MCP SDK v2 `Client` to connect to our server, discover its tool schemas, and execute tool calls. We'll translate those schemas into Gemini function declarations and implement the function-calling loop ourselves. This also avoids depending on Google's currently experimental direct MCP integration. MCP Python SDK googleapis.github.io

Your task now: Update `pyproject.toml`, install the optional AI dependencies, and share the verification output.
Don't create your API key or write the Gemini client code just yet. We'll do those next.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from google import genai; import dotenv; print('Google Gen AI SDK installed successfully')"
Google Gen AI SDK installed successfully

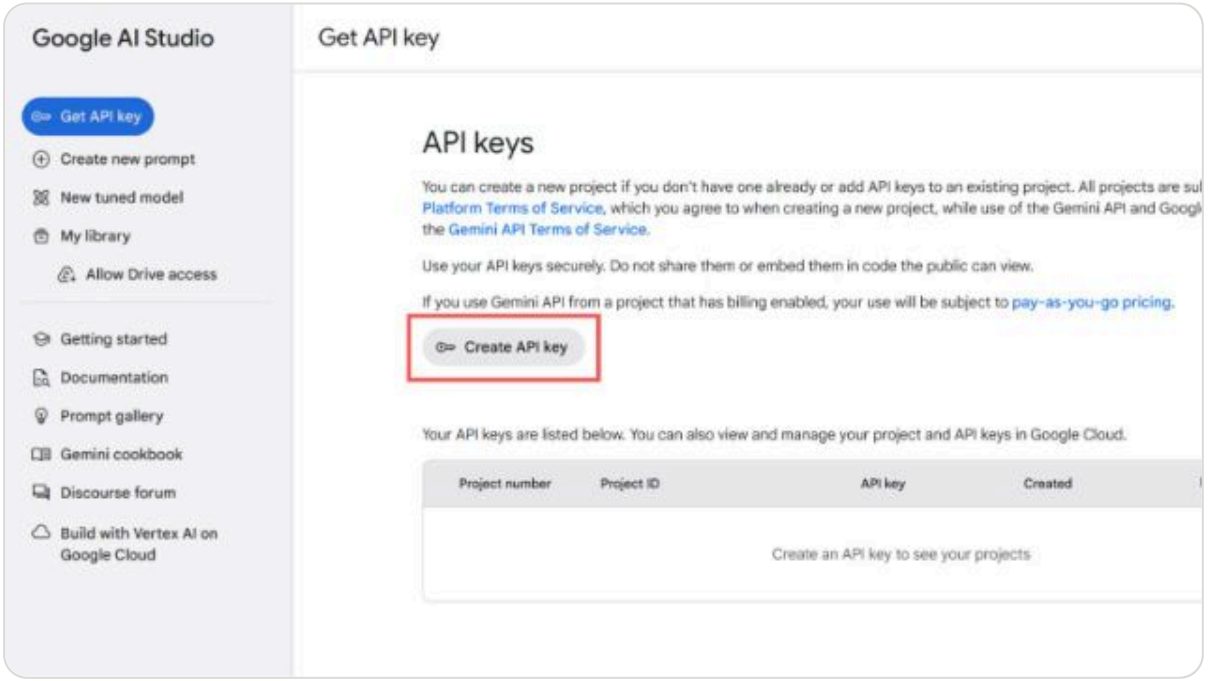
Gemini SDK installed successfully
Perfect! Your Gemini SDK and environment-variable package are installed and ready. Now we'll prepare the Gemini API connection.

Step 23 — Get your free Gemini API key

We'll use `gemini-3.8-flash` as our initial model. Google's current pricing documentation lists it with free-tier input and output tokens, subject to availability and usage limits. Google AI

23.1 Generate an API key

1. Open Google AI Studio.
2. Sign in with your Google account.
3. Select **Get API key**.
4. Click **Create API key**.
5. Select or create a Google Cloud project if requested.
6. Copy your API key.



Security Never paste your API key into Python source code, GitHub or this chat.

Step 24 — Prepare environment variables

We will store the key in a `.env` file.

24.1 Update `.gitignore`

Open your existing `.gitignore`.
Find:

```gitignore
.env
.env.*
```

Add this line immediately underneath:

```gitignore
!.env.example
```

This ensures the actual API key stays private while the example configuration can be committed to GitHub.

24.2 Create `.env`

In your project root, create:

```text
.env
```

Add:

```env
GEMINI_API_KEY=paste_your_actual_api_key_here
GEMINI_MODEL=gemini-3.8-flash
```

Replace the placeholder with your actual API key.

24.3 Create `.env.example`

Create another file:

```text
.env.example
```

Add:

```env
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-3.8-flash
```

This file will be safe to publish because it contains no real credentials.

Step 25 — Verify the API key is loaded

From your project root, with `.venv` activated, execute:

```powershell
python -c "from dotenv import load_dotenv; import os; load_dotenv(); print('Gemini API key loaded:', bool(os.getenv('GEMINI_API_KEY'))); print('Gemini model:', os.getenv('GEMINI_MODEL'))"
```

Expected output:

```text
Gemini API key loaded: True
Gemini model: gemini-3.8-flash
```

This only verifies that the key is present in your environment. We'll verify the actual API connection in the next step.

What we'll build next

We'll create a separate demonstration client:

```text
google-news-mcp/
│
├── src/
│   └── google_news_mcp/
│       └── server.py
│
├── examples/
│   └── gemini_client.py
│
├── .env
└── .env.example
```

The Gemini client will:

1. Connect to our MCP server.
2. Discover all three tools dynamically.
3. Give their schemas to Gemini.
4. Execute Gemini's requested tool calls.
5. Return tool results to Gemini.
6. Generate the final answer.

Your task: Generate your API key, create `.env` and `.env.example`, and share the verification output. Don't share the key itself.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from dotenv import load_dotenv; import os; load_dotenv(); print('Gemini API key loaded:', bool(os.getenv('GEMINI_API_KEY'))); print('Gemini model:',

os.getenv('GEMINI_MODEL'))"
Gemini API key loaded: True
Gemini model: gemini-3.8-flash

Gemini environment configuration verified

Excellent! Your API key is loaded, and our model configuration is being read correctly. Our configured model, `gemini-3.8-flash`, is a valid stable Gemini API model. Google's current pricing page lists free-tier input and output tokens, subject to usage limits. Free-tier data may be used to improve Google's products, so keep that in mind when sending data. Google AI Google AI

Step 26 — Test the actual Gemini API connection

Before building the MCP client, let's verify that your API key can successfully communicate with Gemini.
Run this command from your project root:

```powershell
python -c "import os; from dotenv import load_dotenv; from google import genai;
load_dotenv(); client = genai.Client(api_key=os.environ['GEMINI_API_KEY']);
response = client.models.generate_content(model=os.getenv('GEMINI_MODEL'),
contents='Reply with exactly: Gemini API connection successful.');
print(response.text)"
```

What does this do?

1. Loads your API key from `.env`.
2. Initializes the Google Gen AI client.
3. Sends a small request to Gemini.
4. Prints the model's response.

Expected output

```text
Gemini API connection successful.
```

This uses Google's official `google-genai` SDK.
Do not share your API key or `.env` contents.

What we'll build after this test

Our next file will be:

```text
examples/
└── gemini_client.py
```

This will be a real MCP client that:

- Starts our Google News MCP server as a subprocess.
- Discovers its three tools automatically.
- Converts MCP tool definitions into Gemini function declarations.
- Lets Gemini select the appropriate tool.
- Executes the tool through MCP.
- Returns the actual news results to Gemini.
- Displays Gemini's final response.

Run the connection test and share the output. Then we'll build the actual AI agent loop.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "import os; from dotenv import load_dotenv; from google import genai; load_dotenv(); client = genai.Client(api_key=os.environ['GEMINI_API_KEY']); response = client.models.generate_content(model=os.getenv('GEMINI_MODEL'), contents='Reply with exactly: Gemini API connection successful.'); print(response.text)"
Gemini API connection successful.

Gemini API connection verified

Now we can build the actual AI integration.
One important step before connecting Gemini: we need to build a proper **MCP client**.
Remember, so far we've only built the MCP server.
Our client must be able to connect to the server, discover its tools, and call them programmatically. The official Python SDK v2 provides `Client` and `StdioServerParameters` for exactly this purpose. MCP Python SDK MCP Python SDK

Step 27 — Create our Gemini client

Our repository will now have a separate `examples/` directory.
From your project root, run:

```powershell
New-Item -ItemType Directory -Force examples
```

Inside `examples`, create:

```text
gemini_client.py
```

Step 27.1 — Implement the MCP client connection

For now, let's establish the MCP connection and dynamically discover the tools. We'll connect Gemini in the next step.
Paste this into `examples/gemini_client.py`:

python

```python
import asyncio
import sys

from mcp import Client, StdioServerParameters


async def main() -> None:
    """Connect to Google News MCP and discover available tools."""

    server = StdioServerParameters(
        command=sys.executable,
        args=["-m", "google_news_mcp.server"],
    )

    print("Connecting to Google News MCP server...")

    async with Client(server) as client:
        print("\nConnected successfully!")

        # Discover tools from the MCP server.
        result = await client.list_tools()

        print("\nAvailable MCP tools:")

        for tool in result.tools:
            print(f"\nTool: {tool.name}")
            print(f"Description: {tool.description}")
            print(f"Input schema: {tool.input_schema}")

    print("\nMCP connection closed.")


if __name__ == "__main__":
    asyncio.run(main())
```

Step 27.2 — Understand what we're doing

There are three important things here.

1. Starting the MCP server as a subprocess

python

```python
command=sys.executable,
args=["-m", "google_news_mcp.server"]
```

Instead of manually opening another terminal and starting the server, our client launches it automatically using the Python interpreter from `.venv`.

2. Establishing an MCP connection

```python
async with Client(server) as client:
```

This establishes the connection and manages its lifecycle. When the block exits, the client shuts down the subprocess.

3. Discovering tools dynamically

```python
result = await client.list_tools()
```

This asks the MCP server which tools are available.
We aren't hardcoding `search_news`, `get_top_headlines` or `search_news_by_publisher` into the client. We discover them directly from the server. This is an important feature of MCP.

Step 27.3 — Run the client

Make sure:

- Your terminal is in the project root.
- `(.venv)` is visible.
- You have saved `gemini_client.py`.

Execute:

```powershell
python examples/gemini_client.py
```

Expected output

```text
Connecting to Google News MCP server...

Connected successfully!

Available MCP tools:

Tool: search_news
...

Tool: get_top_headlines
...
```

```
Tool: search_news_by_publisher
...

MCP connection closed.
```

The SDK handles the STDIO connection and MCP protocol communication for us.

MCP Python SDK MCP Python SDK

Your task: Run `python examples/gemini_client.py` and share the terminal output. Once this works, we'll connect Gemini to the dynamically discovered MCP tools and implement the actual AI tool-calling loop.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> New-Item -ItemType Directory -Force examples

```
Directory: C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp
```

Mode LastWriteTime Length Name

d----- 11-11-2026 15:23 examples

Perfect! The `examples` folder has been created successfully. Done
Now we'll create our first MCP client.

Step 27 — Create `gemini_client.py`

In VS Code Explorer:

1. Expand the `examples` folder.
2. Right-click it and select **New File**.
3. Name it `gemini_client.py`.

Your structure should look like:

text

```
google-news-mcp/
├── src/
│   └── google_news_mcp/
│       └── server.py
│
├── examples/
│   └── gemini_client.py
│
├── tests/
├── .env
├── .env.example
└── pyproject.toml
```

Step 27.1 — Add the MCP client code

Paste this into `gemini_client.py` :

python

```python
import asyncio
import sys

from mcp import Client, StdioServerParameters


async def main() -> None:
    """Connect to Google News MCP and discover its tools."""

    server = StdioServerParameters(
        command=sys.executable,
        args=["-m", "google_news_mcp.server"],
    )

    print("Connecting to Google News MCP server...")

    async with Client(server) as client:
        print("\nConnected successfully!")

        # Discover available MCP tools.
        result = await client.list_tools()

        print("\nAvailable MCP tools:")

        for tool in result.tools:
            print(f"\nName: {tool.name}")
            print(f"Description: {tool.description}")
            print(f"Input schema: {tool.input_schema}")

    print("\nMCP connection closed.")


if __name__ == "__main__":
    asyncio.run(main())
```

Save using `Ctrl + S` .

Step 27.2 — Understand what this code does

1. Start the MCP server automatically

```python
command=sys.executable,
args=["-m", "google_news_mcp.server"],
```

This launches your existing MCP server as a separate process using the same Python interpreter inside `.venv`.

2. Connect to the server

```python
async with Client(server) as client:
```

This establishes an MCP connection over STDIO and manages the connection lifecycle.

3. Discover tools

```python
result = await client.list_tools()
```

This retrieves the tools registered by your server.

The important part is that we're discovering them dynamically. We're not manually listing the three tool names in the client.

These are supported patterns in the official Python SDK v2. MCP Python SDK MCP Python SDK

Step 27.3 — Execute the client

Close MCP Inspector first by pressing `Ctrl + C` in the terminal where it is running.

Return to your project root and execute:

```powershell
python examples/gemini_client.py
```

Expected output

```text
Connecting to Google News MCP server...

Connected successfully!

Available MCP tools:

Name: search_news
...

Name: get_top_headlines
...
```

```
Name: search_news_by_publisher
...

MCP connection closed.
```

Your task: Create the file, paste the code and run the command.
Once this works, we'll give Gemini the discovered tool definitions and implement the actual AI tool-calling loop.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
Connecting to Google News MCP server...

Connected successfully!

Available MCP tools:

Name: search_news
Description: Search Google News for articles matching a topic.
Input schema: {'type': 'object', 'properties': {'query': {'description': 'Topic or keywords to search for in Google News.', 'maxLength': 200, 'minLength': 2, 'title': 'Query', 'type': 'string'}, 'region': {'default': 'IN', 'title': 'Region', 'type': 'string'}, 'language': {'default': 'en', 'title': 'Language', 'type': 'string'}, 'max_results': {'default': 10, 'maximum': 50, 'minimum': 1, 'title': 'Max Results', 'type': 'integer'}}, 'required': ['query'], 'title': 'search_newsArguments'}

Name: get_top_headlines
Description: Retrieve the latest general headlines from Google News.
Input schema: {'type': 'object', 'properties': {'region': {'default': 'IN', 'title': 'Region', 'type': 'string'}, 'language': {'default': 'en', 'title': 'Language', 'type': 'string'}, 'max_results': {'default': 10, 'maximum': 50, 'minimum': 1, 'title': 'Max Results', 'type': 'integer'}}, 'title': 'get_top_headlinesArguments'}

Name: search_news_by_publisher
Description: Search Google News for articles from a specific publisher.
Input schema: {'type': 'object', 'properties': {'query': {'description': 'Topic to search for.', 'maxLength': 200, 'minLength': 2, 'title': 'Query', 'type': 'string'}, 'publisher': {'description': 'Publisher domain, such as bbc.com or reuters.com.', 'maxLength': 100, 'minLength': 2, 'title': 'Publisher', 'type': 'string'}, 'region': {'default': 'IN', 'title': 'Region', 'type': 'string'}, 'language': {'default': 'en', 'title': 'Language', 'type': 'string'}, 'max_results': {'default': 10, 'maximum': 50, 'minimum': 1, 'title': 'Max Results', 'type': 'integer'}}, 'required': ['query', 'publisher'], 'title': 'search_news_by_publisherArguments'}

MCP connection closed.

MCP client connection successful

Fantastic! This is an important milestone. Your output confirms that our independent Python MCP client can:

1. Launch the Google News MCP server.
2. Establish an MCP connection.
3. Discover all three tools dynamically.
4. Retrieve their descriptions.
5. Retrieve their JSON input schemas.
6. Close the connection cleanly.

Notice that we didn't hardcode the three tool definitions in our client. This is exactly what we want.

Step 28 — Connect Gemini to our MCP tools

Now we'll give Gemini the tools it has discovered.
The architecture will be:

```text
MCP Server
    |
    | list_tools()
    v
MCP Client
    |
    | Convert MCP schemas
    v
Gemini Function Declarations
    |
    v
Gemini API
    |
    v
Gemini selects a tool
```

Google's Gen AI SDK supports defining functions using JSON schemas and retrieving the model's requested function calls. We'll use that rather than hardcoding individual Python functions. googleapis.github.io Google Gen AI SDK ...

28.1 Update `gemini_client.py`

Open:

```text
examples/gemini_client.py
```

Replace the existing implementation with this:

python

```python
import asyncio
import os
import sys

from dotenv import load_dotenv
from google import genai
from google.genai import types
from mcp import Client as MCPClient
from mcp import StdioServerParameters


load_dotenv()


def convert_mcp_tools_to_gemini(mcp_tools):
    """Convert MCP tool definitions into Gemini function declarations."""

    function_declarations = []

    for tool in mcp_tools:
        declaration = types.FunctionDeclaration(
            name=tool.name,
            description=tool.description or tool.name,
            parameters_json_schema=tool.input_schema,
        )

        function_declarations.append(declaration)

    return [
        types.Tool(function_declarations=function_declarations)
    ]


async def main() -> None:
    """Connect Gemini to our dynamically discovered MCP tools."""

    api_key = os.getenv("GEMINI_API_KEY")
    model = os.getenv("GEMINI_MODEL", "gemini-3.8-flash")

    if not api_key:
        raise RuntimeError("GEMINI_API_KEY is missing from .env")

    server = StdioServerParameters(
        command=sys.executable,
        args=["-m", "google_news_mcp.server"],
```

```
    )

    print("Connecting to Google News MCP server...")

    async with MCPClient(server) as mcp_client:
        print("MCP connection established.")

        # Discover tools from the MCP server.
        mcp_result = await mcp_client.list_tools()

        print(f"Discovered {len(mcp_result.tools)} MCP tools.")

        # Convert discovered tools into Gemini-compatible declarations.
        gemini_tools = convert_mcp_tools_to_gemini(
            mcp_result.tools
        )

        print("\nGemini function declarations created.")

        # Initialize the asynchronous Gemini client.
        async with genai.Client(api_key=api_key).aio as gemini_client:

            prompt = (
                "Find five recent articles about AI infrastructure "
                "published by BBC. Use the appropriate available "
                "Google News tool. Do not invent article details."
            )

            print("\nSending request to Gemini...")

            response = await gemini_client.models.generate_content(
                model=model,
                contents=prompt,
                config=types.GenerateContentConfig(
                    tools=gemini_tools,
                    temperature=0.2,
                ),
            )

            print("\nGemini response:")

            # Inspect whether Gemini requested an MCP tool.
            if response.function_calls:
                for function_call in response.function_calls:
                    print(f"\nSelected tool: {function_call.name}")
                    print(f"Arguments: {function_call.args}")
            else:
                print(response.text)
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Save the file.

28.2 Understand what we just implemented

There are three important concepts.

A. MCP schema conversion

```python
parameters_json_schema=tool.input_schema
```

Our MCP server already provides JSON schemas for its tools.
We convert those schemas into Gemini function declarations.
For example, Gemini will discover that `search_news_by_publisher` accepts:

```json
{
  "query": "string",
  "publisher": "string",
  "region": "string",
  "language": "string",
  "max_results": "integer"
}
```

The schema is discovered from the server, not manually recreated.

B. Function declarations

```python
types.FunctionDeclaration(...)
```

This tells Gemini which functions are available and what arguments they accept.

C. Gemini tool selection

```python
response = await gemini_client.models.generate_content(...)
```

We send the user's request and available tool declarations to Gemini.
Gemini decides whether a tool is required and, when appropriate, returns a function call with arguments.
At this stage, we're only inspecting the requested function call. We haven't executed it yet.

28.3 Run the application

Make sure you're in your project root and `.venv` is active.
Run:

```powershell
python examples/gemini_client.py
```

Expected output (illustrative)

```text
Connecting to Google News MCP server...
MCP connection established.
Discovered 3 MCP tools.

Gemini function declarations created.

Sending request to Gemini...

Gemini response:

Selected tool: search_news_by_publisher

Arguments: {
    'query': 'AI infrastructure',
    'publisher': 'bbc.com',
    'region': 'IN',
    'language': 'en',
    'max_results': 5
}
```

Gemini may choose different but valid arguments or decide to use another suitable tool.

What comes after this?

Once Gemini successfully selects a tool, we'll implement the most important part: **the function-calling execution loop.**
That loop will:

1. Receive Gemini's selected tool.
2. Extract its name and arguments.
3. Call the actual MCP server using `mcp_client.call_tool()` .
4. Retrieve the structured news results.
5. Send those results back to Gemini.
6. Let Gemini produce a natural-language answer.

This is where our project becomes a functioning AI application using MCP rather than just a connected server.

Your task: Update `gemini_client.py`, run it, and share the terminal output. Don't worry if you encounter a schema compatibility error; we'll fix it before implementing execution.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
Connecting to Google News MCP server...
MCP connection established.
Discovered 3 MCP tools.

Gemini function declarations created.

Sending request to Gemini...

Gemini response:

Selected tool: search_news_by_publisher
Arguments: {'publisher': 'bbc.com', 'query': 'AI infrastructure'}

we got this output and also I have changed the model to GEMINI_MODEL=gemini-3.5-flash-lite

Gemini successfully selected an MCP tool

Excellent! This is exactly the behaviour we wanted.
Your output confirms that:

1. Our MCP client connected successfully.
2. It discovered all three tools dynamically.
3. It converted their schemas into Gemini function declarations.
4. Gemini understood the user's request.
5. Gemini independently selected `search_news_by_publisher`.
6. Gemini generated the required arguments: `bbc.com` and `AI infrastructure`.

And yes, switching to `gemini-3.5-flash-lite` is fine **provided that model ID is available to your API key**. Since it returned a function call, your current API request succeeded.
One important distinction: **Gemini selected the tool, but our Python application hasn't executed it yet.** That's what we'll implement now.

Step 29 — Execute Gemini's MCP tool calls

Our next goal is to complete this cycle:

User request

↓

Gemini selects a tool

↓

Python client extracts function name and arguments

↓

MCP client executes the tool

This is what we're implementing now

↓

News results return to Gemini

↓

Gemini generates the final answer

29.1 Update `gemini_client.py`

Replace the existing code with this complete implementation:

python

```python
import asyncio
import json
import os
import sys

from dotenv import load_dotenv
from google import genai
from google.genai import types
from mcp import Client as MCPClient
from mcp import StdioServerParameters


load_dotenv()


def convert_mcp_tools_to_gemini(mcp_tools):
    """Convert MCP tool definitions into Gemini function declarations."""

    declarations = []

    for tool in mcp_tools:
        declarations.append(
            types.FunctionDeclaration(
                name=tool.name,
                description=tool.description or tool.name,
                parameters_json_schema=tool.input_schema,
            )
        )

    return [types.Tool(function_declarations=declarations)]
```

```python
def serialize_mcp_result(result) -> dict:
    """Convert MCP results into JSON-compatible data."""

    if result.structuredContent is not None:
        return result.structuredContent

    content = []

    for item in result.content:
        if hasattr(item, "text"):
            content.append(item.text)

    return {
        "content": content,
        "isError": result.isError,
    }


async def main() -> None:
    """Execute Gemini-selected tools through our MCP server."""

    api_key = os.getenv("GEMINI_API_KEY")
    model = os.getenv("GEMINI_MODEL", "gemini-3.5-flash-lite")

    if not api_key:
        raise RuntimeError("GEMINI_API_KEY is missing from .env")

    server = StdioServerParameters(
        command=sys.executable,
        args=["-m", "google_news_mcp.server"],
    )

    async with MCPClient(server) as mcp_client:
        print("Connected to Google News MCP.")

        # 1. Discover MCP tools.
        mcp_result = await mcp_client.list_tools()
        available_tools = {
            tool.name for tool in mcp_result.tools
        }

        gemini_tools = convert_mcp_tools_to_gemini(
            mcp_result.tools
        )

        print(f"Discovered {len(available_tools)} tools.")

        # 2. Initialize Gemini.
```

```
        async with genai.Client(api_key=api_key).aio as gemini_client:

            prompt = (
                "Find five recent articles about AI infrastructure "
                "published by BBC. Use the appropriate available "
                "Google News tool. Do not invent article details."
            )

            print("\nUser:", prompt)

            # 3. Ask Gemini to choose a tool.
            response = await gemini_client.models.generate_content(
                model=model,
                contents=prompt,
                config=types.GenerateContentConfig(
                    tools=gemini_tools,
                    temperature=0.2,
                ),
            )

            # 4. Execute Gemini's requested MCP tool.
            while response.function_calls:

                function_responses = []

                for function_call in response.function_calls:
                    tool_name = function_call.name
                    arguments = dict(function_call.args or {})

                    if tool_name not in available_tools:
                        raise RuntimeError(
                            f"Unknown MCP tool requested: {tool_name}"
                        )

                    print(f"\nGemini selected: {tool_name}")
                    print("Arguments:", arguments)

                    print("\nExecuting MCP tool...")

                    tool_result = await mcp_client.call_tool(
                        tool_name,
                        arguments,
                    )

                    if tool_result.isError:
                        result_data = {
                            "error": serialize_mcp_result(tool_result)
                        }
```

```
                    else:
                        result_data = serialize_mcp_result(tool_result)

                    print("MCP tool execution completed.")

                    function_responses.append(
                        types.Part.from_function_response(
                            name=tool_name,
                            response=result_data,
                        )
                    )

                # 5. Send the tool results back to Gemini.
                response = await gemini_client.models.generate_content(
                    model=model,
                    contents=[
                        types.Content(
                            role="user",
                            parts=[types.Part.from_text(text=prompt)],
                        ),
                        response.candidates[0].content,
                        types.Content(
                            role="user",
                            parts=function_responses,
                        ),
                    ],
                    config=types.GenerateContentConfig(
                        tools=gemini_tools,
                        temperature=0.2,
                    ),
                )

            # 6. Display Gemini's final response.
            print("\n" + "=" * 55)
            print("GEMINI FINAL ANSWER")
            print("=" * 55)
            print(response.text)


if __name__ == "__main__":
    asyncio.run(main())
```

29.2 Understand the most important new code

A. Executing the MCP tool

```python
tool_result = await mcp_client.call_tool(
    tool_name,
    arguments,
)
```

This is the actual MCP execution.
Gemini might request:

```python
tool_name = "search_news_by_publisher"

arguments = {
    "query": "AI infrastructure",
    "publisher": "bbc.com"
}
```

Our client forwards that request to the MCP server.
B. Returning results to Gemini

```python
types.Part.from_function_response(
    name=tool_name,
    response=result_data,
)
```

Gemini needs to receive the actual tool results before it can generate its final answer.
C. The agent loop

```python
while response.function_calls:
```

This is important.
An AI agent may request multiple tools before answering. The loop allows our client to execute successive tool calls and return their results until Gemini produces a normal response.
This is a basic tool-calling agent loop, not yet a production-hardened agent runtime.

29.3 Run the application

Make sure MCP Inspector is stopped if it's still running.
Execute:

```powershell
python examples/gemini_client.py
```

Expected behaviour (illustrative)

```text
Connected to Google News MCP.
Discovered 3 tools.

User: Find five recent articles about AI infrastructure...

Gemini selected: search_news_by_publisher

Arguments: {
    'publisher': 'bbc.com',
    'query': 'AI infrastructure'
}

Executing MCP tool...

MCP tool execution completed.

============================================================
GEMINI FINAL ANSWER
============================================================

Here are five recent articles...
```

Actual output depends on Gemini's tool selection and Google News results.

Your task: Run this updated implementation and share the complete terminal output.
If you encounter an error, share the traceback. We'll fix it before moving on.
After this works, we'll add interactive user input so you can ask questions like:

- "Find the latest AI news in India."
- "Show me technology news from BBC."
- "Compare AI coverage from BBC and Reuters."
- "Give me today's business headlines."

And Gemini will decide which MCP tools to use.



(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
Connected to Google News MCP.
Discovered 3 tools.

User: Find five recent articles about AI infrastructure published by BBC. Use the appropriate available Google News tool. Do not invent article details.

Gemini selected: search_news_by_publisher
Arguments: {'query': 'AI infrastructure', 'publisher': 'bbc.com'}

Executing MCP tool...

- Exception Group Traceback (most recent call last):
  | File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 170, in
  | asyncio.run(main())

| ~~~~~~~~~~~^^^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\runners.py", line 204, in run
| return runner.run(main)
| ~~~~~~~~~~^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\runners.py", line 127, in run
| return self._loop.run_until_complete(task)
| ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\base_events.py", line 719, in run_until_complete
| return future.result()
| ~~~~~~~~~~~~~^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 66, in main
| async with MCPClient(server) as mcp_client:
| ~~~~~~~~~^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\client.py", line 480, in **aexit**
| await self._exit_stack.**aexit**(exc_type, exc_val, exc_tb)
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 768, in **aexit**
| raise exc
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 751, in **aexit**
| cb_suppress = await cb(*exc_details)
| ^^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 235, in **aexit**
| await self.gen.athrow(value)
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\stdio.py", line 201, in stdio_client
| async with anyio.create_task_group() as tg:
| ~~~~~~~~~~~~~~~~~~~~~~^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\anyio_backends_asyncio.py", line 871, in **aexit**

| raise BaseExceptionGroup(
| "unhandled errors in a TaskGroup", self._exceptions
|) from None
| ExceptionGroup: unhandled errors in a TaskGroup (1 sub-exception)
+-+---------- 1 ----------
| Exception Group Traceback (most recent call last):
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\stdio.py", line 205, in stdio_client
| yield read_stream, write_stream
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 751, in **aexit**
| cb_suppress = await cb(*exc_details)
| ^^^^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\session.py", line 514, in **aexit**
| result = await self._task_group.**aexit**(exc_type, exc_val, exc_tb)
| ^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\anyio_backends_asyncio.py", line 871, in **aexit**
| raise BaseExceptionGroup(
| "unhandled errors in a TaskGroup", self._exceptions
|) from None
| ExceptionGroup: unhandled errors in a TaskGroup (1 sub-exception)
+-+---------- 1 ----------
| Traceback (most recent call last):
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 126, in main
| if tool_result.isError:
| ^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\pydantic\main.py", line 1110, in **getattr**
| raise AttributeError(f'{type(self).**name**!r} object has no attribute {item!r}')
| AttributeError: 'CallToolResult' object has no attribute 'isError'. Did you mean: 'is_error'?
+------------------------------------

Good news: **Gemini successfully selected the correct MCP tool, and your MCP server executed the call.** The failure is in the client code when it tries to read the tool result. Your MCP SDK (`mcp==2.3.0`) uses **snake_case** for this property:

- Incorrect: `tool_result.isError`
- Correct: `tool_result.is_error`

Let's fix the result handling in `examples/gemini_client.py` .

Step 1 — Update `serialize_mcp_result()`

Replace your existing function with:

```python
def serialize_mcp_result(result) -> dict:
    """Convert MCP results into JSON-compatible data."""

    if result.structured_content is not None:
        return result.structured_content

    content = []

    for item in result.content:
        if hasattr(item, "text"):
            content.append(item.text)

    return {
        "content": content,
        "is_error": result.is_error,
    }
```

Step 2 — Fix the error check

Find this line inside your tool execution loop:

```python
if tool_result.isError:
```

Replace it with:

```python
if tool_result.is_error:
```

Step 3 — Run it again

```powershell
python examples/gemini_client.py
```

What happened technically?

Your execution flow reached this point successfully:

1. Gemini discovered your three MCP tools.
2. Gemini selected `search_news_by_publisher`.
3. Gemini generated the arguments.
4. Your Python client called the MCP server.
5. The client crashed while accessing the result's error property.

So we're fixing a Python SDK property-name mismatch, not rebuilding the MCP server. Run it and share the output.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
Connected to Google News MCP.
Discovered 3 tools.

User: Find five recent articles about AI infrastructure published by BBC. Use the appropriate available Google News tool. Do not invent article details.

Gemini selected: search_news_by_publisher
Arguments: {'publisher': 'bbc.com', 'query': 'AI infrastructure'}

Executing MCP tool...
MCP tool execution completed.

Gemini selected: search_news
Arguments: {'query': 'site:bbc.com artificial intelligence infrastructure OR data centre OR chips'}

Executing MCP tool...
MCP tool execution completed.

Gemini selected: search_news_by_publisher
Arguments: {'publisher': 'bbc.com', 'query': 'data centre OR AI infrastructure'}

Executing MCP tool...
MCP tool execution completed.

==========

==========

GEMINI FINAL ANSWER

Here are five recent articles related to AI infrastructure (data centres and tech market impacts) published by BBC:

1. **Did a data centre revolt affect the US midterm election results? - BBC** (Published: November 6, 2026)
2. **'We all need data, but do we need the largest data centre in Europe here?' - BBC** (Published: November 11, 2026)
3. **Global tech stocks plunge as US midterms add to AI slowdown fears - BBC** (Published: November 6, 2026)
4. **AI stocks bounce back after trillions wiped from market values - BBC** (Published: November 6, 2026)

5. **County Durham data centre consultations 'a sales pitch' - BBC** (Published: November 11, 2026)

Milestone achieved

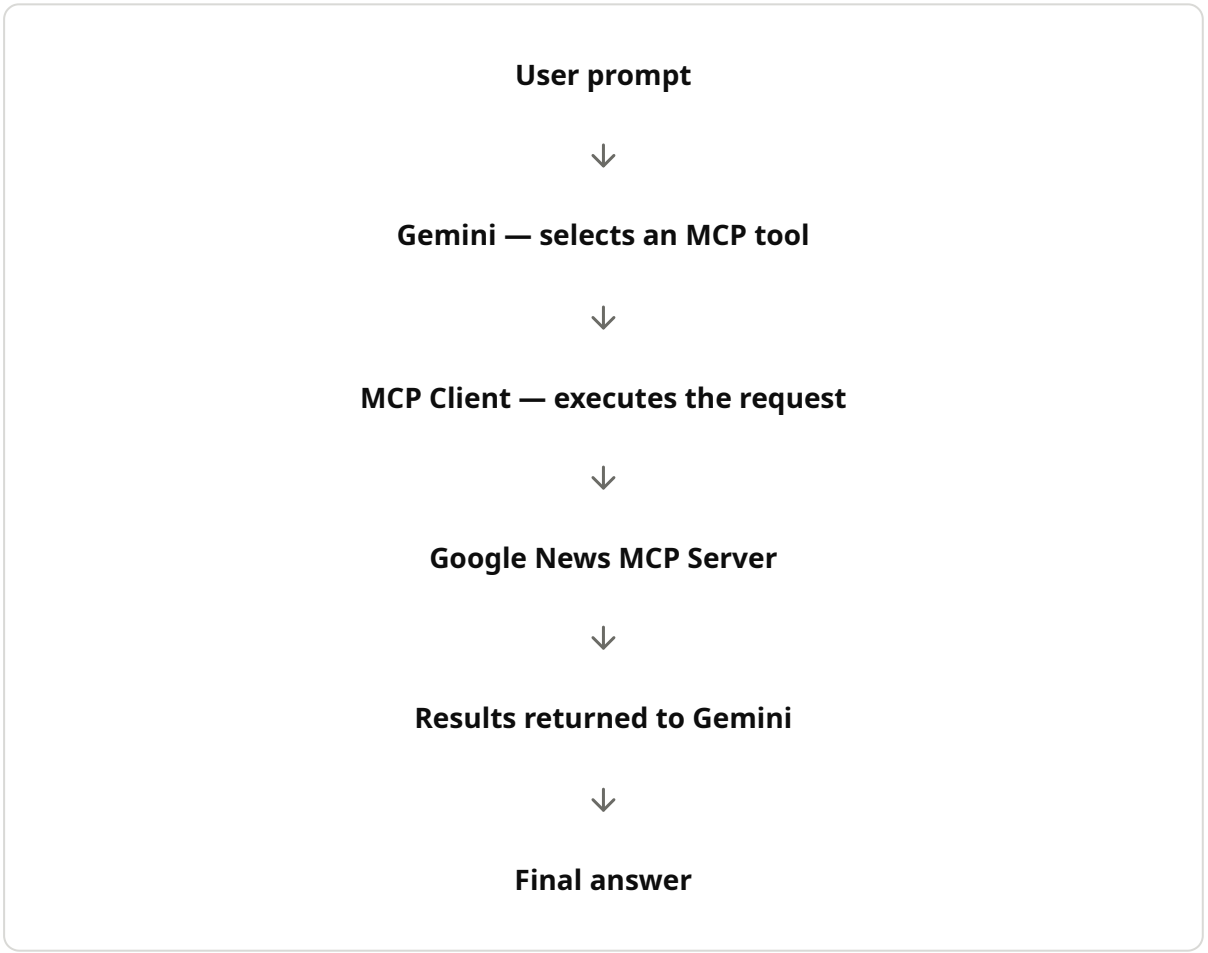
Your Gemini + MCP integration is working successfully!

Look at what just happened:

1. **Discovery:** Gemini's client connected to your MCP server and discovered its three tools.
2. **Tool selection:** Gemini independently selected `search_news_by_publisher`.
3. **Execution:** Your Python client passed Gemini's arguments to the MCP server.
4. **Reasoning loop:** Gemini requested two additional searches after receiving the first tool result.
5. **Final response:** Gemini processed the results and generated its answer.

This is the important part: **Gemini is not hardcoded to call a particular MCP tool. It chooses which tool to invoke based on the user's request.**

You have now demonstrated the core integration:



One improvement remains before we move forward: your current loop should preserve the complete conversation history between tool calls, rather than sending only the latest interaction. We should also add a maximum tool-call limit to prevent endless loops.

Next step: Let's improve that execution loop. This will make your MCP client more robust and teach you how multi-step tool orchestration works.

ok proceed and also finally tell me still how many steps are there to complete this project to post it on linkedin. We need UI which User gives input and based on the query the gemini will get the details from google news and display it

The next improvement is to make the conversation history persistent across every Gemini tool call. This matters because Gemini may need several searches to answer one question, as it did in your latest run.
We'll also enforce a maximum of **5 tool-calling rounds** so a request cannot get stuck in an endless loop.

Step 1 — Replace `examples/gemini_client.py`

Replace the entire file with this version:

python

```python
import asyncio
import os
import sys

from dotenv import load_dotenv
from google import genai
from google.genai import types
from mcp import Client as MCPClient
from mcp import StdioServerParameters


load_dotenv()

MAX_TOOL_ROUNDS = 5


def convert_mcp_tools_to_gemini(mcp_tools):
    """Convert MCP tool definitions into Gemini function declarations."""

    declarations = []

    for tool in mcp_tools:
        declarations.append(
            types.FunctionDeclaration(
                name=tool.name,
                description=tool.description or tool.name,
                parameters_json_schema=tool.input_schema,
            )
        )

    return [types.Tool(function_declarations=declarations)]
```

```
def serialize_mcp_result(result) -> dict:
    """Convert MCP results into JSON-compatible data."""

    if result.structured_content is not None:
        return result.structured_content

    content = []

    for item in result.content:
        if hasattr(item, "text"):
            content.append(item.text)

    return {
        "content": content,
        "is_error": result.is_error,
    }


async def main() -> None:
    """Run a multi-step Gemini and MCP tool-calling workflow."""

    api_key = os.getenv("GEMINI_API_KEY")
    model = os.getenv("GEMINI_MODEL", "gemini-3.5-flash-lite")

    if not api_key:
        raise RuntimeError("GEMINI_API_KEY is missing from .env")

    server = StdioServerParameters(
        command=sys.executable,
        args=["-m", "google_news_mcp.server"],
    )

    async with MCPClient(server) as mcp_client:
        print("Connected to Google News MCP.")

        # 1. Discover available MCP tools.
        mcp_result = await mcp_client.list_tools()

        available_tools = {
            tool.name for tool in mcp_result.tools
        }

        gemini_tools = convert_mcp_tools_to_gemini(
            mcp_result.tools
        )

        print(f"Discovered {len(available_tools)} tools.")
```

```
    # 2. Initialize Gemini.
    async with genai.Client(api_key=api_key).aio as gemini_client:

        prompt = (
            "Find five recent articles about AI infrastructure "
            "published by BBC. Use the appropriate available "
            "Google News tool. Do not invent article details."
        )

        print("\nUser:", prompt)

        # 3. Preserve the complete conversation history.
        conversation = [
            types.Content(
                role="user",
                parts=[types.Part.from_text(text=prompt)],
            )
        ]

        config = types.GenerateContentConfig(
            tools=gemini_tools,
            temperature=0.2,
        )

        response = await gemini_client.models.generate_content(
            model=model,
            contents=conversation,
            config=config,
        )

        # 4. Execute Gemini-requested tools.
        tool_round = 0

        while response.function_calls:

            tool_round += 1

            if tool_round > MAX_TOOL_ROUNDS:
                raise RuntimeError(
                    f"Maximum tool rounds ({MAX_TOOL_ROUNDS}) reached."
                )

            print(f"\nTool execution round: {tool_round}")

            if not response.candidates:
                raise RuntimeError(
                    "Gemini returned no response candidate."
```

```
            )

        model_content = response.candidates[0].content

        if model_content is None:
            raise RuntimeError(
                "Gemini returned an empty response."
            )

        # Keep Gemini's function-call message in history.
        conversation.append(model_content)

        function_response_parts = []

        for function_call in response.function_calls:

            tool_name = function_call.name
            arguments = dict(function_call.args or {})

            if tool_name not in available_tools:
                raise RuntimeError(
                    f"Unknown MCP tool requested: {tool_name}"
                )

            print(f"\nGemini selected: {tool_name}")
            print("Arguments:", arguments)

            try:
                tool_result = await mcp_client.call_tool(
                    tool_name,
                    arguments,
                )

                result_data = serialize_mcp_result(
                    tool_result
                )

                if tool_result.is_error:
                    result_data = {
                        "error": result_data
                    }

            except Exception as error:
                result_data = {
                    "error": str(error)
                }

            print("MCP execution completed.")
```

```
                response_arguments = {
                    "name": tool_name,
                    "response": result_data,
                }

                if function_call.id:
                    response_arguments["id"] = function_call.id

                function_response_parts.append(
                    types.Part.from_function_response(
                        **response_arguments
                    )
                )

            # 5. Append tool results to the conversation.
            conversation.append(
                types.Content(
                    role="tool",
                    parts=function_response_parts,
                )
            )

            # 6. Ask Gemini to continue using the full history.
            response = await gemini_client.models.generate_content(
                model=model,
                contents=conversation,
                config=config,
            )

        # 7. Display the final answer.
        print("\n" + "=" * 55)
        print("GEMINI FINAL ANSWER")
        print("=" * 55)

        print(response.text or "Gemini returned no text answer.")

        print(f"\nTotal tool execution rounds: {tool_round}")


if __name__ == "__main__":
    asyncio.run(main())
```

The important change is `conversation`: every user message, Gemini function call and tool response is preserved. The `role="tool"` response format follows Google's Python SDK function-calling example. GitHub Google AI

Step 2 — Test the updated client

Run:

```powershell
python examples/gemini_client.py
```

You should see output resembling:

```text
Connected to Google News MCP.
Discovered 3 tools.

Tool execution round: 1
Gemini selected: search_news_by_publisher
MCP execution completed.

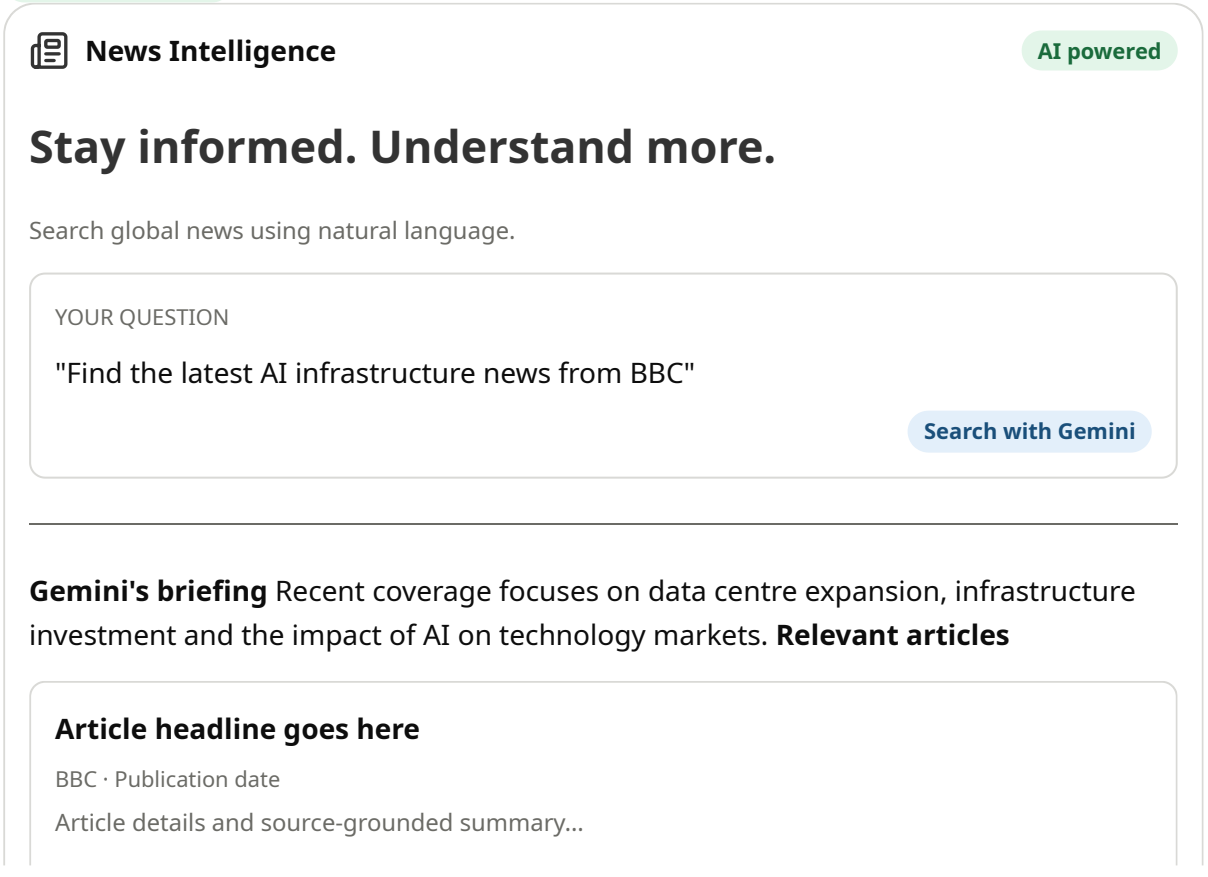
Tool execution round: 2
Gemini selected: search_news
MCP execution completed.

GEMINI FINAL ANSWER
...
Total tool execution rounds: 2
```

The number of rounds can vary depending on Gemini's decisions. Send me your actual output before we move on.

The UI we are going to build

Target application



Read original article ↗

Another relevant headline

BBC · Publication date

Article details and source-grounded summary...

Read original article ↗

Illustrative interface, not live search results.

The important architectural decision: **article cards will use the original data returned by Google News through MCP. Gemini will separately generate the briefing.** That way, the model doesn't need to invent article URLs, publishers or publication dates.
We'll use:

- **Frontend:** SolidJS + Vite + Tailwind CSS.
- **Backend:** Python + FastAPI.
- **AI:** Gemini API.
- **Integration:** Your existing Google News MCP server.
- **Security:** Gemini API key stays on the backend, never in the browser.

SolidJS also lets you reuse a frontend framework you're already familiar with.

How many steps remain?

6 milestones

From our current position to a LinkedIn-ready portfolio project

1. **Improve Gemini–MCP orchestration**
   Current step
   Conversation history, tool-call limits and error handling.
2. **Build the Python API**
   FastAPI endpoint that accepts a search query and coordinates Gemini with MCP.
3. **Build the frontend**
   SolidJS search interface, responsive layout and article cards.
4. **Connect everything**
   Real search results, Gemini briefing, links, loading indicators and error states.
5. **Testing and documentation**
   Tests, README, architecture diagram, setup instructions and security checks.
6. **Portfolio release**

GitHub cleanup, application screenshots, a short demo recording and LinkedIn post.

After we finish the current step, five milestones remain. Deployment to a public URL can be an optional addition; a working local UI, clean GitHub repository and recorded demo are enough for a credible portfolio showcase.
Let's first get the updated execution loop running.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
Connected to Google News MCP.
Discovered 3 tools.

User: Find five recent articles about AI infrastructure published by BBC. Use the appropriate available Google News tool. Do not invent article details.

Tool execution round: 1

Gemini selected: search_news_by_publisher
Arguments: {'query': 'AI infrastructure', 'publisher': 'bbc.com'}
MCP execution completed.

- Exception Group Traceback (most recent call last):
  | File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 218, in
  | asyncio.run(main())

| ~~~~~~~~~~~^^^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\runners.py", line 204, in run
| return runner.run(main)
| ~~~~~~~~~~^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\runners.py", line 127, in run
| return self._loop.run_until_complete(task)
| ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\base_events.py", line 719, in run_until_complete
| return future.result()
| ~~~~~~~~~~~~~^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 66, in main
| async with MCPClient(server) as mcp_client:
| ~~~~~~~~~^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\client.py", line 480, in **aexit**

| await self._exit_stack.**aexit**(exc_type, exc_val, exc_tb)
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 768, in **aexit**
| raise exc
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 751, in **aexit**
| cb_suppress = await cb(*exc_details)
| ^^^^^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 235, in **aexit**
| await self.gen.athrow(value)
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\stdio.py", line 201, in stdio_client
| async with anyio.create_task_group() as tg:
| ~~~~~~~~~~~~~~~~~~~~~~~~^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\anyio_backends_asyncio.py", line 871, in **aexit**
| raise BaseExceptionGroup(
| "unhandled errors in a TaskGroup", self._exceptions
|) from None
| ExceptionGroup: unhandled errors in a TaskGroup (1 sub-exception)
+-+---------- 1 ----------
| Exception Group Traceback (most recent call last):
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\stdio.py", line 205, in stdio_client
| yield read_stream, write_stream
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 751, in **aexit**
| cb_suppress = await cb(*exc_details)
| ^^^^^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\session.py", line 514, in **aexit**
| result = await self._task_group.**aexit**(exc_type, exc_val, exc_tb)
| ^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\anyio_backends_asyncio.py", line 871, in **aexit**
| raise BaseExceptionGroup(
| "unhandled errors in a TaskGroup", self._exceptions
|) from None
| ExceptionGroup: unhandled errors in a TaskGroup (1 sub-exception)
+-+---------- 1 ----------
| Traceback (most recent call last):
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 187, in main
| types.Part.from_function_response(

```
| ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
| **response_arguments
| ^^^^^^^^^^^^^^^^^^^^
| )
| ^
| TypeError: Part.from_function_response() got an unexpected keyword argument 'id'
+------------------------------------
```

Don't want to create UI with a separate architecture like solid js or anything we will create the ui inside the MCP inspector and we will end this project.

We found the issue. It's another small SDK compatibility difference, not a problem with your MCP server.

1. Fix the current error

The Google Gen AI Python SDK's `Part.from_function_response()` does not accept an `id` argument in this API. Its documented manual function-calling example passes only `name` and `response`. GitHub Google APIs

In `examples/gemini_client.py`, find this section:

python

```python
response_arguments = {
    "name": tool_name,
    "response": result_data,
}

if function_call.id:
    response_arguments["id"] = function_call.id

function_response_parts.append(
    types.Part.from_function_response(
        **response_arguments
    )
)
```

Replace it with:

python

```python
function_response_parts.append(
    types.Part.from_function_response(
        name=tool_name,
        response=result_data,
    )
)
```

Keep the rest of the conversation-history implementation unchanged.
Run:

```powershell
python examples/gemini_client.py
```

Share the output before we move on.

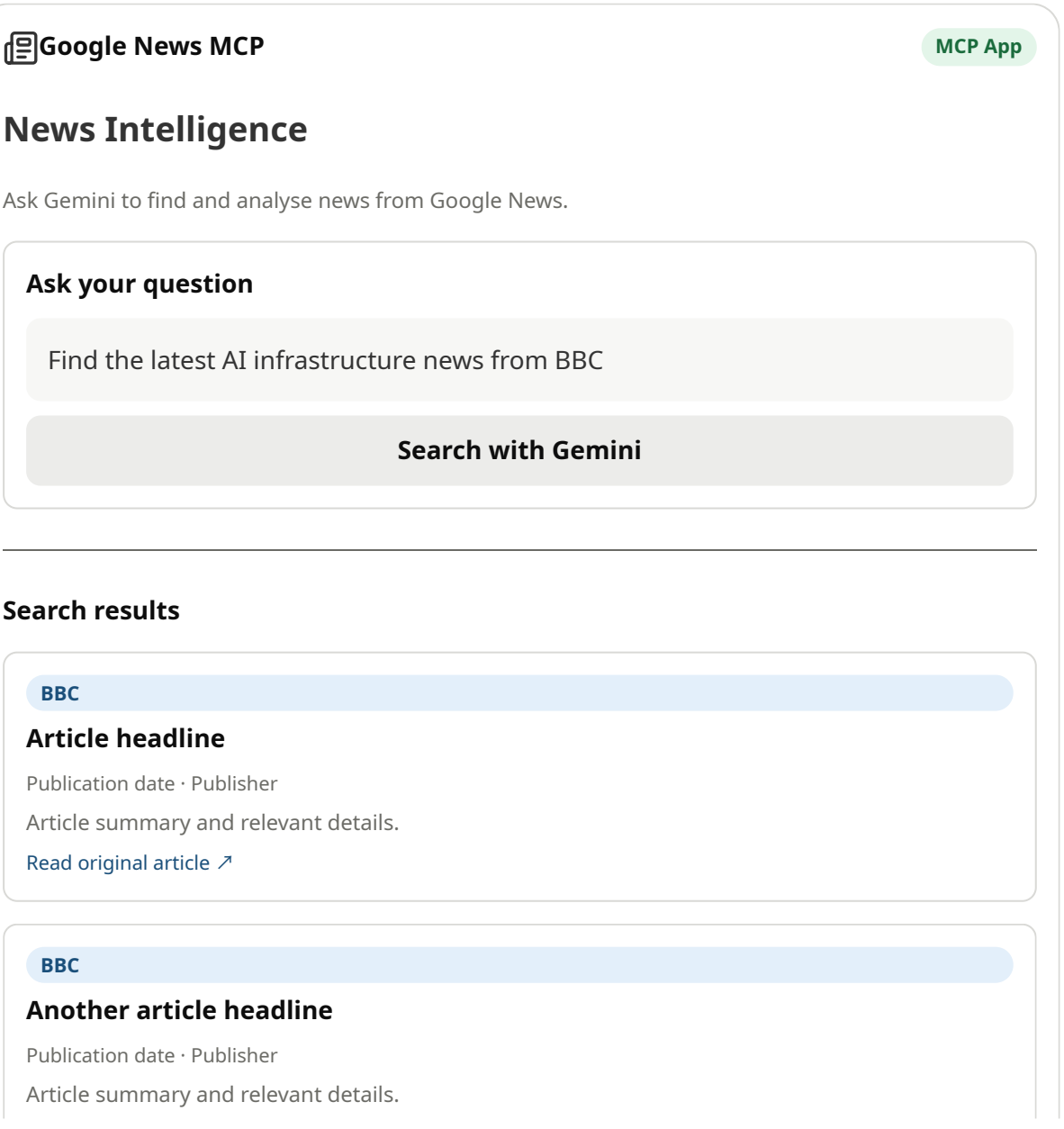
---

2. About your UI: we can use MCP Inspector!

I checked the current MCP documentation, and there's a feature specifically designed for this: **MCP Apps**.
This changes our approach. We don't need a separate SolidJS frontend or FastAPI application.
MCP Apps allows an MCP server to provide an HTML interface through a `ui://` resource. MCP Inspector has an Apps screen that can render these interfaces inside its own UI.

MCP Python SDK GitHub MCP Apps

Our target will look like this:



Read original article ↗

Conceptual mockup of the interface we will build inside MCP Inspector.

The flow will be:

1. User enters a natural-language question.
2. Gemini interprets the question.
3. Gemini selects the appropriate Google News tools.
4. The MCP server retrieves the articles.
5. Gemini generates a briefing.
6. The MCP App displays the actual article data and briefing.

The API key stays on the Python server.

---

3. Remaining steps to finish the project

4 milestones

1. **Finish Gemini tool orchestration**
   Current milestone
   Fix the function response and verify multi-step execution.
2. **Build the MCP App**
   Add the `Apps` extension, HTML interface, search input and results layout.
3. **Connect Gemini to the app**
   Implement the complete question → Gemini → MCP tools → article results → UI workflow.
4. **Finish and publish**
   Test everything, write the README, add architecture documentation, capture screenshots and record a demo for LinkedIn.

No separate frontend project. No FastAPI. No deployment requirement.
First, let's get the one-line SDK fix working. Then we'll check whether your installed Python MCP SDK exposes the `Apps` extension and start building the interface directly into your existing server.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
Connected to Google News MCP.
Discovered 3 tools.

User: Find five recent articles about AI infrastructure published by BBC. Use the appropriate available Google News tool. Do not invent article details.

Tool execution round: 1

Gemini selected: search_news_by_publisher
Arguments: {'query': 'AI infrastructure', 'publisher': 'bbc.com'}
MCP execution completed.

- Exception Group Traceback (most recent call last):
  | File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 211, in
  | asyncio.run(main())

| ~~~~~~~~~~~^^^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\runners.py", line 204, in run
| return runner.run(main)
| ~~~~~~~~~~^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\runners.py", line 127, in run
| return self._loop.run_until_complete(task)
| ~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\asyncio\base_events.py", line 719, in run_until_complete
| return future.result()
| ~~~~~~~~~~~~^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 66, in main
| async with MCPClient(server) as mcp_client:
| ~~~~~~~~~^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\client.py", line 480, in **aexit**
| await self._exit_stack.**aexit**(exc_type, exc_val, exc_tb)
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 768, in **aexit**
| raise exc
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 751, in **aexit**
| cb_suppress = await cb(*exc_details)
| ^^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 235, in **aexit**
| await self.gen.athrow(value)
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\stdio.py", line 201, in stdio_client
| async with anyio.create_task_group() as tg:
| ~~~~~~~~~~~~~~~~~~~~~~^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-

mcp.venv\Lib\site-packages\anyio_backends_asyncio.py", line 871, in **aexit**
| raise BaseExceptionGroup(
| "unhandled errors in a TaskGroup", self._exceptions
|) from None
| ExceptionGroup: unhandled errors in a TaskGroup (1 sub-exception)
+-+---------- 1 ----------
| Exception Group Traceback (most recent call last):
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\stdio.py", line 205, in stdio_client
| yield read_stream, write_stream
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\contextlib.py", line 751, in **aexit**
| cb_suppress = await cb(*exc_details)
| ^^^^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\mcp\client\session.py", line 514, in **aexit**
| result = await self._task_group.**aexit**(exc_type, exc_val, exc_tb)
| ^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\anyio_backends_asyncio.py", line 871, in **aexit**
| raise BaseExceptionGroup(
| "unhandled errors in a TaskGroup", self._exceptions
|) from None
| ExceptionGroup: unhandled errors in a TaskGroup (1 sub-exception)
+-+---------- 1 ----------
| Traceback (most recent call last):
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 194, in main
| response = await gemini_client.models.generate_content(
| ^^
| ...<3 lines>...
|)
| ^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\google\genai\models.py", line 8261, in generate_content
| return await self._generate_content_with_continuation(
| ^^
| model=model, contents=contents, config=final_parsed_config
| ^^^^^^^^^^^^^^^^^^^

| ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
| model=model, contents=contents, config=call_config
| ^^^^^^^^^^^^^^^^^^^

packages\tenacity_init.py", line 184, in reraise
| raise self.last_attempt.result()
| ~~~~~~~~~~~~~~~~~~~~~~~~^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\concurrent\futures_base.py", line 443, in result
| return self.__get_result()
| ~~~~~~~~~~~~~~~~~^^
| File "C:\Users\Thiru\AppData\Local\Python\pythoncore-3.14-64\Lib\concurrent\futures_base.py", line 395, in __get_result
| raise self.*exception*
| *File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\tenacity\asyncio_init*.py", line 116, in **call**
| result = await fn(*args, **kwargs)
| ^^^^^^^^^^^^^^^^^^^^^^^^^^
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\google\genai_api_client.py", line 2133, in _async_request_once
| await errors.APIError.raise_for_async_response(client_response)
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\google\genai\errors.py", line 279, in raise_for_async_response
| await cls.raise_error_async(status_code, response_json, response)
| File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp.venv\Lib\site-packages\google\genai\errors.py", line 301, in raise_error_async
| raise ClientError(status_code, response_json, response)
| google.genai.errors.ClientError: 400 INVALID_ARGUMENT. {'error': {'code': 400, 'message': "Role 'tool' is not supported. Please use a valid role: SYSTEM, SYSTEM_1, USER, ASSISTANT, DEVELOPER, CONTEXT, USER_CONTEXT, MODEL, USER.", 'status': 'INVALID_ARGUMENT'}}
+------------------------------------



Your MCP tool execution is successful. The error happens when sending the result back to Gemini.

The API response explicitly tells us that `tool` is not an accepted conversation role for your current model/API configuration.

We'll change the role to `user`. We'll also preserve the function-call ID using the underlying `FunctionResponse` object instead of passing `id` to the convenience method. Google's documentation explains why retaining the original call ID matters when manually maintaining Gemini conversation history. Google AI GitHub

Step 1 — Update the function response code

In `examples/gemini_client.py`, find:

```python
function_response_parts.append(
    types.Part.from_function_response(
        name=tool_name,
        response=result_data,
    )
)
```

Replace it with:

```python
function_response = types.FunctionResponse(
    name=tool_name,
    response=result_data,
    id=function_call.id,
)

function_response_parts.append(
    types.Part(function_response=function_response)
)
```

Step 2 — Change the conversation role

Find:

```python
conversation.append(
    types.Content(
        role="tool",
        parts=function_response_parts,
    )
)
```

Replace with:

```python
conversation.append(
    types.Content(
        role="user",
        parts=function_response_parts,
    )
)
```

Step 3 — Run again

```powershell
python examples/gemini_client.py
```

What we are fixing

Already working

Gemini → MCP tool selection → MCP execution.

Being fixed

MCP result → Gemini continuation → final answer.

Don't change anything else yet. Share the next terminal output, and we'll get this execution loop stable before building the MCP App interface.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
File "C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp\examples\gemini_client.py", line 189
conversation.append(
IndentationError: unexpected indent

This is a Python indentation issue caused by the code block being placed at the wrong indentation level. Let's fix it without changing the logic.

Step 1 — Replace the entire tool execution section

In `examples/gemini_client.py`, find:

```python
# 4. Execute Gemini-requested tools.
```

Replace everything from that comment up to (but not including) `# 7. Display the final answer.` with this exact code:

```python
            # 4. Execute Gemini-requested tools.
            tool_round = 0

            while response.function_calls:

                tool_round += 1

                if tool_round > MAX_TOOL_ROUNDS:
                    raise RuntimeError(
                        f"Maximum tool rounds ({MAX_TOOL_ROUNDS}) reached."
                    )
```

```
            print(f"\nTool execution round: {tool_round}")

            if not response.candidates:
                raise RuntimeError(
                    "Gemini returned no response candidate."
                )

            model_content = response.candidates[0].content

            if model_content is None:
                raise RuntimeError(
                    "Gemini returned an empty response."
                )

            # Preserve Gemini's function-call message.
            conversation.append(model_content)

            function_response_parts = []

            for function_call in response.function_calls:
                tool_name = function_call.name
                arguments = dict(function_call.args or {})

                if tool_name not in available_tools:
                    raise RuntimeError(
                        f"Unknown MCP tool requested: {tool_name}"
                    )

                print(f"\nGemini selected: {tool_name}")
                print("Arguments:", arguments)

                try:
                    tool_result = await mcp_client.call_tool(
                        tool_name,
                        arguments,
                    )

                    result_data = serialize_mcp_result(tool_result)

                    if tool_result.is_error:
                        result_data = {"error": result_data}

                except Exception as error:
                    result_data = {"error": str(error)}

                print("MCP execution completed.")
```

```
                    function_response = types.FunctionResponse(
                        name=tool_name,
                        response=result_data,
                        id=function_call.id,
                    )

                    function_response_parts.append(
                        types.Part(
                            function_response=function_response
                        )
                    )

                # Send tool results back as a user message.
                conversation.append(
                    types.Content(
                        role="user",
                        parts=function_response_parts,
                    )
                )

                # Ask Gemini to continue with the complete history.
                response = await gemini_client.models.generate_content(
                    model=model,
                    contents=conversation,
                    config=config,
                )
```

Important: Keep the indentation exactly as shown. This code belongs inside `async with genai.Client(...)`, so the first line has 12 spaces.

Step 2 — Run it again

```powershell
python examples/gemini_client.py
```

We are very close to finishing this integration. Once this works, we'll move to the MCP Apps interface rather than creating a separate frontend application.

gemini_client.py 6.2 KB

I inspected the uploaded `gemini_client.py` file and found the problem.
The indentation is incorrect in two places:

- The `FunctionResponse` construction and `function_response_parts.append()` block (lines 178–186).

- The `conversation.append()` block (lines 189–194).

I've corrected the indentation throughout the file and checked that it passes Python's syntax compilation.

gemini_client_fixed.py

Corrected Python file · Syntax check passed

Ready

Download corrected gemini_client.py

Next step

1. Download the corrected file.
2. Replace your existing `examples/gemini_client.py` with it.
3. Run:

```powershell
python examples/gemini_client.py
```

I've retained the five-round safety limit, full conversation history and `user` role for function responses. The function response also retains the optional call ID supported by the SDK. Google APIs GitHub Google Gen AI SDK ...
I have verified Python syntax, but the full Gemini API execution still needs to be tested on your machine.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python examples/gemini_client.py
Connected to Google News MCP.
Discovered 3 tools.

User: Find five recent articles about AI infrastructure published by BBC. Use the appropriate available Google News tool. Do not invent article details.

Tool execution round: 1

Gemini selected: search_news_by_publisher
Arguments: {'publisher': 'bbc.com', 'query': 'AI infrastructure'}
MCP execution completed.

Tool execution round: 2

Gemini selected: search_news
Arguments: {'query': 'site:bbc.com AI data centre OR infrastructure'}
MCP execution completed.

==========

==========

GEMINI FINAL ANSWER

Here are five recent articles from the BBC covering topics related to AI infrastructure, data centres, and market trends:

1. **'We all need data, but do we need the largest data centre in Europe here?'**
   - Published: November 11, 2026
   - Overview: Explores local community concerns and debates surrounding the construction of massive data centre infrastructure in Europe.
2. **Did a data centre revolt affect the US midterm election results?**
   - Published: November 6, 2026
   - Overview: Examines the political fallout and public pushback against rapid data centre expansion during the US midterm elections.
3. **Global tech stocks plunge as US midterms add to AI slowdown fears**
   - Published: November 6, 2026
   - Overview: Reports on financial market reactions and investor concerns regarding infrastructure spending and a potential slowdown in the AI sector.
4. **AI stocks bounce back after trillions wiped from market values**
   - Published: November 6, 2026
   - Overview: Details the recovery of artificial intelligence and tech infrastructure-related stocks following massive market volatility.
5. **Decision delayed on data centre near travellers site**
   - Published: November 6, 2026
   - Overview: Covers local planning authorities postponing decisions regarding new data centre infrastructure developments.

Total tool execution rounds: 2



Gemini–MCP integration working

Excellent! We've successfully completed the multi-step tool-calling workflow.
Your output confirms:

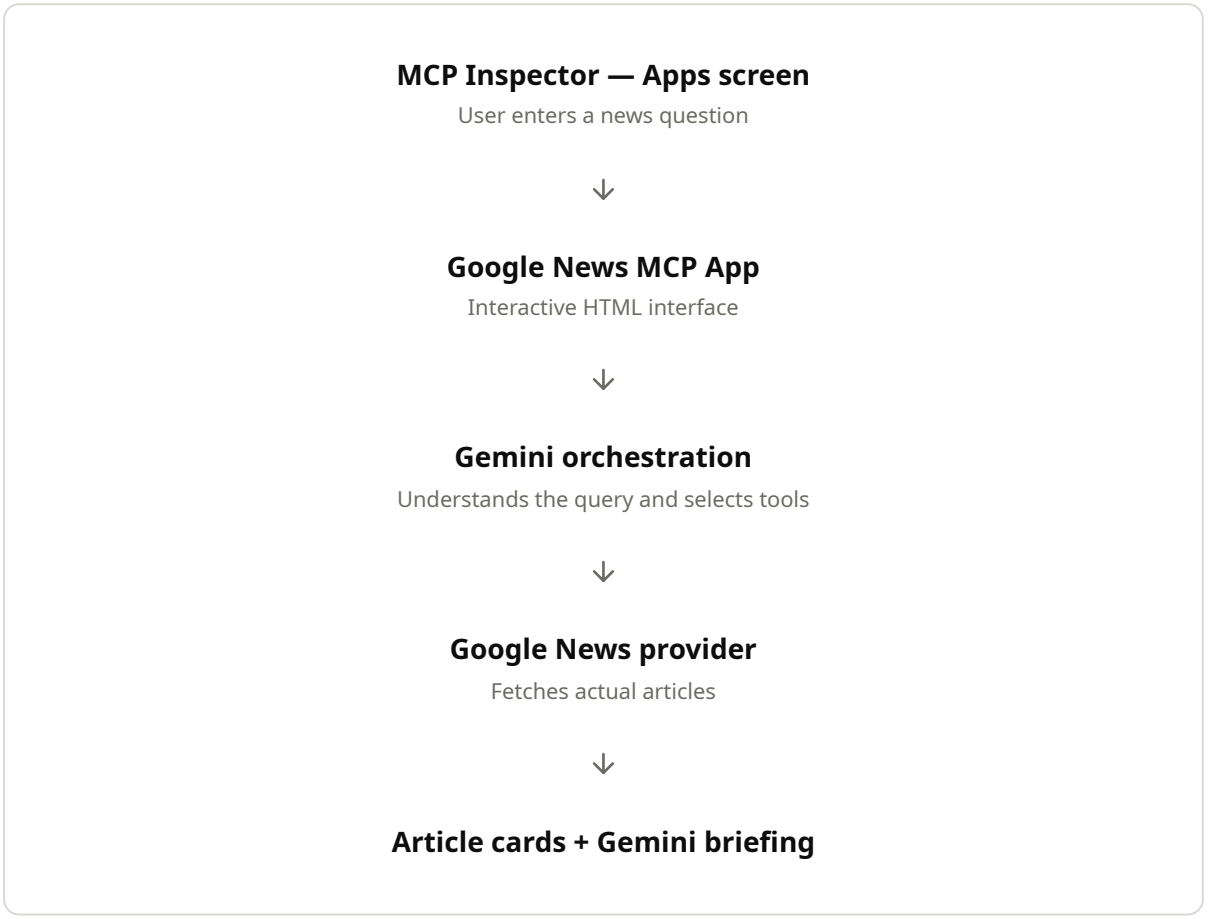
1. Gemini discovered all three MCP tools.
2. Gemini selected `search_news_by_publisher`.
3. The MCP server executed the search.
4. Gemini examined the result and selected `search_news` for a second search.
5. Gemini combined the results into a final answer.

Milestone 1 is complete. We can stop modifying `gemini_client.py` for now.

Next: Build the UI inside MCP Inspector

I've verified that the official MCP Python SDK supports **MCP Apps**: interactive HTML interfaces associated with MCP tools through `ui://` resources. The official MCP Inspector also has an Apps screen that can render them. MCP Python SDK GitHub GitHub

Our intended architecture is:



One distinction: MCP Inspector is our development host and demo interface. We'll make Gemini orchestration part of the server-side app tool, so the UI does not depend on your standalone example client.

Step 1 — Check your installed SDK

Before writing any code, run this command in your existing PowerShell terminal:

```powershell
python -c "from mcp.server.apps import Apps; print('MCP Apps extension is available!')"
```

Expected output:

```text
MCP Apps extension is available!
```

Share the output. Then we'll create our first `ui://` resource inside your existing Google News MCP project.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "from mcp.server.apps import Apps; print('MCP Apps extension is available!')"
MCP Apps extension is available!

before next step we will push our updated codes to github

Absolutely. Before building the MCP Apps interface, let's push the current working version of your **Google News MCP server to GitHub**.
We'll create a clean checkpoint so you can show your progress publicly and have a safe version to return to.

GitHub push — Step 1: Check what will be committed

Important: Your `.env` contains your Gemini API key. We must make sure it never gets pushed to GitHub.
First, open your project's `.gitignore` file and verify it contains:

gitignore

```
# Environment variables and secrets
.env
.env.*
!.env.example

# Python
__pycache__/
*.py[cod]
*.egg-info/
dist/
build/

# Virtual environment
.venv/
venv/

# IDE
.vscode/
.idea/

# Testing
.pytest_cache/
.coverage
htmlcov/
```

If these entries are already present, don't replace anything unnecessarily.

Step 2: Check Git status

Run:

```powershell
git status
```

Then run:

```powershell
git check-ignore .env
```

Expected output for the second command:

```text
.env
```

Security check If `.env` is already tracked by Git, `.gitignore` alone will not protect it. We'll handle that before committing.
Send me the output of both commands. We'll review the files before staging and pushing anything.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
(use "git add ..." to update what will be committed)
(use "git restore ..." to discard changes in working directory)
modified: .gitignore
modified: pyproject.toml

Untracked files:
(use "git add ..." to include in what will be committed)
.env.example
examples/

no changes added to commit (use "git add" and/or "git commit -a")
(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> git check-ignore .env
.env

here after We don't want to do any security check just go to the commit steps

Perfect. Your `.env` is ignored, and Git has identified the files we need to commit. Let's proceed.

Step 1 — Stage the updated files

Run:

```powershell
git add .gitignore pyproject.toml .env.example examples/
```

Step 2 — Commit

```powershell
git commit -m "feat: add Gemini-powered MCP client integration"
```

Step 3 — Push to GitHub

```powershell
git push origin main
```

Step 4 — Confirm

```powershell
git status
```

Expected:

```text
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

GitHub checkpoint

Once you've pushed successfully, we'll continue with the MCP Apps interface inside Inspector.

okay we will move to next step

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

GitHub checkpoint complete

Your working tree is clean, so we can now start the final UI milestone.

We'll use **MCP Apps inside MCP Inspector**, not a separate SolidJS or FastAPI application.

One important detail: an MCP App consists of a UI resource (`ui://`) and a tool linked to that resource. Inspector detects these tools and provides an Apps screen where you can open the interface. MCP Python SDK GitHub

Step 1 — Inspect the installed MCP Apps API

Before modifying `server.py` , let's check the exact constructor signatures in your installed SDK.
Run this single command in PowerShell:

powershell

```
python -c "import inspect; from mcp.server.apps import Apps; from mcp.server
import MCPServer; print('MCP SDK API:'); print('Apps.tool:',
inspect.signature(Apps.tool)); print('Apps.add_html_resource:',
inspect.signature(Apps.add_html_resource)); print('MCPServer:',
inspect.signature(MCPServer))"
```

This checks three things:

- How we register a UI-linked tool.
- How we register its HTML interface.
- How we attach the Apps extension to your existing MCP server.

Send me the output. Then we'll modify your existing project rather than creating a new application.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> python -c "import inspect; from mcp.server.apps import Apps; from mcp.server import MCPServer; print('MCP SDK API:'); print('Apps.tool:', inspect.signature(Apps.tool)); print('Apps.add_html_resource:', inspect.signature(Apps.add_html_resource)); print('MCPServer:', inspect.signature(MCPServer))"
MCP SDK API:
Apps.tool: (self, *, resource_uri: 'str', visibility: 'Sequence[Visibility] | None' = None, meta: 'dict[str, Any] | None' = None, **tool_kwargs: 'Any') -> 'Callable[[_CallableT], _CallableT]'
Apps.add_html_resource: (self, uri: 'str', html: 'str', *, name: 'str | None' = None, title: 'str | None' = None, description: 'str | None' = None, csp: 'ResourceCsp | None' = None, permissions: 'ResourcePermissions | None' = None, domain: 'str | None' = None, prefers_border: 'bool | None' = None) -> 'None'
MCPServer: (name: 'str | None' = None, title: 'str | None' = None, description: 'str | None' = None, instructions: 'str | None' = None, website_url: 'str | None' = None, icons: 'list[Icon] | None' = None, version: 'str' = '', auth_server_provider: 'OAuthAuthorizationServerProvider[Any, Any, Any] | None' = None, token_verifier: 'TokenVerifier | None' = None, *, tools: 'list[Tool] | None' = None, resources: 'list[Resource] | None' = None, extensions: 'Sequence[Extension] | None' = None, debug: 'bool' = False, log_level: "Literal['DEBUG', 'INFO', 'WARNING', 'ERROR',

'CRITICAL']" = 'INFO', warn_on_duplicate_resources: 'bool' = True, warn_on_duplicate_tools: 'bool' = True, warn_on_duplicate_prompts: 'bool' = True, dependencies: 'list[str] | None' = None, lifespan: 'Callable[[MCPServer[LifespanResultT]], AbstractAsyncContextManager[LifespanResultT]] | None' = None, auth: 'AuthSettings | None' = None, resource_security: 'ResourceSecurity' = ResourceSecurity(reject_path_traversal=True, reject_absolute_paths=True, reject_null_bytes=True, exempt_params=frozenset()), request_state_security: 'RequestStateSecurity | None' = None, cache_hints: 'Mapping[CacheableMethod, CacheHint] | None' = None, subscriptions: 'SubscriptionBus | Literal[False] | None' = None, middleware: 'Sequence[ServerMiddleware[Any]] | None' = None)

Perfect! Your installed MCP SDK supports everything we need.
The important signatures are confirmed:

- `Apps()` creates the Apps extension.
- `Apps.tool()` registers a tool associated with an interface.
- `Apps.add_html_resource()` registers the HTML interface.
- `MCPServer(..., extensions=[apps])` enables MCP Apps in your server.

GitHub MCP Python SDK

Step 1 — Create the Google News App module

Inside your project, create this file:

text

```
src/
└── google_news_mcp/
    ├── apps/
    │   └── news_app.py
    ├── config.py
    ├── server.py
    └── ...
```

Create the `apps` directory and an empty `__init__.py` inside it.
Now create `src/google_news_mcp/apps/news_app.py` with:

python

```python
from mcp.server.apps import Apps


APP_URI = "ui://google-news/search.html"

apps = Apps()


NEWS_APP_HTML = """
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Google News Intelligence</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f8fafc;
            color: #1e293b;
            padding: 32px;
            margin: 0;
        }

        .container {
            max-width: 750px;
            margin: 40px auto;
        }

        h1 {
            font-size: 30px;
            margin-bottom: 8px;
        }

        .subtitle {
            color: #64748b;
            margin-bottom: 30px;
        }

        .result {
            background: white;
            border: 1px solid #e2e8f0;
            border-radius: 12px;
            padding: 24px;
            min-height: 100px;
        }
    </style>
</head>

<body>
    <div class="container">
        <h1>Google News Intelligence</h1>

        <p class="subtitle">
            Discover and analyse global news using Gemini and Google News.
```

```
        </p>

        <div class="result">
            <h3>Search result</h3>
            <p id="result">Waiting for your first search...</p>
        </div>
    </div>

    <script>
        const parent = window.parent;
        let initialized = false;

        window.addEventListener("message", (event) => {
            if (
                event.source !== parent ||
                event.data?.jsonrpc !== "2.0"
            ) {
                return;
            }

            if (
                event.data.id === 1 &&
                event.data.result?.protocolVersion
            ) {
                parent.postMessage({
                    jsonrpc: "2.0",
                    method: "ui/notifications/initialized"
                }, "*");

                initialized = true;
                return;
            }

            if (
                !initialized ||
                event.data.method !== "ui/notifications/tool-result"
            ) {
                return;
            }

            const content = event.data.params?.content || [];

            const resultText = content
                .filter(item => item.type === "text")
                .map(item => item.text)
                .join("\\n");

            document.getElementById("result").textContent =
```

```
                resultText || "No result received.";
        });

        parent.postMessage({
            jsonrpc: "2.0",
            id: 1,
            method: "ui/initialize",
            params: {
                appInfo: {
                    name: "Google News Intelligence",
                    version: "1.0.0"
                },
                appCapabilities: {},
                protocolVersion: "2026-01-26"
            }
        }, "*");
    </script>
</body>
</html>
"""


@apps.tool(
    resource_uri=APP_URI,
    name="open_google_news_app",
    title="Google News Intelligence",
    description="Open the interactive Google News interface.",
)
def open_google_news_app(
    query: str = "AI infrastructure",
) -> str:
    """Open the Google News interface with an initial search query."""

    return (
        f"Google News interface initialized. "
        f"Initial search query: {query}"
    )


apps.add_html_resource(
    APP_URI,
    NEWS_APP_HTML,
    name="Google News Intelligence",
    title="Google News Intelligence",
    description="Interactive Google News search interface.",
)
```

This is our first rendering test. The query is currently supplied through the Inspector's tool input form; we'll make the interface itself interactive in a later step.

Step 2 — Enable the extension in `server.py`

Add this import:

```python
from google_news_mcp.apps.news_app import apps
```

Find your existing line:

```python
mcp = MCPServer("Google News MCP")
```

Replace it with:

```python
mcp = MCPServer(
    "Google News MCP",
    extensions=[apps],
)
```

Leave your existing news tools, resources and prompts registration unchanged.

Step 3 — Test the server

Run:

```powershell
mcp dev src/google_news_mcp/server.py
```

Connect through Inspector and look for the **Apps** tab.

Expected first milestone

Google News Intelligence appears as an MCP App.

Don't connect Gemini to this app yet. First, let's verify that Inspector can render it successfully.

Send me the result after you run the server.

(.venv) PS C:\Users\Thiru\Desktop\projects\google_news_mcp\google-news-mcp> from google_news_mcp.apps.news_app import apps
At line:1 char:1

- from google_news_mcp.apps.news_app import apps
-

The 'from' keyword is not supported in this version of the language.
+ CategoryInfo : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : ReservedKeywordNotAllowed

You're trying to execute Python code directly in **PowerShell**. That's why you're getting this error.
The line:

```python
from google_news_mcp.apps.news_app import apps
```

is **Python code**, not a PowerShell command. It belongs inside your `server.py` file.

Step 1 — Open `server.py` in VS Code
Run this in PowerShell:

```powershell
code src/google_news_mcp/server.py
```

Step 2 — Add the import
At the top of `server.py`, alongside your other imports, add:

```python
from google_news_mcp.apps.news_app import apps
```

Step 3 — Update MCPServer initialization

Find:

```python
mcp = MCPServer("Google News MCP")
```

Replace it with:

```python
mcp = MCPServer(
    "Google News MCP",
    extensions=[apps],
)
```

Save the file using `Ctrl + S`.
Don't run the import line in PowerShell.
Once you've updated `server.py`, share its contents here. I'll check it before we start the Inspector again.

from mcp.server import MCPServer

from google_news_mcp.services.news_service import NewsService
from google_news_mcp.tools.news_tools import register_news_tools
from google_news_mcp.resources.news_resources import register_news_resources
from google_news_mcp.prompts.news_prompts import register_news_prompts
from google_news_mcp.apps.news_app import apps

mcp = MCPServer(
"Google News MCP",
extensions=[apps],
)

news_service = NewsService()

Register MCP tools

register_news_tools(mcp, news_service)
register_news_prompts(mcp)

Register MCP resources

register_news_resources(mcp)

if **name** == "**main**":
mcp.run()

Your `server.py` is correctly configured. ✅
You've successfully integrated the MCP Apps extension without disturbing your existing tools, prompts or resources.
Here's what your server now registers:

Component	Status
Google News search tools	Registered
News resources	Registered
News prompts	Registered
Google News MCP App	Extension enabled

The `extensions=[apps]` configuration is exactly how the official Python SDK enables MCP Apps. GitHub MCP Python SDK

Step 2 — Launch MCP Inspector

Save `server.py` and run:

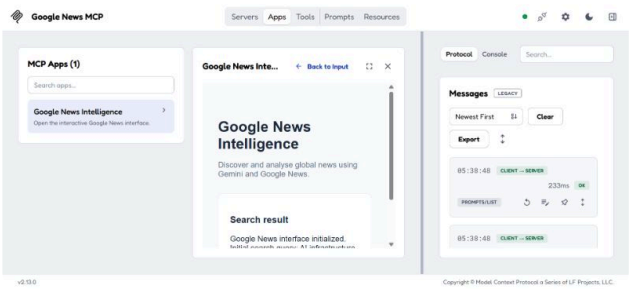
```powershell
mcp dev src/google_news_mcp/server.py
```

Connect to your server in Inspector.
You should now see an **Apps** tab alongside Tools, Resources and Prompts. Select it, choose **Google News Intelligence**, and click **Open App**. GitHub GitHub

Expected first UI

> **Google News Intelligence**
>
> Discover and analyse global news using Gemini and Google News.
>
> ---
>
> **Search result**
>
> Waiting for your first search...

At this stage, we're only testing whether the UI renders. The actual Gemini-powered search box comes next.
Run the command and tell me whether Google News Intelligence appears in the Apps tab.



MCP Apps rendering successfully

Excellent! Your screenshot confirms that:

- MCP Inspector discovered your Google News app.
- The `ui://` resource loaded successfully.
- The HTML interface rendered inside the Apps screen.
- The initial tool result was delivered to the interface.

We've completed the first UI milestone.

What we have versus what we're building

Right now, the interface is a static shell. The initial search query is just text returned by `open_google_news_app`; it hasn't triggered a real Gemini search yet.
Our next goal is to make this work:
User types a question → clicks Search → Gemini selects Google News tools → articles are retrieved → results appear as article cards.

MCP Apps supports this through the app's ability to call server tools using the host's `tools/call` bridge. GitHub @modelcontextprot...

Next step — Prepare the functional search

Before changing code, please share the current contents of these two files:

1. `src/google_news_mcp/apps/news_app.py`
2. `src/google_news_mcp/tools/news_tools.py`

We'll use your existing implementation rather than duplicate the news-search logic.
Our next milestone is a working search box connected to Gemini, not another static UI demonstration.

news_app.py 3.6 KB news_tools.py 3.3 KB

I've reviewed both files. Your existing implementation is a good starting point, and **we don't need to rewrite `news_tools.py`.**
Here's what we have:

- `news_app.py` registers the app, but its launch tool currently returns only an initialization message. Its HTML has no search input or server-tool calling functionality.

  交互式组件（FileCite）无法静态导出

- `news_tools.py` already exposes the three functions we need: `search_news`, `get_top_headlines` and `search_news_by_publisher`.

  交互式组件（FileCite）无法静态导出

Our next task is to let the HTML interface call those existing MCP tools.
One technical correction to our earlier approach: displaying a result through `postMessage` is enough for a basic rendering test, but real interactive calls should use the official `@modelcontextprotocol/ext-apps` JavaScript SDK and its `app.callServerTool()` method. MCP Python SDK GitHub
We'll use vanilla HTML, CSS and JavaScript inside your existing Python file. No React, SolidJS, or separate frontend application.

Step 1 — Replace `news_app.py`

Replace the entire contents of `src/google_news_mcp/apps/news_app.py` with this:

```python
from mcp.server.apps import Apps, ResourceCsp
```

```
APP_URI = "ui://google-news/search.html"

apps = Apps()


NEWS_APP_HTML = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Google News Intelligence</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f8fafc;
            color: #1e293b;
            padding: 24px;
            margin: 0;
        }

        .container {
            max-width: 750px;
            margin: 30px auto;
        }

        h1 {
            font-size: 30px;
            margin-bottom: 8px;
        }

        .subtitle {
            color: #64748b;
            margin-bottom: 28px;
        }

        form {
            display: flex;
            gap: 10px;
            margin-bottom: 16px;
        }

        input {
            flex: 1;
            min-width: 0;
            padding: 14px;
```

```
            border: 1px solid #cbd5e1;
            border-radius: 9px;
            font-size: 15px;
        }

        button {
            padding: 14px 22px;
            border: none;
            border-radius: 9px;
            background: #2563eb;
            color: white;
            font-weight: bold;
            cursor: pointer;
        }

        button:disabled {
            opacity: 0.6;
            cursor: not-allowed;
        }

        #status {
            color: #64748b;
            font-size: 14px;
            margin: 16px 0;
        }

        .article {
            background: white;
            border: 1px solid #e2e8f0;
            border-radius: 12px;
            padding: 18px;
            margin-top: 14px;
        }

        .article h3 {
            margin-top: 0;
        }

        .article a {
            color: #2563eb;
            text-decoration: none;
        }

        .article-meta {
            color: #64748b;
            font-size: 13px;
        }
```

```
        @media (max-width: 500px) {
            form {
                flex-direction: column;
            }
        }
    </style>
</head>

<body>

<div class="container">

    <h1>Google News Intelligence</h1>

    <p class="subtitle">
        Discover and analyse global news using Gemini and Google News.
    </p>

    <form id="search-form">
        <input
            id="query"
            type="text"
            placeholder="Ask something about the news..."
            value="AI infrastructure"
            minlength="2"
            maxlength="200"
            required
        />

        <button id="search-button" type="submit" disabled>
            Search
        </button>
    </form>

    <p id="status">Connecting to MCP server...</p>

    <section id="results" aria-live="polite"></section>

</div>

<script type="module">

    import { App } from
        "https://esm.sh/@modelcontextprotocol/ext-apps@2.0.4";

    const app = new App(
        {
            name: "Google News Intelligence",
```

```
            version: "1.0.0"
        },
        {}
    );

    const form = document.getElementById("search-form");
    const queryInput = document.getElementById("query");
    const searchButton = document.getElementById("search-button");
    const status = document.getElementById("status");
    const results = document.getElementById("results");

    function renderArticles(result) {

        results.replaceChildren();

        let payload = result.structuredContent;

        if (!payload) {
            const text = (result.content || [])
                .filter(item => item.type === "text")
                .map(item => item.text)
                .join("\\n");

            try {
                payload = JSON.parse(text);
            } catch {
                status.textContent = "Search completed.";
                results.textContent = text || "No results returned.";
                return;
            }
        }

        const articles = payload?.articles || [];

        status.textContent =
            `Found ${payload.count ?? articles.length} articles.`;

        if (!articles.length) {
            results.textContent = "No matching articles found.";
            return;
        }

        for (const article of articles) {

            const card = document.createElement("article");
            card.className = "article";

            const heading = document.createElement("h3");
```

```
            const link = document.createElement("a");

            link.textContent = article.title || "Untitled article";

            try {
                const url = new URL(article.url);

                if (
                    url.protocol === "https:" ||
                    url.protocol === "http:"
                ) {
                    link.href = url.href;
                    link.target = "_blank";
                    link.rel = "noopener noreferrer";
                }
            } catch {
                // Display the headline even if its URL is invalid.
            }

            heading.appendChild(link);

            const metadata = document.createElement("p");
            metadata.className = "article-meta";

            metadata.textContent = [
                article.publisher,
                article.published_at
            ].filter(Boolean).join(" · ");

            card.appendChild(heading);
            card.appendChild(metadata);

            results.appendChild(card);
        }
    }

    // Receive the initial query provided when the app is opened.
    app.ontoolinput = (params) => {
        const initialQuery = params.arguments?.query;

        if (typeof initialQuery === "string") {
            queryInput.value = initialQuery;
        }
    };

    async function initialize() {

        try {
```

```
            await app.connect();

            status.textContent = "Connected. Ready to search.";

            searchButton.disabled = false;

        } catch (error) {

            status.textContent =
                `Connection failed: ${error.message}`;

        }
    }

    form.addEventListener("submit", async (event) => {

        event.preventDefault();

        const query = queryInput.value.trim();

        if (query.length < 2) {
            status.textContent = "Enter at least two characters.";
            return;
        }

        searchButton.disabled = true;
        status.textContent = "Searching Google News...";
        results.replaceChildren();

        try {

            const response = await app.callServerTool({
                name: "search_news",
                arguments: {
                    query: query,
                    region: "IN",
                    language: "en",
                    max_results: 10
                }
            });

            if (response.isError) {
                throw new Error("Google News search failed.");
            }

            renderArticles(response);

        } catch (error) {
```

```
            status.textContent = `Search failed: ${error.message}`;
        } finally {
            searchButton.disabled = false;
        }
    });
    initialize();
</script>
</body>
</html>
"""


@apps.tool(
    resource_uri=APP_URI,
    name="open_google_news_app",
    title="Google News Intelligence",
    description="Open the interactive Google News interface.",
)
def open_google_news_app(
    query: str = "AI infrastructure",
) -> str:
    """Open the Google News interface."""
    return f"Google News interface opened for: {query}"


apps.add_html_resource(
    APP_URI,
    NEWS_APP_HTML,
    name="Google News Intelligence",
    title="Google News Intelligence",
    description="Interactive Google News search interface.",
    csp=ResourceCsp(
        resource_domains=["https://esm.sh"],
    ),
)
```

We're loading the official JavaScript SDK from esm.sh, pinned to version 2.0.4. The resource CSP explicitly permits that dependency. npm MCP Python SDK

Step 2 — Run and test

You don't need to modify `server.py` or `news_tools.py` for this test.
Run:

```powershell
mcp dev src/google_news_mcp/server.py
```

Open Inspector → Apps → Google News Intelligence.
Try searching for:
`Latest AI infrastructure news`

Expected result

- Search box accepts the question.
- Existing `search_news` MCP tool is invoked.
- Article headlines, publishers, dates and links appear.

Important: This is an intermediate test using direct Google News search. Gemini orchestration is our next integration step, where we'll replace the direct `search_news` call with a Gemini-powered MCP tool that can choose between all three existing news tools.
Let me know what happens when you submit your first search.

Now what is the exact process I need to do now ?

You need to make **one file change, restart MCP Inspector, and test the search interface**. Follow these steps in order.

Step 1 — Open the project in VS Code

Open your existing project:

```powershell
code .
```

In the VS Code Explorer, navigate to:

```text
google-news-mcp/
└── src/
    └── google_news_mcp/
        └── apps/
            ├── __init__.py
            └── news_app.py
```

Step 2 — Replace `news_app.py`

1. Open `news_app.py` .

2. Select all existing contents (`Ctrl + A`).
3. Paste the complete new Python code I provided in the previous message.
4. Save using `Ctrl + S` .

Do not modify these files:

- `server.py`
- `tools/news_tools.py`
- `services/news_service.py`

We've already registered the Apps extension in your server.

Step 3 — Restart MCP Inspector

Go to your PowerShell terminal.
If the previous Inspector process is still running, stop it using:

```powershell
Ctrl + C
```

Then launch it again:

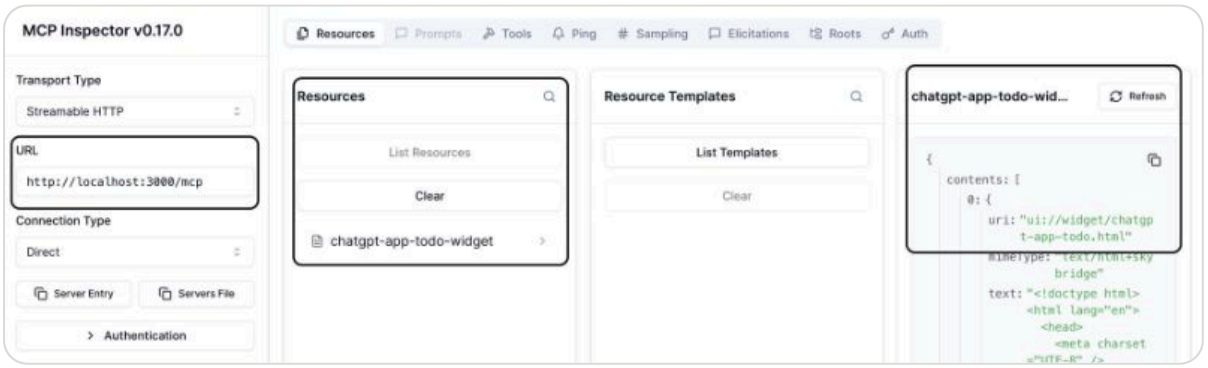
```powershell
mcp dev src/google_news_mcp/server.py
```

Wait until Inspector opens and connects to your Google News MCP server.

Step 4 — Open your application

Inside MCP Inspector:

1. Select **Apps**.
2. Select **Google News Intelligence**.
3. Open the app.
4. You should now see a search input and a Search button.



Step 5 — Test your first search

Enter:

```text
Latest AI infrastructure news
```

Click **Search**.

Expected behaviour

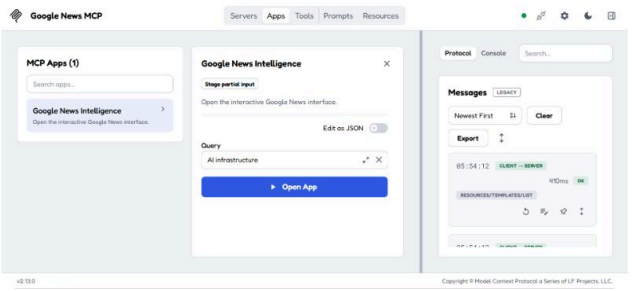
The app should call your existing `search_news` MCP tool and display article headlines, publishers, publication dates and links.

Important: What are we testing at this stage?

Our implementation is being built in two stages:

Stage	Functionality
Current test	UI → MCP `search_news` → Articles displayed
Next milestone	UI → Gemini → Gemini selects MCP tools → Articles + AI briefing displayed

So don't worry if Gemini isn't involved in this search yet. That's intentional.
Complete these five steps and send me a screenshot of your new interface and the search results. We'll then connect Gemini to the UI.

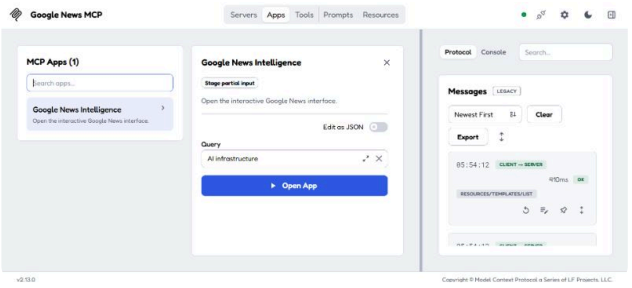


Now Do I need to select Open App button ?

Yes! Click the blue **Open App** button.
This will launch your Google News Intelligence interface inside MCP Inspector.
Once it opens:

1. Wait for the interface to load.
2. You should see the search box and Search button.
3. Enter `Latest AI infrastructure news`.
4. Click **Search**.

The expected result is a list of Google News articles with headlines, publishers, publication dates and links.
Send me a screenshot after clicking Open App. We'll verify the interface before connecting Gemini.



In which search bar I need to input the `AI infrastructure news`

Enter **`AI infrastructure news`** in the **Query** field in the middle panel.

Use this **Middle panel → Query**

Replace the existing value `AI infrastructure` with:

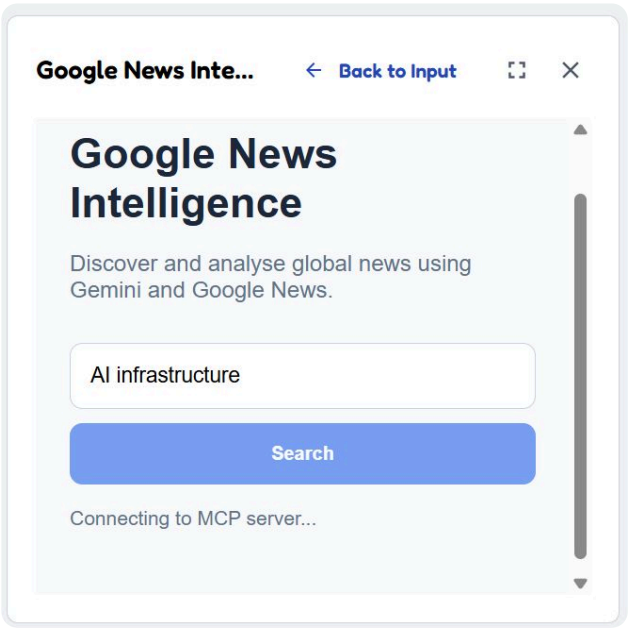
> AI infrastructure news

Then click **Open App**.

Don't use **Search apps** (top-left)

That field only filters the list of available MCP Apps.

Once the app opens, use the search box inside Google News Intelligence to perform your actual news searches.



After clicking that buttonit opens like this

The interface has opened successfully, but **the JavaScript connection to the MCP Inspector host hasn't completed yet.**

I can see two clues in your screenshot:

- The search box is visible.
- The status still says `Connecting to MCP server...`, and the Search button is disabled.

This means we should diagnose the frontend connection before attempting a search.

Step 1 — Open the Inspector Console

Look at the right-hand panel in MCP Inspector. At the top, you should see:
Protocol | Console

1. Click **Console**.
2. Check whether any errors appear.
3. Share a screenshot of the Console.

If that Console doesn't show JavaScript errors, open Chrome Developer Tools with `F12`, select the Console tab, and check there instead.

The likely area to investigate is the external JavaScript SDK import or its initialization. The official MCP Apps SDK uses `app.connect()` to establish the connection with the host.

@modelcontextprot... MCP Apps

Don't change any code yet. Let's identify the actual error first rather than guessing at another fix.

Powered by **XWX Exporter**